

SECURED ENGINEERS PVT. LTD.

SEPL ERP

Complete System Documentation — User Manual, Management
Overview & Technical Reference

Scope: All modules · all screens · all automations

Platform: Web application + installable PWA (phone / laptop / desktop)

Stack: React · Node.js / Express · SQLite

Generated: 2026-06-16

Table of Contents

- How to read this document
- About the SEPL ERP
- End-to-end business flow
- Complete module map
- Who uses what (typical roles)
- Key system-wide concepts

1. Getting Started

- 1.1 Logging In
- 1.2 Roles & Permissions — the Access Model
- 1.3 Users Administration
- 1.4 Roles & Permissions Administration
- 1.5 Navigation
- 1.6 Mobile / PWA Behavior
- 1.7 Technical Reference

2. CRM

- 2.1 Partners (Influencers / Referral Partners)
- 2.2 CRM Sales Funnel
- 2.3 Sales Funnel (Leads)
- 2.4 Business Book
- 2.5 Customers
- 2.6 Full Kitting (CRM Full Kitting)
- Cross-module relationships (summary)

3. Quotes & Orders

- 3.1 Quotations (BOQ & Quotations)
- 3.2 AI Auto-Quotation (Estimator)
- 3.3 PO/FOC Stripped
- 3.4 Labour Rate
- 3.5 Extra-indent Auto-Quotation (SEPL format)
- 3.6 Relationship: Quotation → Business Book order

4. Procurement

- 4.1 Items — Item Master
- 4.2 RFQ Queue — Price Required
- 4.3 Vendors
- 4.4 Indent → Dispatch (the purchasing engine)
- 4.5 Order to Planning
- 4.6 Schedule (Gantt) — Procurement Schedule
- 4.7 Automation status (cross-reference)

5. Projects

- 5.1 Indent Labour Payment (Project Execution & Billing)
- 5.2 Daily Reports (DPR)
- 5.3 Snags
- 5.4 Fire NOC Renewal
- 5.5 Sales Billing

6. Finance

- 6.1 Cheques (Cheque FMS)
- 6.2 Payables — Payment Required
- 6.3 Collections (Receivables)
- 6.4 Invoices / Billing
- 6.5 Cash Flow
- 6.6 Expenses
- 6.7 How the Finance modules interconnect

7. HRMS (Human Resources)

- 7.1 Hiring — Applicant Tracking System (ATS)
- 7.2 Public offer-accept (no-login) & print outputs
- 7.3 Sub-contractor Hiring (14-step / 2-phase tracker)
- 7.4 Onboarding / Induction
- 7.5 Training (LMS)
- 7.6 Attendance
- 7.7 Payroll
- 7.8 Employees (master)
- 7.9 Performance — MIS Scorecards
- 7.10 Sub-contractor Master
- 7.11 Permissions & cross-module notes

8. Inventory & Assets

- 8.1 Tool Rentals
- 8.2 Company Assets
- 8.3 Inventory
- 8.4 Tools
- 8.5 Room Rentals
- 8.6 Cross-module summary

9. Tasks & Service Desk

- 9.1 Delegations
- 9.2 PMS Tasks
- 9.3 Checklists
- 9.4 Complaints
- 9.5 Help Tickets
- 9.6 Cross-module summary

10. Dashboards & Executive Views

- 10.1 Overview — the dashboard landscape
- 10.2 Home Dashboard
- 10.3 The executive data layer
- 10.4 War Room (Director's War Room)
- 10.5 Operating Console — CMD Stage 1
- 10.6 TOC View — CMD Stage 2
- 10.7 CMD Audit feed (/audit)
- 10.8 Daily scheduled jobs
- 10.9 Summary

11. Administration & Settings

- 11.1 Activity Log (Daily Activity / Word Count)
- 11.2 Location Tracking
- 11.3 Database Backups
- 11.4 AI Agent ("Ask ERP" assistant)
- 11.5 Email (SMTP) Settings
- 11.6 Email Triggers (rule-based automated emails)
- 11.7 Audit Log
- 11.8 Users, Roles & Permissions (cross-reference)

12. Automations, Scheduled Jobs & Integrations

- 12.1 How scheduling works (no node-cron)
- 12.2 Master table — every scheduled job
- 12.3 The `server/scripts/` directory — full inventory
- 12.4 Integrations
- 12.5 Notifications model
- 12.6 Where credentials live — quick reference

13. Technical Reference

- 13.1 High-Level Architecture
- 13.2 Technology Stack

13.3 Repository Layout

13.4 Data Model

13.5 Build & Run

13.6 Deployment & Operations

13.7 Front-End Application Notes

13.8 Security Notes

Secured Engineers Pvt. Ltd. (SEPL) — Enterprise Resource Planning System

Product	SEPL ERP (internal name: <code>business-erp</code>)
Owner	Secured Engineers Pvt. Ltd.
Document type	Complete reference — User Manual + Management Overview + Technical Reference
Scope	All modules, all screens, all automations
Platform	Web application (browser) + installable PWA (phone / laptop / desktop)
Technology	React + Vite front-end · Node.js / Express API · SQLite (better-sqlite3) database

How to read this document

This manual is written for **three audiences at once**, so each module is described from three angles:

1. **End users** (CRM staff, accountants, site engineers, HR, store keepers) — *what the screen does and the step-by-step way to use it.*
2. **Management / MD** (for demos and review) — *what the module is for, the workflow it enforces, and the value it delivers.*
3. **Developers / administrators** — *the API endpoints, database tables, automations and formulas behind each screen.*

You can read it front-to-back, or jump straight to the module you need using the **Table of Contents**. Every chapter is self-contained.

- **Sections 1–11** follow the application's own left-hand menu, group by group.
- **Section 12** documents every background automation, scheduled job and outside integration (email, WhatsApp, push, AI).
- **Section 13** is the technical reference (architecture, data model, deployment).

About the SEPL ERP

The SEPL ERP is a **single, integrated business system** that runs the whole company — from the first sales enquiry to the final payment collected, and everything in between: buying material, executing projects on site, paying staff and vendors, tracking tools and stock, and giving management a live picture of the business.

It replaces a scattered set of spreadsheets and WhatsApp messages with **one source of truth**, shared by every department, with role-based access so each person sees only what they should.

The system is purpose-built for SEPL's line of work — **fire-fighting, MEP and allied engineering contracts** — so it understands the things a generic ERP does not: BOQs (Bills of Quantity), SITC (Supply, Installation, Testing & Commissioning) line items, Fire NOC renewals, site DPRs (Daily Progress Reports), labour-rate costing, and milestone-based client billing.

The business in one line

Lead → Quotation → Order (Business Book) → Procurement (Indent → Dispatch) → Project Execution (DPR) → Sales Billing → Collections

Around that core sales-to-cash spine sit the supporting functions: **HR & Payroll, Finance, Inventory & Assets, Tasks, Service Desk**, and **Executive Dashboards**.

End-to-end business flow

The modules are not islands — data flows from one to the next automatically. The typical journey of a job through the ERP:

Stage	Module(s)	What happens
1. Find the work	CRM → Partners, CRM Sales Funnel, Sales Funnel (Leads)	A referral partner or enquiry creates a lead; it is qualified through a staged pipeline.
2. Price the work	Quotes & Orders → Quotations / AI Auto-Quotation	A BOQ is priced (material + labour + margin) into a client quotation.
3. Win & book the order	CRM → Business Book	The won order is recorded with client, PO, value, GST and project details. It becomes the anchor every later module links to.
4. Plan & buy material	Procurement → Items, RFQ Queue, Vendors, Indent → Dispatch , Schedule	Indents are raised, approved (L1/L2/CRM), quoted by vendors, turned into POs, received, and dispatched to site.
5. Execute on site	Projects → DPR, Snags, Indent Labour Payment, Fire NOC	Daily progress, manpower, labour cost and quality snags are tracked against the project.
6. Bill the client	Projects → Sales Billing · Finance → Invoices	Sequential bills (Order → Material → Installation → Final) are raised with correct GST.
7. Collect the money	Finance → Collections, Cash Flow	Receivables are aged and chased; cash position and runway are tracked.
8. Pay vendors & staff	Finance → Payables, Cheques · HRMS → Payroll	Vendor payments run through a multi-level approval; staff are paid via attendance-driven payroll.
Throughout	HRMS, Inventory, Tasks, Service Desk, Dashboards, Admin	Hiring, attendance, tools/stock, delegated tasks, complaints and the executive view run continuously.

Complete module map

The application's left sidebar organises **all modules into ten groups plus a Settings group**. This is the same structure used by the chapters that follow.

Group	Modules (menu items)	Documented in
(top)	Dashboard (home)	\$10
CRM	Partners · CRM Sales Funnel · Sales Funnel · Business Book · Customers · Full Kitting	\$2
Quotes & Orders	Quotations · AI Auto-Quotation · PO/FOC Stripped · Labour Rate	\$3
Procurement	Items · RFQ Queue · Vendors · Indent to Dispatch · Order to Planning · Schedule (Gantt)	\$4
Projects	Indent Labour Payment · Daily Reports (DPR) · Snags · Fire NOC Renewal · Sales Billing	\$5
Finance	Cheques · Payables · Collections · Invoices · Cash Flow · Expenses	\$6
HRMS	Hiring · Sub-contractor Hiring · Onboarding · Training · Attendance · Payroll · Employees · Performance · Sub-contractor Master	\$7
Inventory	Tool Rentals · Assets · Inventory · Tools · Room Rentals	\$8
Tasks	Delegations · PMS Tasks · Checklists	\$9
Service Desk	Complaints · Help Tickets	\$9
Executive (<i>admin</i>)	War Room · Operating Console · TOC View	\$10
Admin / Settings (<i>admin</i>)	Activity Log · Location · Backups · AI · Email · Email Triggers · Users · Roles & Permissions · Audit Log	\$11

Visibility rule: a module is hidden from a user until their role is granted permission to it (admin sees everything). A handful of items are open to all staff (Help Tickets, Onboarding, Training, RFQ Queue). See \$1 for the permission model.

Who uses what (typical roles)

Role	Primary modules
CRM / Sales	Partners, Sales Funnel, CRM Sales Funnel, Business Book, Quotations, Collections
Estimation	AI Auto-Quotation, Labour Rate, PO/FOC Stripped, Item Master
Procurement / Stores	Indent to Dispatch, RFQ Queue, Vendors, Items, Inventory, Tools, Schedule
Site Engineer	DPR, Snags, Indent Labour Payment, Attendance, Full Kitting
Accounts / Finance	Payables, Cheques, Collections, Invoices, Cash Flow, Expenses
HR	Hiring, Sub-contractor Hiring, Onboarding, Training, Attendance, Payroll, Employees, Performance
Management / MD	Executive dashboards, all approvals (L3 / release), Audit Log
Administrator	Users, Roles & Permissions, Backups, AI, Email, Locations

Key system-wide concepts

A few ideas appear across many modules — understanding them once makes the rest of the manual easier:

- **Business Book order = the anchor.** Almost every downstream document (PO, DPR site, sales bill, receivable, cash-flow entry) links back to a Business Book order.
- **Multi-level approvals.** Money-out flows (vendor payments, indents) pass through named approval levels — e.g. **L1 Accountant** → **L2 Nitin Jain** → **L3 MD (Ankur Kaplesh)** → **Release (Aanchal)** for Payables.
- **GST is computed, not typed.** Local/Punjab clients get **CGST + SGST**; others get **IGST** — the split is derived from the client's state, with the state code auto-filled.
- **Everything is audited.** Every create/update/delete is written to an audit log with the user, time and change (see §11).
- **Automations run in the background.** Nightly backups, daily DPR prompts, the 09:00 CMD email, fortnightly installation billing, Fire NOC auto-advance, and more run on schedule without anyone clicking a button (see §12).
- **Works on phones.** The ERP installs as a PWA; site staff punch attendance and the app pings GPS every 30 seconds for geofenced attendance and location tracking.

The chapters that follow document each module in full. Section numbers match the Table of Contents.

1. Getting Started

This section covers how to sign in to the SEPL ERP, how administrators create and manage user accounts, how the role-and-permission model controls what each user can see and do, and how to find your way around the application. It is written for three audiences at once: the everyday end user who just needs to log in, the manager who wants to understand who can access what, and the developer who needs the technical reference (endpoints, JWT, database tables).

The product is a Node/Express + React single-page application for **Secured Engineers Pvt Ltd (SEPL)**. The backend stores everything in a SQLite database; the frontend is a React app that talks to the API under `/api`.

1.1 Logging In

The login screen

The login page is the first thing every user sees when they are not authenticated. It is a two-panel layout: a sign-in form on the left and an SEPL brand panel ("Build secure. Track smart.") on the right. The SEPL logo is served from `/sepl-logo.webp`, with an inline SVG shield fallback if that image is ever missing, so the page never shows a broken image.

How to sign in

1. Open the ERP URL in a browser (or the installed PWA — see [1.6](#)).
2. In **Email / Username**, type either your email address **or** your username. The system matches against both, case-insensitively.
3. Type your **Password**. Use the eye icon to reveal/hide what you typed.
4. Optionally tick **Remember me** — this only saves your *username* on this device (in `localStorage` under `sepl_remember_identifier`); it does **not** save your password or keep you logged in.
5. Click **Sign in**. On success you get a "Welcome back, *name*!" toast and land on the Dashboard.

If you don't have credentials, the page tells you to **contact your admin**. There is no public self-registration — only an admin can create accounts (see [1.3](#)).

What happens on a successful login (session model)

On a correct username/email + password, the backend issues a **JWT (JSON Web Token)** and returns it together with your user profile, your effective permissions, and your assigned role names. The frontend stores the token in `localStorage` under the key `token` and attaches it as a `Bearer` token on every subsequent API request.

Login outcome	What the user sees
Correct credentials	Logged in, redirected to Dashboard
Wrong password / unknown user	"Invalid credentials" (deliberately generic)
Account disabled (<code>active = 0</code>)	"Your account is disabled. Please contact admin."
Missing username or password	"Username/email and password required"

Every login attempt — success or failure — is written to the **audit log** with the IP address and user-agent, so admins can spot brute-force patterns and review session history. Disabled-account and bad-password attempts are logged distinctly.

Staying logged in (sliding session)

The JWT has a base lifetime of **7 days**. The session is a **sliding** session: while you are actively using the app, the middleware silently hands back a fresh token (via the `X-Refresh-Token` response header) once the current token is more than a day old, and the client swaps it in. The practical effect is that an active user is never kicked out mid-work; only a session left completely idle for the full 7-day lifetime expires.

Change password (self-service)

Any logged-in user can change their own password from the sidebar footer:

1. Click your name area at the bottom of the sidebar, then **Change Password**.
2. Enter **Current Password**, then **New Password** and **Confirm New Password**.
3. The new password must be **at least 4 characters** and the two new-password fields must match.
4. Click **Change Password**.

The server verifies the current password before accepting the new one. Passwords are stored only as **bcrypt hashes** — the plain text is never kept and cannot be recovered.

Forgot password (recovery code)

The system supports self-service password reset without involving an admin, using a personal **recovery code** that each user sets for themselves:

- A user sets a memorable recovery code (any phrase, min 4 characters). It is stored as a bcrypt hash, so even database access cannot reveal it.
- On the forgot-password flow, the user supplies their **username**, their **recovery code**, and a **new password**. If the code matches, the password is reset — and the account is also re-activated if it had been disabled.
- There is also an **owner-only emergency code** (stored hashed in `app_settings` under `emergency_reset_hash`) that works for *any* user, so management can unlock an employee who never set a personal code.

Forgot-password failures use deliberately vague error text ("Username or recovery code is incorrect, or no recovery code is set") so the system never reveals which usernames exist.

1.2 Roles & Permissions — the Access Model

Access in the ERP is governed by two independent concepts that work together:

1. **System role** — a single coarse tier on the user record: `user`, `manager`, or `admin`.
2. **Permission roles** — one or more named roles (e.g. "Site Engineer", "Accountant") assigned to the user. Each permission role carries a grid of per-module permissions.

Admin bypass

A user whose **system role is admin** always has full access to everything — every module, every action — regardless of permission roles. This bypass is enforced in three places so it is consistent end-to-end:

- the backend permission middleware (`requirePermission`),
- the `getUserPermissions()` helper, which returns a complete all-modules grant for admins,
- the frontend `can()` helper in `AuthContext`.

The permission grid

For every (role × module) pair the system stores six boolean flags:

Flag	Meaning
<code>can_view</code>	See the module at all (gates the sidebar link and read APIs)
<code>can_create</code>	Add new records
<code>can_edit</code>	Modify existing records
<code>can_delete</code>	Remove records
<code>can_approve</code>	Approve records (for workflows with sign-off)
<code>can_see_all</code>	Scope override: see every record in the module, not just the ones the user raised/owns

`can_see_all` is decoupled from `can_approve`. It is meant for auditor-style roles that need full read visibility across all records but no approval power. When it is OFF (the default), a user only sees records they own; when ON, they see everything in that module.

When a user has **multiple permission roles**, the effective permission for each flag is the **highest privilege** across all their roles (logical OR per flag).

How a module stays hidden until granted

A key behavior (added after "when I create new module it shows to everyone"): **a module is hidden by default until access is explicitly granted**. In the sidebar, an item is visible only if:

- it is explicitly flagged `open: true` — the handful of features open to all staff (Help Tickets, Onboarding, Training, RFQ Queue), **or**
- the user's role can view that item's `module` key (admins pass everything).

An item with no `open` flag and no/unknown module key is hidden for non-admins. So forgetting to wire up a permission key fails *safe* — it hides the feature rather than leaking it.

Live permission refresh

Permissions used to be read only at login, which meant a grant didn't take effect until the user logged out and back in ("if I give permission, not working proper"). Now the frontend re-pulls the user's permissions from `/auth/me` whenever the browser tab regains focus and every 2 minutes while active (debounced). The backend always enforces permissions live, so a freshly granted module appears without a re-login.

Module keys (reference)

These are the module keys used by the `role_permissions` table and shown in the Roles & Permissions matrix. Renaming a label in the UI (e.g. "Procurement" → "Indent to Dispatch") does not change the underlying key. Grouped here the way the admin matrix groups them:

Group	Module keys
Finance & Daily Ops	<code>dashboard</code> , <code>cashflow</code> , <code>cheques</code> , <code>payment_required</code> , <code>attendance</code> , <code>collections</code> , <code>dpr</code> , <code>delegations</code> , <code>pms_tasks</code> , <code>checklists</code>
Sales & CRM	<code>leads</code> , <code>crm_funnel</code> , <code>fire_noc</code> , <code>rental_tools</code> , <code>influencers</code> , <code>crm_kitting</code> , <code>quotations</code> , <code>business_book</code>
Materials / Vendors / Procurement	<code>item_master</code> , <code>orders</code> , <code>vendors</code> , <code>sub_contractors</code> , <code>customers</code> , <code>procurement</code> , <code>procurement_schedule</code> , <code>indent_fms</code> , <code>inventory</code>
Execution / Site	<code>installation</code> , <code>billing</code> , <code>complaints</code> , <code>snags</code> , <code>company_assets</code> , <code>help_tickets</code>
HR / People	<code>hr_system</code> , <code>subcon_hiring</code> , <code>hr</code> , <code>payroll</code> , <code>scoring</code> , <code>tools</code> , <code>rentals</code> , <code>employees</code> , <code>expenses</code>
Platform	<code>ai_agent</code> , <code>users</code>

Note: the all-modules grant the backend hands to admins uses a slightly different internal list; the table above reflects the admin-facing **Roles & Permissions** matrix, which is the authoritative list of what can be granted/revoked.

Indent approval roles (separate from permissions)

In addition to permission roles, a user can be tagged with an **indent approval role** that gates the multi-level indent (procurement) sign-off, set on the user record as `approval_role`:

<code>approval_role</code>	Meaning
<code>l1</code>	L1 Approver — first sign-off
<code>l2</code>	L2 Approver — final sign-off
<code>hr</code>	HR Approver — single sign-off for RGP indents
<code>(none)</code>	Ordinary user, cannot approve indents

Admins always pass either gate as a safety net. This tag is independent of the `can_approve` permission flag.

1.3 Users Administration

User management lives at **Settings** → **Users** (`/admin/users`) and is **admin-only**. The page shows summary cards (Total / Active / Inactive / Admins) and a table of every user (including inactive ex-employees, which stay visible to admins so their historical records remain intact, but are excluded from assignment pickers elsewhere).

The table shows: Name, Username, Email, Phone, System Role (with L1/L2/HR badges if tagged), Assigned permission Roles, Department, and Status (Active/Inactive).

Create a user

1. Click **Add User**.
2. Fill in **Full Name** (required) and **Email** (required).
3. Optionally set a **Username** (spaces are auto-converted to dots, e.g. `Monika.devi`). Leave blank to log in by email only.
4. Set **Phone**, **Department** (optional).
5. Set a **Password** (required for new users).
6. Choose the **System Role**: User / Manager / Admin.

7. Optionally choose an **Indent Approval Role** (L1 / L2 / HR).
8. Under **Assign Permission Roles**, tick the permission roles this user should have.
9. Click **Create User**.

Edit a user

Open a user with the edit (pencil) icon. You can change every field above. The password field reads "New Password (leave blank to keep)" — leaving it empty preserves the current password. Role assignments are replaced with whatever is ticked at save time.

Reset a password (admin)

Because stored passwords are bcrypt-hashed and cannot be read back, an admin **sets a new one** rather than viewing the old:

1. Click the key icon on a user row.
2. Either type a custom new password, or leave it blank to auto-generate a 10-character mixed-case + digit password (ambiguous characters like 0/O and 1/l are excluded).
3. Click **Reset & Show Once**. The new password is displayed **once** with a copy button — share it with the user through a secure channel; it is not shown again.

Custom admin-set passwords must be at least 3 characters (relaxed so short office defaults work); the reset modal recommends 6+.

Activate / deactivate

The user/user-check icon toggles a user's `active` flag. **Deactivating is the recommended, reversible way to stop someone logging in** — their historical records stay linked. A deactivated user gets the "Your account is disabled" message at login.

Delete a user

The trash icon hard-deletes a user. This is guarded:

- It is a **two-step confirmation** — a dialog, then you must type `DELETE <name>` exactly.
- You cannot delete **your own** account, and you cannot delete the **last active admin**.
- If the user is referenced elsewhere (e.g. as `created_by` on indents, candidates, payment approvals), a plain delete is blocked. The UI then offers **Force Delete** (`?force=1`), which nulls every foreign-key reference pointing at the user across all tables, then deletes the row. Denormalized audit-trail snapshots (the saved `user_name` fields) stay intact, so old rows keep working — they just lose the "created by" link.

Location-tracking opt-out

A map-pin / eye-off toggle per user controls whether they appear in **Admin** → **Location** tracking. This flips the user's `track_location` flag; office staff or anyone management doesn't want on the live map can be hidden here. (For how the GPS pings are produced, see [1.5](#).)

Bulk import users

Bulk Import accepts a CSV with columns **Name, Email, Phone, Department, Role Name** (a downloadable template is provided). Imported users get the default password `sep1@123` unless specified, and are matched to an existing permission role by name. A preview table shows the rows before you confirm the import; duplicates (by email) are skipped.

1.4 Roles & Permissions Administration

Role administration lives at **Settings** → **Roles & Permissions** (`/admin/roles`), also admin-only. The screen is a two-pane layout: the list of roles on the left, and the permission matrix for the selected role on the right.

Create / edit / delete a role

- **Create**: click the + in the Roles panel header, give it a **Name** (required) and an optional **Description**.
- **Edit**: hover a role and click the pencil icon.
- **Delete**: hover a role and click the trash icon. **System roles (`is_system`) cannot be deleted**. Deleting a role removes its permissions; any user holding it loses those permissions.

Edit a role's permissions

1. Click a role in the left panel — the matrix loads on the right.
2. The matrix lists every module as a row and the six permission flags as columns: **View, Create, Edit, Delete, Approve, See All**.
3. Click any cell to toggle it. Helpful rules built into the grid:

- Enabling any action (Create/Edit/etc.) **auto-enables View** for that module.
- Disabling **View** clears all the other flags for that module.
- Clicking a **column header** toggles that action for **all** modules at once.

4. Click **Save Permissions**. Changes take effect live for affected users (no re-login needed — see live refresh in [1.2](#)).

1.5 Navigation

Once logged in, every page renders inside a common **Layout**: a left sidebar, a top header, the page content, and two floating helpers (a Help Ticket button and the AI Agent chat).

Sidebar structure

- **Dashboard** sits standalone at the very top (single link).
- Everything else is organized into **collapsible accordion groups**. Groups are independently expandable (more than one can be open at once); the open/closed set is remembered per browser via `localStorage` (`sidebar_open_groups`). Landing on a deep route (e.g. `/cashFlow`) auto-expands the group that contains it.
- **Settings** is pinned to the bottom of the nav.

The groups (and a sample of what lives under each):

Group	Examples of items
CRM	Partners, CRM Sales Funnel, Sales Funnel, Business Book, Customers, Full Kitting
Quotes & Orders	Quotations, AI Auto-Quotation, PO/FOC Stripped, Labour Rate
Procurement	Items, RFQ Queue, Vendors, Indent to Dispatch, Order to Planning, Schedule (Gantt)
Projects	Indent Labour Payment, Daily Reports (DPR), Snags, Fire NOC Renewal, Sales Billing
Finance	Cheques, Payables, Collections, Invoices, Cash Flow, Expenses
HRMS	Hiring, Sub-contractor Hiring, Onboarding, Training, Attendance, Payroll, Employees, Performance, Sub-contractor Master
Inventory	Tool Rentals, Assets, Inventory, Tools, Room Rentals
Tasks	Delegations, PMS Tasks, Checklists
Service Desk	Complaints, Help Tickets
Executive (<i>admin only</i>)	War Room, Operating Console, TOC View
Admin (<i>admin only</i>)	Activity Log, Location
Settings (<i>admin only</i>)	Backups, AI, Email, Email Triggers, Users, Roles & Permissions, Audit Log

Each link is only shown if the current user may view its module (admins see all) or it is flagged open-to-all; whole groups marked admin-only are hidden from non-admins. See [1.2](#).

Menu search

A **search box** at the top of the sidebar ("Search menu... (e.g. attendance)") lets you jump to any item without remembering which group it lives in. It is a case-insensitive substring match on menu labels. While the box has text, every group with a matching child is force-expanded and non-matching items/groups are hidden; clearing the box (or pressing **Escape** inside it) restores the saved accordion state. If nothing matches you get a "No menu items match" message.

Sidebar footer

The footer shows the logged-in user's name, `@username` , email, and their assigned role chips, plus the **Change Password** and **Logout** buttons.

Header

The top header contains:

- A **hamburger / expand** button that hides or shows the sidebar. On desktop, hiding the sidebar reclaims its 256px and a floating "Menu" tab appears at the left edge to bring it back.
- The **page title**, derived from the current route's sidebar label.
- A **push-notification toggle** (`EnablePushButton`) — each device (phone, laptop, desktop) must be enabled separately.

- An **announcement bell** (`AnnouncementBell`) — a single unified inbox with two tabs: HR/system notifications (interview reminders, offer expiries, pending approvals) and company announcements. This replaced an earlier confusing three-bell layout.
- A **build stamp** chip (e.g. `v...`) showing the build timestamp, so you can confirm which deploy is currently loaded (especially useful on the iPhone PWA).

GPS location tracking

Location tracking runs **globally from the Layout**, not just on the Attendance page. As long as any ERP page (or the installed PWA) is open and the user is logged in:

- The browser requests GPS and **pings the backend every 30 seconds** (`POST /attendance/track-location`) with latitude, longitude, and accuracy.
- Each ping doubles as a **heartbeat for backend auto-punch** (attendance).
- The app requests a best-effort **screen wake lock** so the tab/phone is less likely to suspend mid-day.
- GPS accuracy (uncertainty radius, in meters) is sent so the backend can apply tolerance to geofence checks and not wrongly tag an on-site user as "Outside".

Permission denial or timeout is handled silently. A fully closed browser cannot ping — true 24/7 tracking would require a native wrapper. Per-user opt-out is managed in User Management (see [1.3](#)).

1.6 Mobile / PWA Behavior

The app is responsive and installable as a PWA:

- Below 768px the layout switches to **mobile mode**: the sidebar becomes a slide-in overlay (with a dark backdrop) instead of a fixed column, and it auto-closes when you navigate to a new route.
 - The header and content respect iOS **safe-area insets** (`env(safe-area-inset-top/bottom)`) so the hamburger button isn't buried under the status bar / Dynamic Island when running as a full-screen installed app, and content isn't hidden behind the home indicator.
 - The build-stamp chip is rendered in a compact form on small screens to help verify the PWA has the freshest bundle.
-

1.7 Technical Reference

Auth endpoints

All live under `/api/auth` . "Auth" = requires a valid Bearer token; "Admin" = also requires system role `admin` .

Method & path	Auth	Purpose
POST /auth/login	—	Authenticate by username or email + password; returns { token, user, permissions, userRoles }
GET /auth/me	Auth	Current user profile + live permissions + role names
POST /auth/change-password	Auth	Self-service change (verifies current password; new min 4 chars)
POST /auth/recovery-code	Auth	Set/replace personal recovery code (stored hashed)
POST /auth/forgot-password	—	Reset password via personal or emergency recovery code
GET /auth/my-permissions	Auth	Effective permissions for the current user
POST /auth/register	Admin	Create a user (+ optional role assignments)
GET /auth/users	Auth	List users (?active_only=1 to exclude inactive)
PUT /auth/users/:id	Admin	Update a user, roles, and/or password
PATCH /auth/users/:id/track-location	Admin	Toggle a user's location-tracking opt-out
POST /auth/users/:id/reset-password	Admin	Reset a user's password; returns the new password once
DELETE /auth/users/:id	Admin	Delete a user (?force=1 nulls FK refs first)
POST /auth/bulk-import	Admin	Bulk create users from parsed CSV rows
GET /auth/roles	Auth	List roles
POST /auth/roles	Admin	Create a role
PUT /auth/roles/:id	Admin	Rename / re-describe a role
DELETE /auth/roles/:id	Admin	Delete a non-system role
GET /auth/roles/:id/permissions	Auth	Get a role's permission rows
PUT /auth/roles/:id/permissions	Admin	Bulk-replace a role's permission matrix

JWT and middleware

- Tokens are signed with `jsonwebtoken` using `process.env.JWT_SECRET` (falling back to a hard-coded dev secret — **set a real secret in production**).
- The token payload carries { id, email, role, name }, with a base lifetime of **7 days**.
- `authMiddleware` reads the `Authorization: Bearer <token>` header, verifies it, attaches `req.user`, and performs the sliding-session refresh (issues `X-Refresh-Token` when the token is more than ~1 day old).
- `adminOnly` requires `req.user.role === 'admin'`.
- `requirePermission(module, action)` is the per-route gate: admins pass automatically; otherwise it looks up the user's role permissions for that module and checks the mapped flag (`view` → `can_view`, etc.), returning 403 if missing.
- `getUserPermissions(userId)` builds the effective permission map the frontend consumes — full grant for admins, otherwise the OR-merge of all the user's roles.

Where roles & permissions live (database)

Table	Holds
users	Account record: name, email, username, bcrypt password, system role, department, phone, active, approval_role, recovery_code_hash, track_location
roles	Named permission roles (name, description, is_system)
user_roles	Many-to-many join of users ↔ roles (cascades on user delete)
role_permissions	One row per (role, module): can_view, can_create, can_edit, can_delete, can_approve, can_see_all
app_settings	Misc settings incl. emergency_reset_hash for owner-only password recovery

Frontend auth state

`AuthContext` (`client/src/context/AuthContext.jsx`) is the single source of auth truth on the client. It exposes `user` , `token` , `permissions` , `userRoles` , the `login` / `logout` actions, and the permission helpers `can` , `canView` , `canCreate` , `canEdit` , `canDelete` , `canApprove` , `canSeeAll` , and `isAdmin` . On mount it validates the stored token via `/auth/me` ; it also re-pulls permissions on tab focus and every 2 minutes so admin grants apply without a re-login.

2. CRM

The **CRM** sidebar group contains six modules that, together, cover the full pre-sales and order-booking journey at SEPL — from first touch with a referral partner, through lead qualification and quotation, to the moment an order is **booked** in the Business Book and starts feeding procurement, project, and billing modules. Two of the six are project-tracking aids that live under CRM because the CRM/PMC team owns them (Customers master, Full Kitting checklist).

#	Sidebar item	Purpose (one line)	Primary table
2.1	Partners	Influencer / referral-partner master (architects, designers, channel partners)	<code>influencers</code>
2.2	CRM Sales Funnel	Flat 3-step spreadsheet-style tracker (Quotation → Negotiation → Win/Loss)	<code>crm_funnel</code>
2.3	Sales Funnel (Leads)	11-stage governed pipeline with SLAs, gates, MOM, BOQ, follow-ups	<code>sales_funnel</code>
2.4	Business Book	Booked orders ledger; the hand-off point into procurement / DPR / billing	<code>business_book</code>
2.5	Customers	Lightweight customer/company master with codes	<code>customers</code>
2.6	Full Kitting	3-stage project readiness checklist (131 checkpoints) with photo evidence	<code>crm_kitting_*</code>

There are **two parallel funnels** by design. `crm_funnel` (2.2) is the simple flat tracker the sales team used in a Google Sheet before the ERP. `sales_funnel` (2.3) is the full 11-stage governed pipeline built to mam's *SEPL_Sales_Funnel_ERP_Build_Spec*. They are independent tables and screens; do not confuse them. A third legacy table, `leads`, still backs a small `/api/leads` CRUD endpoint but is largely superseded by `sales_funnel`.

2.1 Partners (Influencers / Referral Partners)

Files: `client/src/pages/Influencers.jsx`, `server/routes/influencers.js` **Mounted at:** `/api/influencers` **Permission key:** `influencers`

What it is / business purpose

A master of **referral partners and influencers** — architects, interior designers, MEP consultants, builders/developers, channel partners, vendors — who bring SEPL business. It mirrors the 6-section "Influencer Enquiry Form" Excel mam shared, and supports bulk Excel import/export so the sales team can migrate or mass-update partner data.

The most important downstream use is the **Source → Partner dropdown** on the Lead Capture form (Sales Funnel, 2.3): when a lead's source is *Influencer*, the rep picks the partner from this master.

Who uses it

CRM / business-development team and admins. Full view/create/edit/delete is admin-gated through the `influencers` permission. A separate lightweight `GET /lookup` endpoint is open to any authenticated user so sales reps can fill the partner dropdown without holding the admin-only `influencers:view` permission.

Main screen, columns

A single searchable table with Template / Import Excel / Export Excel buttons (import flow modelled on the Fire-NOC import). Filters: search box, Primary Category, Relationship Stage, City.

Table columns: **Form ID · Name · Category · Company · Mobile · City · ...** (plus Actions). Each row is `INF-####` auto-numbered.

Key fields (6 sections)

The form/record carries 35+ fields grouped into six sections:

Section	Representative fields
1. Basic Identity	salutation, full_name* , date_of_birth, anniversary_date, gender, hometown
2. Professional Category	primary_category , primary_category_other, years_in_industry, decision_making_role
3. Company / Firm Details	company_name, designation, company_size, year_established, office_address, city, pincode, gst_number, website
4. Contact Information	primary_mobile* , secondary_mobile, whatsapp_number, office_landline, personal_email, office_email, preferred_contact_method, best_time_to_call
5. Digital / Social Presence	linkedin_url, facebook_url, instagram_handle, twitter_handle, youtube_channel, google_business_profile, other_listings
6. Relationship & Business Intelligence	source_of_contact, referred_by, first_meeting_date, relationship_stage , typical_project_type, typical_project_value_range, past_projects_count, past_projects_total_value, ongoing_projects_with_us, client_payment_behavior, commission_terms, competitors

Plus audit fields: `form_id` , `date_of_entry` , `entered_by` , `assigned_relationship_manager` , `created_by` , timestamps.

Required: `full_name` and `primary_mobile` . The frontend Category dropdown offers: *Architect, Interior Designer, MEP Consultant, Builder / Developer, Channel Partner, Influencer, Vendor, Others*.

Workflow

- CRM user opens **Partners** and clicks **Add** (or **Import Excel** for bulk).
- Fills the 6-section form. `form_id` auto-generates as `INF-0001` , `INF-0002` ... if left blank.
- On save, the row is created (`POST /api/influencers`) and an audit event is written.
- For bulk loads: download the **Template** (`.xlsx` with all fields + a sample row), fill it, and **Import**. The importer maps human headers back to field keys case-insensitively, converts Excel date serials to `YYYY-MM-DD` , coerces int/number fields, auto-assigns `form_id` when blank, and returns a per-row created/failed report.
- The partner is now selectable in the Sales-Funnel Lead Capture form's **Source = Influencer** → **Partner** dropdown (via `/lookup`).

Automations / notes

- `form_id` auto-numbering (`INF-####`).
- Excel **import** (header-mapped, date/number normalised) and **export** (full dataset, dated filename) and a downloadable **template**.
- Schema is created idempotently at module load.

API endpoints

Method & Path	Purpose	Permission
GET <code>/api/influencers/lookup</code>	id / name / company / category for dropdowns	any auth user
GET <code>/api/influencers</code>	List with <code>search</code> , <code>primary_category</code> , <code>relationship_stage</code> , <code>city</code> filters	view
GET <code>/api/influencers/:id</code>	Single record	view
POST <code>/api/influencers</code>	Create (auto <code>form_id</code>)	create
PUT <code>/api/influencers/:id</code>	Update	edit
DELETE <code>/api/influencers/:id</code>	Delete	delete
GET <code>/api/influencers/import/template</code>	Download <code>.xlsx</code> template	view
GET <code>/api/influencers/export</code>	Download all rows as <code>.xlsx</code>	view
POST <code>/api/influencers/import</code>	Bulk Excel upload	create

DB tables touched

- `influencers` (created/owned here), `users` (FK `created_by`), audit log.

2.2 CRM Sales Funnel

Files: `client/src/pages/CRMFunnel.jsx`, `server/routes/crmFunnel.js` **Mounted at:** `/api/crm-funnel` **Permission key:** `crm_funnel`

What it is / business purpose

A **flat, spreadsheet-style FMS** (Funnel Management Sheet) that reproduces the simple workflow the sales team filled in a Google Sheet before the ERP:

Step 1 — Quotation submit → Step 2 — Negotiation → Step 3 — Win/Loss.

It is intentionally simpler than the 11-stage governed pipeline (2.3) and runs in parallel to it. Each row is one enquiry/opportunity tracked through three steps on one screen.

Who uses it

Sales team and sales head for day-to-day, low-ceremony tracking and quick win-rate reporting.

Main screen, steps, columns

Top of the page shows three step chips and KPI cards (Won Deals + amount, Win Rate %). Step filter chips:

Step chip	Meaning	Filter logic (<code>{step=}</code>)
Step 1	Quotation not yet submitted	<code>quotation_submitted=0</code>
Step 2	Quotation submitted, not closed	<code>quotation_submitted=1 AND final_status empty</code>
Step 3	Closed	<code>final_status IN ('win','loss')</code>
(open)	Any not-yet-closed	<code>final_status empty</code>

Table columns: **Lead # · Client · Company · Mobile · Source · Type · Category · State · BOQ · Quote · Qty Amount · Neg Status · Neg Amount · Stage · Loss Reason · Actions.** Win/Loss render as coloured badges.

Key fields & dropdowns

- **Identity:** `client_name*`, `company_name`, `mobile`, `email`, `address`, `state`, `district`, `remarks`.
- **Classification dropdowns:**
 - **Source** (enforced): *Tenders, Referral, Direct, Website, Channel* (free-text blocked server-side via `validateFunnelSource` ; blank allowed for legacy rows).
 - **Type:** *Private, Government*.
 - **Category:** *Hospital, Hotel, Office, Industrial, Residential, Retail, Educational, Government, Other*.
 - **Lead Type:** *New, Extra Enquiry* (server whitelists these two).
- **Step 1 (Quotation):** `boq_file_link` (file upload, ≤15 MB), `cust_boq_link`, `quotation_link`, `quotation_amount`, `quotation_submitted` (Y/N), `quotation_submit_date`.
- **Step 2 (Negotiation):** `negotiation_status`, `negotiation_amount`, `negotiation_remarks`.
- **Step 3 (Final):** `final_status` (*win / loss*), `loss_reason` (shown when loss), `closed_at`.

Workflow

1. Sales user creates a row (`POST`) with client + classification. `Lead_no` auto-generates as `CRM-0001` ... The server validates `client_name` and the `source` value, and persists any uploaded BOQ to `/uploads/ ...` .
2. The row sits in **Step 1** until a quotation is submitted.
3. User edits the row, attaches the quotation, sets **Quotation submitted = Y**. The server stamps `quotation_submit_date` on that first transition; the row moves to **Step 2**.
4. During negotiation, user records negotiation status/amount/remarks.
5. User sets **final_status = win or loss**. On the first close, the server stamps `closed_at` . Loss requires a reason. Row is now **Step 3**.

Automations / statuses

- `Lead_no` auto-numbering (`CRM-####`).
- **Transition date-stamping:** `quotation_submit_date` and `closed_at` are set only on the first Y/close transition (not overwritten on later edits).

- **Source whitelist** enforced on both create and edit.
- BOQ file upload persisted to the shared `/uploads` directory.
- KPIs (Won, Win Rate, win amount = `negotiation_amount || quotation_amount`) are computed on the client.

API endpoints

Method & Path	Purpose	Permission
GET <code>/api/crm-funnel</code>	List; filters <code>q</code> , <code>step</code> , <code>state</code> , <code>source</code> , <code>type</code>	view
GET <code>/api/crm-funnel/:id</code>	Single row	view
POST <code>/api/crm-funnel</code>	Create (multipart, optional <code>boq_file</code>)	create
PUT <code>/api/crm-funnel/:id</code>	Update (multipart, optional <code>boq_file</code>)	edit
DELETE <code>/api/crm-funnel/:id</code>	Delete	delete

DB tables touched

- `crm_funnel` only (self-contained). Uses `nextSequence` for `lead_no`.

2.3 Sales Funnel (Leads)

Files: `client/src/pages/Leads.jsx`, `server/routes/salesfunnel.js` (11-stage pipeline); plus a separate legacy `server/routes/leads.js` (`/api/leads`). **Mounted at:** `/api/sales-funnel` **Permission key:** `leads` (the sales funnel reuses the `leads` permission set).

What it is / business purpose

The **full governed sales pipeline** built to mam's *SEPL_Sales_Funnel_ERP_Build_Spec*: an 11-stage funnel (plus a terminal *Lost*) with per-stage SLAs, approval **gates**, role ownership, an immutable audit trail, BOQ history, and follow-ups. This is where a serious opportunity is worked end to end before it becomes a booked order in the Business Book.

Who uses it

Cross-functional: BD captures leads, Sales Head qualifies, Site Engineer surveys, Designer drafts, Estimation costs the BOQ, CFO/Sales Head gate pricing, Sales submits the bid, and Legal+CFO gate the contract — each stage's `who` is declared in code.

Stage pipeline

GET `/api/sales-funnel/stages` returns the canonical list:

#	Stage key	Label	Owner	SLA (hrs)	Gate
1	lead_capture	Lead/Tender Capture	BD	1	—
2	qualification	Qualified or Not	Sales Head	24	—
3	site_survey	Site Survey + Feasibility	Site Eng	72	—
4	concept_design	Concept Design / Drawings	Designer	168	—
5	boq_costing	BOQ + Vendor Costing	Estimation	168	—
6	pricing_review	Internal Pricing Review	CFO	24	gate
7	quote_submitted	Quote / Bid Submission	Sales	—	—
8	technical_clarification	Technical Clarification	Sales + Tech	24	—
9	commercial_negotiation	Commercial Negotiation	Sales Head	—	—
10	contract_signed	Contract + LOI / PO	Legal + CFO	—	gate
11	project_kickoff	Project Kickoff	PM	—	—
—	lost	Lost (terminal)	—	—	—

Legacy stage keys (`new_lead`, `qualified`, `meeting_assigned`, `mom_uploaded`, `drawing_uploaded`, `boq_created`, `quotation_sent`, `won`) are transparently mapped to the new keys via `LEGACY_STAGE_MAP` so older clients keep working.

Main screen, tabs, columns

- **Tabs / chips:** *Dashboard*, *All Leads (list)*, and one chip **per stage** (each showing the count in that stage).
- **Dashboard:** funnel bar chart by stage, Won/Lost tiles, this-month count, by-category and by-SC breakdowns, recent leads, today/overdue follow-up counts.
- **List columns:** **Lead No** · **Client** · **Company** · **Category** · **Location** · **SC (sales coordinator)** · **Stage** · **SLA** · **Date** · **Actions**. SLA renders as "due in N min / overdue" using `stage_entered_at + sla_hours`.
- **View modal:** a clickable stage rail; selecting a future stage opens that stage's fields to advance; past stages are read-only. Includes BOQ history and follow-ups panels.

Key fields & dropdowns

- **Lead kind (radio):** *Private (Quote)* vs *Government (Tender)*.
- **Source** dropdown; when *Influencer*, a **Partner** picker appears, lazy-fed from `/api/influencers/lookup` (stores `influencer_id` + denormalised `influencer_name`).
- **Category:** Low Voltage, Fire Fighting, Fire NOC, Electrical, SOLAR, MEP, HVAC, Plumbing. Plus a separate **Building Category** list.
- **Stage-1 capture fields:** `client_name*`, `company_name`, `phone`, `email`, `project_name*`, `project_location`, `pin_code`, `estimated_value`, `tentative_timeline`, `sub_trades_scope` (M/E/P/F/BMS/ELV/Solar), `assigned_sc` / `assigned_asm`, `remarks`, `state/district/city/address`.
- **Government-only:** `tender_id*`, `bid_deadline` (must be $\geq$ today), `emd_amount`, `pbg_required`.
- **Per-stage fields:** `qualification` (score/remarks), `site_survey` (date/location/surveyor + MOM: purpose, pain_points, requirements, action_planned, file), `concept_design` (3 drawing slots), `boq_costing` (`boq_file_link`, `boq_amount`), `quote_submitted` (`quotation_number/file/amount`), `contract_signed` (`won_amount`, `result_remarks`).

Workflow

1. **Stage 1 — Capture.** BD creates the lead (`POST /`). `lead_no` = `SEPL9000+`. Stage-1 validation runs: GST/PAN format (when present), `estimated_value` ≥ 0 , and for Government leads a Tender ID + a future bid deadline. `stage_entered_at` is stamped so the 1-hour first-call SLA starts; an audit row is written.
2. **Advance stage** via `POST /:id/stage` with the target `stage` key and its fields. A big `switch` applies stage-specific updates, re-stamps `stage_entered_at` (resetting that stage's SLA), and writes an `enter_stage` audit row.
3. **Qualification (Stage 2)** sets GO/NO-GO; *not_qualified* / first-call *not interested* drops the lead to terminal **lost**.
4. **Site Survey (Stage 3)** records the survey/meeting; an MOM submit (`mom_uploaded`) advances the lead to **Concept Design**.
5. **BOQ (Stage 5)** updates the latest BOQ columns and **also appends to** `sales_funnel_boqs` so every re-sent BOQ is retained (clients re-send BOQs over time). A standalone `POST /:id/boq` can add a BOQ at any stage.
6. **Pricing review (6)** and **Contract (10)** are **gates** (no auto-advance; require explicit approval; slab-based routing noted in spec — currently advance + remarks).
7. **Contract signed (10)** records `won_amount`; the dashboard counts both `contract_signed` and legacy `won` as Won.
8. **Drop:** `POST /:id/drop` closes a lead with a mandatory reason $\rightarrow$ stage `lost`, audited.
9. **Follow-ups:** add (`POST /:id/followup`), log outcome (`PUT /followup/:fid`, optionally auto-scheduling the next one), and today/overdue lists feed the dashboard.

Automations / statuses

- `lead_no` auto-numbering (`SEPL9000+`).
- **SLA engine** (`withSla`): computes `sla_due_at`, `sla_minutes_left`, `sla_overdue` per row from the owning stage's `sla_hours`.
- **Gates:** stages 6 and 10 flagged `gate: true`.
- **Audit trail:** every create/edit/stage/drop/add-boq writes to `sales_funnel_audit` (read back via `GET /:id/audit`).
- **BOQ history** in `sales_funnel_boqs`.
- **Follow-up auto-scheduling** when a `next_followup_date` is set.
- Legacy-key translation for backward compatibility.

API endpoints (sales-funnel)

Method & Path	Purpose
GET /api/sales-funnel	List; filters <code>stage</code> , <code>search</code> , <code>assigned_sc</code> , <code>category</code> (with SLA)
GET /api/sales-funnel/stages	Stage definitions
GET /api/sales-funnel/meta	Sources / categories / sub-trades / stages
GET /api/sales-funnel/dashboard	Pipeline KPIs
GET /api/sales-funnel/:id	Single lead (with SLA)
GET /api/sales-funnel/:id/audit	Audit timeline
POST /api/sales-funnel	Create (Stage 1 capture)
PUT /api/sales-funnel/:id	Edit Stage-1 fields
POST /api/sales-funnel/:id/stage	Advance to a stage
POST /api/sales-funnel/:id/drop	Drop with reason
GET /api/sales-funnel/:id/boqs · POST /api/sales-funnel/:id/boq	BOQ history list / add
GET /api/sales-funnel/:id/followups · POST /api/sales-funnel/:id/followup	Follow-ups
PUT /api/sales-funnel/followup/:fid	Log follow-up outcome
GET /api/sales-funnel/followups/today · /overdue	Follow-up queues
DELETE /api/sales-funnel/:id	Delete

Legacy /api/leads endpoint

`server/routes/leads.js` backs an older, simpler `leads` table (company_name, contact_person, phone, email, source_id → `lead_sources` , status, assigned_to, notes) with `GET /sources` , list/stats CRUD. It is largely superseded by the sales_funnel pipeline but remains mounted.

DB tables touched

- `sales_funnel` , `sales_funnel_audit` , `sales_funnel_boqs` , `lead_followups` , `users` , `influencers` (via lookup). Legacy: `leads` , `lead_sources` .

2.4 Business Book

Files: `client/src/pages/BusinessBook.jsx` , `server/routes/businessbook.js` **Mounted at:** `/api/business-book` **Permission key:** `business_book`

What it is / business purpose

The **Business Book is the central ledger of booked orders** — the "Master Business Sheet". When a deal is won, its full commercial + project detail is recorded here, and creating a Business Book entry is the **hand-off point** that wires the order into the rest of the ERP: it auto-creates Order Planning, a DPR site, a cash-flow inflow for any advance, and a receivable for the balance. The Business Book also feeds **procurement** (PO items / purchase orders keyed by `business_book_id`) and **Sales Billing** (the sales bill recovers client + GST detail from the Business Book). Each entry gets a `SEPL20000+` Lead No.

Who uses it

Sales/operations who book the order, plus finance and project teams who consume the auto-created downstream records. Read endpoints feed dashboards and dropdowns across the app.

Main screen, columns

A searchable, filterable table with summary stats. Filters: search (client, company, project, lead no, PO number, customer code), Status, Category, Order Type, Lead Type, date range. A footer shows the **Sale Total** (sum of `sale_amount_without_gst`) to match Cash Flow.

Table columns: **Lead No · Type · Client Name · Project / Location · Category · Order · PO Number · Sale Amt · Advance · Balance · Employee · Status · Actions.**

Dropdown vocabularies (frontend):

- **Status:** booked, advance_received, planning, execution, completed.
- **Category:** Low Voltage, Fire Fighting, Fire NOC, Fire Alarm, CCTV, Access Control, PA System, Networking, Solar, Other.
- **Order Type:** Supply, SITC, AMC, Service.
- **Lead Type:** Private, Government.
- **Source of enquiry:** Inbound Enquiry, Indiamart, WhatsApp, LinkedIn, Reference, Tender, Other.

Key fields

The record carries ~70 fields. Notable groups:

- **Client / location:** client_name*, company_name, project_name, contacts, emails, source_of_enquiry, **district, state, state_code, gstin**, billing_address, shipping_address.
- **Commercial:** sale_amount_without_gst, **po_amount (computed)**, guarantee_required/percentage, order_type, penalty_clause(+date), committed_start/delivery/completion dates, freight_extra.
- **Payment terms:** payment_advance / against_delivery / against_installation / against_commissioning / retention / credit %, credit_days, advance_received, **balance_amount (computed)**.
- **People:** employee_assigned/id, lead_by, and management / operations / PMC / architect / accounts person name+contact.
- **TPA & margin:** tpa_items_count/qty, tpa_material_amount, tpa_labour_amount, accessory_amount, required_labour_per_day, actual_margin_pct.
- **PO & documents:** po_number, po_date, po_copy_link, boq/tpa/drawing/working sheet links (and signed variants).

GST / State auto-fill

On the form, selecting a **State** (from `STATES`) automatically:

1. clears the district, and
2. sets **State Code** via `gstStateCode(state)` (e.g. Punjab → `03`).

District options then come from `DISTRICTS_BY_STATE[state]` . The **State Code** and **GSTIN** flow into the auto-generated **Sales Bill / Tax Invoice** (the GSTIN field placeholder is e.g. `03AABCS1234A1Z5` , a 15-char GSTIN whose first 2 digits are the state code).

PO Amount rule (with GST)

Per mam (2026-05-21): **PO Amount (with GST) is always** `Sale × 1.18` (18% GST). This is non-negotiable and enforced in three places:

1. A one-time, idempotent **backfill** at module load (guarded by an `app_settings` sentinel `bb_po_eq_sale_x118_v1`) recomputes `po_amount` and `balance_amount` for all existing rows.
2. **Force-compute on every INSERT** (`computePoAmount` overrides any client value).
3. **Force-compute on every UPDATE** (same). `balance_amount = po_amount - advance_received` .

Workflow

1. User clicks **Add**, fills the form. State selection auto-fills State Code; GSTIN entered manually.
2. On save (`POST /`):
 - `client_name` required; if a `po_number` is given it's validated against a regex/junk blocklist (`validatePoNumber`).
 - **Duplicate guard:** rejects same PO number, and same client + project (same opportunity).
 - `lead_no` auto = `SEPL20000+` .
 - `po_amount` forced to `Sale × 1.18` ; `balance_amount` computed.
 - The row is inserted, then **auto-links are created** (see below).
3. **Edit** (`PUT /:id`) re-applies the PO-amount rule and PO validation.
4. **Delete** (`DELETE /:id`) cascades to linked records — but is **guarded**: if the order's sites have any DPRs or attendance, it returns `409` and requires `?force=1` (after a mistaken delete once cascade-wiped a client's DPRs).

Automations (the hand-off)

On create, the Business Book entry **feeds other modules**:

Auto-created	Where	Detail
Order Planning row	<code>order_planning</code>	planned start/end from committed dates; note tags <code>lead_no</code> /project/category
DPR Site	<code>sites</code>	name from company/project; address from shipping/billing; linked via <code>business_book_id</code>
Cash-flow inflow	<code>cash_flow_daily</code> + <code>cash_flow_entries</code>	only if <code>advance_received > 0</code> ; "Advance Received" inflow; daily totals recomputed
Receivable	<code>receivables</code>	only if <code>po_amount > advance_received</code> ; outstanding = balance; due in <code>credit_days</code> (default 30); status <code>green</code>

Downstream consumers (not created here, but keyed to it): **procurement** (`po_items` , `purchase_orders` with `business_book_id`), **project finance** (`project_finance`), **DPR** (via the auto-created site), and **Sales Billing** (recovers client/GST from the Business Book). Full Kitting (2.6) also groups its projects by `business_book.company_name` .

API endpoints

Method & Path	Purpose	Permission
GET <code>/api/business-book</code>	List; filters status/category/order_type/lead_type/search/date range	view
GET <code>/api/business-book/stats/summary</code>	Totals + by status/category/order-type	view
GET <code>/api/business-book/:id</code>	Single entry (+ employee name)	view
POST <code>/api/business-book</code>	Create (forces PO=Sale×1.18, auto-links)	create
PUT <code>/api/business-book/:id</code>	Update (re-forces PO rule)	edit
DELETE <code>/api/business-book/:id</code>	Delete (guarded; <code>?force=1</code> to cascade DPRs)	delete

DB tables touched

- **Owns:** `business_book` . **Writes on create/delete:** `order_planning` , `sites` , `cash_flow_daily` , `cash_flow_entries` , `receivables` , `project_finance` , `po_items` , `purchase_orders` , and (on delete) DPR family (`dpr` , `dpr_work_items` , `dpr_manpower` , `dpr_machinery` , `attendance` , `geofence_settings`). Reads `users` . Uses `app_settings` (backfill sentinel) and `nextSequence` .

2.5 Customers

Files: `client/src/pages/Customers.jsx` , `server/routes/customers.js` **Mounted at:** `/api/customers` **Permission key:** `customers`

What it is / business purpose

A **lightweight customer / company master** with auto-generated customer codes (`CUST-01000+`). It is a simple reference list of companies and their concern persons, separate from the heavier Business Book — useful for category-based filtering and bulk import of an existing customer list.

Who uses it

CRM / sales for maintaining the company directory.

Main screen, columns

A searchable table with a Category filter and category counts, plus Excel bulk-import and CSV export. Columns: **Customer Code** · **Company Name** · **Sub Company** · **Category** · **Contact No** · **Email** · **Concern Person** · **Actions**. Category renders as a coloured chip.

Category dropdown: FF, ELE, LV, Solar, HVAC, INTERIOR, Govt, Private, OTHER.

Key fields

`customer_code` (auto, read-only), `category`, `company_name*`, `sub_company_name`, `company_registration_address`, `contact_no`, `email`, `concern_person_name`, `concern_person_email`, `concern_person_address`.

Workflow

1. **Add** a customer — `company_name` required. On `POST` :

- **Duplicate guard:** rejects same company_name; also flags same contact_no.
 - customer_code auto-generates (CUST-01000 , CUST-01001 ...).
2. **Edit** updates all fields except the read-only customer_code .
 3. **Bulk import:** upload an Excel file. The importer auto-detects the header row (scans first 10 rows for a "company/customer/name" column), fuzzy-maps columns (company, sub-company, category, registration address, contact, email, concern person fields), generates a code per row, and returns {added, errors, total} . Rows without a company name are skipped.

Automations / notes

- customer_code auto-numbering via nextSequence .
- Duplicate detection on company name and phone.
- Header-detecting, fuzzy-mapped Excel bulk import.

API endpoints

Method & Path	Purpose	Permission
GET /api/customers	List; filters search , category	view
GET /api/customers/:id	Single	view
POST /api/customers	Create (auto code, dup guard)	create
PUT /api/customers/:id	Update (code read-only)	edit
DELETE /api/customers/:id	Delete	delete
POST /api/customers/bulk-import	Excel upload	create

DB tables touched

- customers only. Uses nextSequence and duplicateGuard .

2.6 Full Kitting (CRM Full Kitting)

Files: client/src/pages/CRMKitting.jsx , server/routes/crmKitting.js **Mounted at:** /api/crm-kitting **Permission key:** crm_kitting

What it is / business purpose

A **3-stage project-readiness checklist** ("full kitting") that tracks, for each project, whether 131 readiness checkpoints across **Pre-Start → Execution → Handover** are done — with a **photo per checkpoint** and full history. Field staff can upload **today's or a back-dated photo (up to 5 days)** and still see all previous photos for that checkpoint. Projects are derived from the Business Book (grouped by company_name , mirroring Cash Flow's project grouping).

Who uses it

CRM owners, PMs, and site staff. Admins manage the checkpoint master; field staff record statuses + photos.

Stages & checkpoints

Checkpoints live in crm_kitting_checkpoint , grouped by stage_no (1–3) and a section , seeded (v2, guarded by crm_kitting_seed_v2) to:

Stage	Name	Items	Sections (examples)
1	PRE-START	55	DRAWINGS, SITE, CORE MAT, LONG-LEAD, CONSUMABLES, PROCUREMENT, RESOURCES, PLAN, COMMERCIAL, PERMITS
2	EXECUTION	35	DAILY, WEEKLY, QC, SAFETY, MAT TRACK, CHANGES
3	HANDOVER	41	TECHNICAL, DOCS, QC SIGN, COMMERCIAL, DEMOB, DLP, FINAL

Each checkpoint status is one of the dropdown values **Yes / No / Partially / N/A**.

Main screen

A single **matrix** screen (table-layout: fixed): rows = projects (rolled up by company_name), columns = the 131 checkpoints grouped by stage + section, with left-hand project-meta columns (**CRM Owner, Phase/Zone, PM Owner, Target Start**). A stage selector switches

which stage's columns are shown. Each cell shows the latest status (coloured) + a history-count badge; clicking a cell opens a panel to set status, add remarks, set observation date, and upload a photo. The whole grid loads in one round-trip via `/matrix`.

Data model

- `crm_kitting_checkpoint` — master checkpoint list (stage_no, section, sort_order, label, is_active). Admin-editable; **soft-deleted** (`is_active=0`) to preserve history.
- `crm_kitting_entry` — **append-only history**; every dropdown change / photo upload inserts a new row. The "current" status is the most recent row per (`project_key`, `checkpoint_id`). Keyed by `project_key` = Business Book `company_name` (so multiple BB rows / POs for the same company share one kitting state).
- `crm_kitting_project_meta` — per-project CRM owner / phase / PM / target start.

Workflow

- Admin reviews/edits the **checkpoint master** (`/checkpoints` CRUD; create/edit/delete are admin-only inside the handler).
- The **project list** comes from `business_book` grouped by `company_name` (`/projects`), with rolled-up sale/PO amounts and a `bb_entry_count`.
- For a project, the matrix shows each checkpoint with its latest entry (`/matrix` or `/project?key=`), plus a per-stage summary (yes/no/partially/na/pending counts).
- Field user clicks a cell and submits a new entry (`POST /entry`, multipart): chooses status (one of the four), optional remarks, an **observation_date** (defaults to today; validated to be **not future and at most 5 days back**, `UPLOAD_BACK_DAYS=5`), and an optional **photo** (≤10 MB, stored under `/uploads/crm-kitting/`). `uploaded_at` is always the real now() (the audit timestamp); `observation_date` is what the user claims.
- Previous photos/statuses for a checkpoint are viewable via `/history?key=&cp=`.
- Project-meta (CRM owner / phase / PM / target start) is upserted via `/project-meta`.

Automations / statuses

- Append-only history** — never overwrites; latest row wins.
- Back-dated uploads** allowed up to 5 days; future dates rejected.
- Soft-delete** of checkpoints keeps historical entries valid.
- Project derivation** from Business Book by `company_name` (Cash-Flow-style grouping); entries validate the `project_key` still maps to a BB row.
- Idempotent schema + v2 seed (131 checkpoints) at module load, with defensive `ALTER /backfill` migrations for older installs (e.g. `project_id` → `project_key`).
- Audit events on checkpoint and entry writes via `LogAuditEvent`.

API endpoints

Method & Path	Purpose	Permission
GET <code>/api/crm-kitting/checkpoints</code>	Active checkpoint master	view
POST <code>/api/crm-kitting/checkpoints</code>	Add checkpoint (admin)	create
PUT <code>/api/crm-kitting/checkpoints/:id</code>	Edit checkpoint (admin)	edit
DELETE <code>/api/crm-kitting/checkpoints/:id</code>	Soft-delete (admin)	delete
GET <code>/api/crm-kitting/projects</code>	BB-derived project list	view
GET <code>/api/crm-kitting/project?key=</code>	Project + checkpoints + latest entries + summary	view
GET <code>/api/crm-kitting/matrix</code>	Whole grid in one payload	view
POST <code>/api/crm-kitting/entry</code>	New status/photo entry (multipart)	edit
GET <code>/api/crm-kitting/history?key=&cp=</code>	Full history for one checkpoint	view
PUT <code>/api/crm-kitting/project-meta</code>	Upsert CRM owner / phase / PM / target start	edit

DB tables touched

- `crm_kitting_checkpoint`, `crm_kitting_entry`, `crm_kitting_project_meta`, `app_settings` (seed/migration sentinels), `business_book` (project source), `users` (uploader names).

Cross-module relationships (summary)

- **Partners** → **Sales Funnel**: influencer master feeds the *Source = Influencer* partner dropdown on lead capture (`/influencers/lookup`).
- **Sales Funnel** → **Business Book**: a won lead is booked as a Business Book entry (separate tables; the funnel does not auto-create the BB row).
- **Business Book** → **everything downstream**: on create it auto-creates Order Planning, a DPR Site, a cash-flow advance inflow, and a receivable; it is the FK anchor (`business_book_id`) for procurement (PO items / purchase orders), project finance, and DPR; and it supplies client + **GSTIN/State** to Sales Billing.
- **Business Book** → **Full Kitting**: kitting projects are the distinct `business_book.company_name` values.
- **Customers** is an independent directory used for reference/filtering; it is not the FK source for orders (that is the Business Book).

3. Quotes & Orders

This section documents the modules that turn a sales lead into a priced quotation and, ultimately, a booked order. It covers five closely related screens:

- 1. **Quotations (BOQ / Drawings + Quotations)** — the classic, manual flow: build a Bill of Quantities, then issue a numbered quotation against it.
- 2. **AI Auto-Quotation (Estimator)** — upload a client BOQ (Excel / PDF / Word), let the AI match each line to the Item Master, and build a sale price automatically (PP → ACC → LAB → margin → SP).
- 3. **PO/FOC Stripped** — define each saleable PO item together with its free-of-cost (FOC) accessory kit, labour and margin, and run it through an approval workflow.
- 4. **Labour Rate** — the master sheet of item-wise labour / sub-contractor rates that feed PO/FOC and the Estimator.
- 5. **Extra-indent Auto-Quotation** — for Extra (Schedule / Non-Schedule) indents, auto-generate a SEPL-format client quotation priced from previous BOQs.

All quotation, BOQ, PO/FOC, labour and estimate endpoints live under the `/quotations` router (`server/routes/quotations.js`), except the Extra-indent quotation which lives under `/procurement` (`server/routes/procurement.js`). The whole router requires authentication (`authMiddleware`).

3.1 Quotations (BOQ & Quotations)

File(s): `client/src/pages/Quotations.jsx` , `server/routes/quotations.js`

Business purpose

The original, manual quote flow. The sales team records what the client asked for as a **Bill of Quantities (BOQ)** — a list of items, quantities and rates — and then raises a formal, auto-numbered **Quotation** against that BOQ with an optional discount.

Who uses it

Sales / estimation staff who own a lead. Deletion of BOQs and quotations is gated behind the `quotations` delete permission (`canDelete('quotations')`).

Main screen — two tabs

The page is a single screen with two toolbar tabs (URL-synced via `useUrlTab` , default `boq`):

Tab	Purpose
BOQ / Drawings	List + create Bills of Quantities
Quotations	List + create numbered quotations

BOQ TAB COLUMNS

Column	Source
Title	<code>boq.title</code>
Client	<code>leads.company_name</code> (joined)
Drawing	<code>drawing_required</code> (Yes/No)
Total	<code>boq.total_amount</code>
Status	<code>boq.status</code> (<code>draft</code> / <code>submitted</code> / <code>approved</code>)
Date	<code>boq.created_at</code>
Actions	Delete (permission-gated)

QUOTATIONS TAB COLUMNS

Column	Source
Number	quotation_number (e.g. QTN-0001)
Client	leads.company_name
Total	total_amount
Discount	discount
Final	final_amount
Status	status (draft / sent / negotiation / accepted / rejected)
Actions	Status dropdown (inline update) + Delete

Both tabs have an **Export Excel** button (client-side CSV via `exportCsv`).

Key fields

- **BOQ:** Lead/Client, Title (required), Drawing Required (checkbox), plus a repeating item grid of { `description`, `quantity`, `unit`, `rate`, `item_id` } .
- **BOQ item:** picked from the Item Master via a `SearchableSelect` (`/item-master/dropdown`), or typed as free text. Picking a catalogue item auto-fills description, unit and the rate from `current_price` .
- **Quotation:** Lead/Client, BOQ Reference, Total Amount, Discount, Final Amount (read-only, auto = total – discount), Valid Until, Notes.

Workflow — manual quote

1. Open **BOQ / Drawings** → **Create BOQ**.
2. Pick the Lead/Client and enter a Title; tick **Drawing Required** if applicable.
3. Add item rows. For each, either pick an Item Master entry (rate auto-fills from `current_price`) or type a free-text description.
4. When an item is picked, the **AI rate suggestion** panel appears (see below) showing the last rate quoted to this client and a 6-month avg/low/high; click **Use this** to adopt a rate.
5. Save — `POST /quotations/boq` . The total is computed server-side as $\sum (\text{quantity} \times \text{rate})$.
6. Switch to **Quotations** → **Create Quotation**. Pick the client and the BOQ reference, set Total/Discount (Final auto-computes), Valid Until and Notes.
7. Save — `POST /quotations` mints the next `QTN-####` number via `nextSequence` .
8. Drive the quotation through its lifecycle using the inline status dropdown (`draft` → `sent` → `negotiation` → `accepted` / `rejected`).

AI rate suggestion (price intelligence)

When a catalogue item is selected on a BOQ row, the page calls `GET /ai-agent/rate-suggestion?item_id=...&lead_id=...` . The `RateSuggestion` component renders:

- **Last quoted to this client** (rate + who + how long ago), or
- **Last quoted (any client)** as a fallback, plus
- **Last 6 months** stats: count, average, low, high.

Each line has a one-click **Use this** to copy the suggested rate into the row. This keeps the team quoting consistently and prevents accidental underselling.

Price-history side effect (the "AI Agent")

On `POST /quotations/boq` , for every line that is **linked to a catalogue item AND has rate > 0**, the server:

1. Inserts a row into `item_price_history` (`source = 'boq'` , with lead/company/boq context and who created it).
2. Updates `item_master.current_price` to that rate — so the newest quoted rate becomes the default suggestion everywhere.

Print / PDF outputs

The manual Quotations page itself has no PDF; it offers **Export Excel** (CSV) for both BOQs and quotations. (The SEPL-format printable quotation is the Extra-indent auto-quotation in \$3.5.)

Delete guards

- `DELETE /quotations/:id` is refused (409) if any `purchase_orders` reference the quotation.
- `DELETE /quotations/boq/:id` is refused (409) if any `quotations` reference the BOQ; otherwise it cascades the BOQ items.

API endpoints

Method & path	Purpose
GET /quotations/boq	List BOQs (with client + creator names)
POST /quotations/boq	Create a BOQ + items; logs price history
GET /quotations/boq/:id	One BOQ with its items
DELETE /quotations/boq/:id	Delete BOQ (blocked if quotations reference it)
GET /quotations	List quotations
POST /quotations	Create quotation (auto QTN-####)
PUT /quotations/:id	Update totals / status / notes
DELETE /quotations/:id	Delete (blocked if POs reference it)

DB tables

- **boq** — id, lead_id, title, drawing_required, drawing_file, total_amount, status, created_by, created_at .
- **boq_items** — id, boq_id, description, quantity, unit, rate, amount (the route also writes item_id).
- **quotations** — id, lead_id, boq_id, quotation_number (UNIQUE), total_amount, discount, final_amount, status, valid_until, notes, created_by, created_at .
- **item_price_history** — id, item_id, rate, quantity, lead_id, company_name, boq_id, source, created_by, created_by_name, created_at .

3.2 AI Auto-Quotation (Estimator)

File(s): client/src/pages/Estimator.jsx (UI) and server/routes/quotations.js (/auto-match-boq , /estimates , /estimate-export).

Business purpose

A from-scratch (or BOQ-driven) **sale-price builder** that mirrors SEPL's own quotation spreadsheet. Pick items, the material rate auto-fills, add labour, set a margin **per category**, and the sale price is computed line by line. The big differentiator: you can **upload the client's BOQ** (Excel / PDF / Word) and AI matches every line to your Item Master and fills in the rates.

Who uses it

Estimation staff preparing client quotations. Saved estimates are organised **by client**.

Main screen

A header bar with **Build / Saved (by client)** toggle plus a **New** button. Build view sections:

1. **Header inputs:** Client/Lead, Quotation Title, **Accessories %** (a % of material applied to every line).
2. **Auto-build from Client BOQ:** an upload box accepting .xlsx/.xls/.csv/.pdf/.doc/.docx .
3. **Margin % per category:** one input per distinct category present in the rows.
4. **Items table** (the core grid).
5. **Manpower / Additional Cost** block (feeds the Excel SUMMARY sheet).
6. Action bar: Add Item, Save / Update, **Export Quotation (Excel)**, and a running Cost / Margin / Sale Price summary.

ITEMS TABLE COLUMNS

Column	Meaning
#	row index
Item (from Item Master)	searchable picker + free-text description + AI badges + accessory/FOC sub-rows
Category	the item's department (drives the margin)
Qty	quantity
PP ₹	P urchase P rice = material rate (auto from Item Master <code>current_price</code>)
ACC ₹	accessories total (computed)
LAB ₹	labour rate (manual, or from PO/FOC kit)
TP ₹	per-unit base = PP + LAB
TPA ₹	line total (cost)
Margin	category margin %
SP ₹	sale price (computed)
Rate ₹	sale rate per unit = SP ÷ Qty

Auto-pricing formula

Per row, computed in `calc()` :

```
subsCharged = Σ (sub.rate × sub.qty)  for each accessory/FOC sub that is NOT marked FOC
ACC          = subsCharged + PP × Qty × (Accessories% / 100)
TP           = PP + LAB                (per-unit base: material + labour)
TPA          = TP × Qty + ACC          (line cost)
SP           = TPA × (1 + categoryMargin% / 100)
Rate         = SP ÷ Qty               (sale rate per unit)
```

Totals: `cost = Σ TPA` , `sp = Σ SP` , `marginAmt = sp - cost` .

Notes:

- **FOC sub-items** are listed but priced at ₹0 (they don't add to cost). A **non-FOC accessory** adds `rate × qty` to ACC.
- **Margin is per category** (`margins[category]`), not per line.
- All money is rounded to 2 decimals (`r2`).

PO/FOC kit integration

On load, the page fetches `GET /quotations/po-foc` and indexes kits by `po_item_id` (approved kits win over drafts). When you pick an item that has a kit (`kitFields`), the row's **PP**, **labour (LAB)** and **FOC sub-items** are pulled from the PO/FOC kit (a  PO/FOC badge appears). This links the Estimator directly to the approved PO/FOC pricing in §3.3.

Auto-build from a client BOQ — matching logic

`uploadBoq` posts the file to `POST /quotations/auto-match-boq` (multipart). The server:

1. **Extracts line items** by file type:
 - **PDF** → `pdf-parse` text → AI extraction (`llmExtractItems`) or fallback `textToLines` heuristic.
 - **Word (.doc/.docx)** → `mammoth` raw text → same AI / fallback path.
 - **Excel/CSV** → finds the header row (≥2 keyword matches among `description/qty/unit/rate/ ...`), reads the Description/Qty/Unit columns from each sheet; uses the sheet with the most rows.
2. **Loads candidates** = Item Master rows of `type='PO'` only (the things actually quoted), plus their PO/FOC kits keyed by `po_item_id` .
3. **Fuzzy-scores** each BOQ line against the catalogue (`scoreMatch`): tokenises descriptions (drops stop-words), scores by coverage of the item's keywords, weighted down hard for thin evidence (a single common material word can't score high; needs ~3 matched keywords for full weight).
4. **AI refinement** (`llmRefine` , best-effort): for each line it hands the top-8 fuzzy candidates to Claude, which picks the one true match or `null` for composite WORK lines (e.g. "construct brick masonry manhole"). Falls back to pure fuzzy if AI isn't configured or errors.
5. **Decides confidence** per line: a very strong fuzzy match (≥85 %) is auto-applied; otherwise the AI's pick (high ≥70, medium ≥40, low <40) or the fuzzy best (high ≥0.6, medium ≥0.3, low <0.3, none = 0).

6. Returns { count, rows[], matched_by: 'ai' | 'keyword' }. Each row has description, qty, unit, confidence, match, alternatives[]. A matched item carries kit_pp, kit_labour, kit_focs from its PO/FOC kit.

In the UI, each row shows a confidence badge (✅ high / ⚠️ medium / ❗ low / ❗ no match) and a **try:** list of alternative matches that can be applied with one click. Low/none rows are highlighted red for quick review.

The AI key/model are read from app_settings (ai_api_key , ai_model , default claude-opus-4-7), set via the AI Settings UI. No key → matching silently degrades to keyword fuzzy.

Saved estimates (by client)

- **Save / Update:** POST /quotations/estimates (or PUT .../:id when editing). Stores title, lead, client name, acc_pct, margins, full rows (with subs), manpower, and totals (cost , sp).
- **Saved view:** lists estimates grouped by client (GET /quotations/estimates), each with title, SP and date; **Edit** loads it back into Build, **Delete** removes it.

Print / PDF / Excel outputs

Two Excel exports:

1. **Export (client-side CSV)** — exportSheet, a flat sheet with S.NO / Description / Unit / Qty / Rate / Amount / PP / ACC / LAB / TP / TPA / MARGIN % / SP / CATEGORY + a TOTAL line. (FOC sub-items are not listed separately.)
2. **Export Quotation (Excel)** — exportXlsx posts to POST /quotations/estimate-export, which builds a **multi-sheet workbook (SEPL "saizar" format)**:
 - One sheet **per category** (S.NO / DESCRIPTION / MAKE / UNIT / QTY / RATE / AMOUNT / PP / ACCESS / LAB / TP / TPA / MARGIN / SP + category TOTAL).
 - A **SUMMARY** sheet (placed first) with the SEPL letterhead, client/quotation meta, per-category SP subtotal, a **Manpower** block (qty × monthly_cost × months) and a GRAND TOTAL (= subtotal + manpower).

API endpoints

Method & path	Purpose
POST /quotations/auto-match-boq	Upload client BOQ → matched lines + rates
GET /quotations/estimates	List saved estimates (by client)
GET /quotations/estimates/:id	One estimate (parsed margins/rows/manpower)
POST /quotations/estimates	Save a new estimate
PUT /quotations/estimates/:id	Update a saved estimate
DELETE /quotations/estimates/:id	Delete a saved estimate
POST /quotations/estimate-export	Multi-sheet quotation Excel (SUMMARY + per-category)
GET /ai-agent/rate-suggestion	(shared) last-quoted + 6-month rate stats

DB table

- **estimate_quotations** — id, title, lead_id, client_name, acc_pct, margins_json ({category: marginPct}), rows_json (full estimator rows incl subs), manpower_json, cost, sp, created_by, created_at, updated_at.

3.3 PO/FOC Stripped

File(s): client/src/pages/PoFocStripped.jsx, client/src/pages/PoFocPrint.jsx, server/routes/quotations.js (/po-foc*).

Business purpose

Each saleable item (a **PO item**, type='P0' in the Item Master) is sold together with a **FOC kit** — free-of-cost accessories/consumables — plus **labour** and a **margin**. This module is where that bundle ("kit") is defined, costed and approved. The resulting kit feeds directly into the Estimator (§3.2) and the BOQ auto-match (§3.2 step 2).

Who uses it

Estimation / pricing staff (define kits) and an approver (approve them). The list can hold 800+ kits, so the screen is performance-tuned (in-memory rate maps server-side, in-place approve client-side, capped/searchable lists).

Main screen — three status tabs

Tab	Status	Colour
Non-Approved	non_approved	amber
Approved	approved	emerald
Re-Approved	re_approved	blue

Above the tabs is a **Category filter** (the PO item's Item Master department, e.g. Fire Fighting) that applies to every tab.

Non-Approved tab is special: besides showing saved draft kits (searchable, capped at 30), it lists **PO items that still need a FOC kit** (capped at 50, searchable) with a **Define FOC** button each — so you can systematically work through every uncovered PO item.

Each kit (card) shows

PO item name + status badge; Qty, PO ₹, Margin %, FOC count, labour (name, rate × qty, labour margin); the FOC list; and on the right the **TPA** and **cost**. Approved/Re-Approved cards have a **View PDF** link.

Key fields (the edit modal)

Field	Notes
PO Item	<code>SearchableSelect</code> over Item Master <code>type=PO</code> ; picking it sets <code>po_rate</code> from <code>current_price</code> . An ↗ opens that item in Item Master in a new tab.
Qty	shows the item's UOM next to the label
PO Rate ₹	per the item's UOM
Margin %	dropdown: 10/20/30/40/50/75/100; applies to PO + FOC
Labour Item	from the Labour Rate sheet; ↗ edits it, + adds a new one
Labour Rate ₹	per labour UOM
Labour Margin %	own margin, default 50 %
FOC Items	up to 10 items (Item Master <code>type=FOC</code>), each with qty (1–10), UOM and rate

Auto-pricing formula

Computed both client-side (`calc`) and server-side (`computePoFoc`):

```
poAmt    = poRate × qty
focAmt    = Σ (foc.rate × foc.qty)
labourAmt = labour × qty                (labour RATE × PO qty)
cost      = poAmt + focAmt + labourAmt
TPA       = (poAmt + focAmt) × (1 + margin% / 100)
          + labourAmt × (1 + labourMargin% / 100)
```

So PO + FOC carry the item margin, while labour carries its own (default 50 %).

Workflow & approval state machine

1. Open **New PO/FOC** (or **Define FOC** for a pending PO item).
2. Pick the PO item (rate auto-fills), set Qty and Margin.
3. Optionally pick a Labour item (rate auto-fills) and its margin.
4. Add up to 10 FOC items.
5. **Save (keep pending)** → status `non_approved` , or **Approve** → save then `POST .../approve` → `approved` .
6. Editing an **approved** kit moves it to `re_approved` (server rule in `PUT`); re-approving it sends it back to `approved` .
7. **Approve** from a card flips it in place (no full reload) for speed.

Live rates (important behaviour)

`GET /po-foc` resolves every entry **live** against the Item Master and Labour Rate sheet (`liveResolvePoFoc` + `buildLiveMaps`): PO rate, FOC rates/names/UOM and labour rate are re-read by id on every load, and **cost/TPA are recomputed** from those live rates. Stored

values are only a fallback for deleted or manually-typed items. So editing an item's rate in the master automatically updates all kits that reference it. The two bulk reads (items + labour into Maps) replaced thousands of per-row queries to keep the 800-kit page fast.

Print / PDF output

PoFocPrint.jsx at `/po-foc/:id/print` (linked from approved cards) renders a printable **PO / FOC SHEET** with the SEPL letterhead: a header table (PO item, status, qty, PO rate, margin, labour), a line table (the PO line + each FOC line + a labour line), and a footer showing **Cost (PO + FOC + Labour)** and **TPA**. Print/Save-as-PDF via the browser.

API endpoints

Method & path	Purpose
GET <code>/quotations/po-foc</code>	All kits (live-resolved) + status counts
GET <code>/quotations/po-foc/:id</code>	One kit (live-resolved)
POST <code>/quotations/po-foc</code>	Create kit (<code>non_approved</code>)
PUT <code>/quotations/po-foc/:id</code>	Update (approved→re_approved)
POST <code>/quotations/po-foc/:id/approve</code>	Approve
DELETE <code>/quotations/po-foc/:id</code>	Delete kit

DB table

- `po_foc_entries` — `id`, `po_item_id`, `po_name`, `po_rate`, `qty`, `labour (rate)`, `labour_item_id`, `labour_name`, `labour_margin` (default 50), `margin` (default 30), `focs_json` ([{`item_id`,`name`,`qty`,`rate`}]), `cost`, `tpa`, `status` (`non_approved/approved/re_approved`), `created_by`, `approved_by`, `approved_at`, `created_at`, `updated_at`.

3.4 Labour Rate

File(s): `client/src/pages/LabourRate.jsx`, `server/routes/quotations.js` (`/labour-rates*`). (`server/routes/labourPayment.js` is a separate payroll concern and not part of this sheet.)

Business purpose

The master sheet of **item-wise labour / sub-contractor rates** by UOM and category. These rates are the source for the **Labour Item** picker in PO/FOC (§3.3) and, transitively, the Estimator's LAB column. Seeded from SEPL's uploaded sheet.

Who uses it

Estimation / pricing staff. It can be opened deep-linked from PO/FOC (the labour  /  buttons) via URL params:

- `?edit=<name>` auto-opens that item's edit modal,
- `?add=1` opens a blank add modal,
- `?search=<text>` pre-filters the list.

Main screen

A toolbar (Duplicates, Import, Export Excel, Add Labour Item), category pill filters (**All / Low Voltage / ELECTRICAL / Fire Fighting**) + a search box, and the table (capped at 200 rows rendered, search/filter to narrow).

COLUMNS

Column	Source
Task ID	<code>LR-<id></code>
Item Name	<code>item_name</code> / <code>specification</code> / <code>size</code>
Rate ₹	<code>rate</code> (purchase / sub-contractor rate)
UOM	<code>uom</code>
Category	<code>category</code>
(actions)	Edit / Delete

Key fields

Item Name (required), Specification, Size, Rate ₹, UOM (Kg, PCS, Nos, Each, Per Ltr, mtrs, RMT, RFT, R mtr, Per Point), Category (Low Voltage / ELECTRICAL / Fire Fighting).

Workflow

1. **Add Labour Item** (or Edit) → fill the form → Save (`POST` / `PUT` `/labour-rates`).
2. **Export Excel** downloads the filtered set as a real `.xlsx` (respects search/category).
3. **Import** opens a modal with a downloadable template; upload `.xlsx/.xls/.csv` (header row 1, `Item Name` required). Rows are **added** (existing items untouched); skipped rows are reported.
4. **Duplicates** scans for labour items sharing the same name (case/space-insensitive) — these splinter one task across rows and break the live link from PO/FOC kits. Pick the row to **keep** (the one used by kits is pre-selected and badged), then **Merge**: any PO/FOC kits using the removed rows are repointed to the kept row in one transaction, then the duplicates are deleted.

Print / PDF outputs

No PDF; Excel **export** + a blank **template** are provided. Downloads go through axios as a blob so the JWT header is sent.

API endpoints

Method & path	Purpose
<code>GET /quotations/labour-rates</code>	List (filters: <code>search</code> , <code>category</code>)
<code>POST /quotations/labour-rates</code>	Add a labour item
<code>PUT /quotations/labour-rates/:id</code>	Update
<code>DELETE /quotations/labour-rates/:id</code>	Delete
<code>GET /quotations/labour-rates/export</code>	Download as <code>.xlsx</code> (respects filters)
<code>GET /quotations/labour-rates/template</code>	Blank import template <code>.xlsx</code>
<code>GET /quotations/labour-rates/duplicates</code>	Same-name groups + kit-usage count
<code>POST /quotations/labour-rates/merge</code>	Merge dupes (repoint kits, delete others)
<code>POST /quotations/labour-rates/import</code>	Bulk import from uploaded file

DB table

- `labour_rates` — `id`, `item_name` (required), `specification`, `size`, `rate`, `uom`, `category`, `created_by`, `created_at`, `updated_at` . Referenced by `po_foc_entries.labour_item_id` .

3.5 Extra-indent Auto-Quotation (SEPL format)

File(s): `client/src/pages/QuotationPrint.jsx` , `server/routes/procurement.js` (`buildExtraQuotation` + `GET /procurement/indents/:id/quotation`).

Business purpose

For **Extra (Schedule / Non-Schedule)** indents — additional work beyond the original order — the system auto-generates a printable client quotation in SEPL's BARAWARE format, so the team can quickly bill extras.

- **Extra-Schedule:** auto-priced (each chargeable line is priced from a previous BOQ — see below).
- **Extra-Non-Schedule:** quotation is prepared **manually** (no schedule rate exists for those items).

Who uses it

Procurement / billing staff handling extra-work indents. Opened at `/quotation/:indentId/print` .

Auto-pricing logic (`buildExtraQuotation`)

1. Load the indent; resolve the **client** from the linked planning Business Book, else match a Business Book row by the indent's site name.
2. Select **chargeable lines** = all indent items EXCEPT free items (`item_type` `FOC` / `RGP`) and from-store lines.
3. For each line, take the BOQ name (the linked `po_items.description` if present, else the line description) and look up its **rate**:

the MOST RECENT previous BOQ (`po_items`) row whose `description` **exactly** matches (case/space-insensitive) the item name, with `rate > 0` — ordered by `id DESC LIMIT 1` .

4. Compute `amount = qty × rate` (rounded 2 dp) and sum to `supply_total` . Lines with no matching previous BOQ rate get **rate 0** and a `rate_found: false` flag.

The endpoint returns `{ company, quotation {no, date}, client, items[], supply_total, basic_amount, indent_number }` , where the quotation number is `SEPL/QTN/<indent_number or id>` .

Print layout (`QuotationPrint.jsx`)

A royal-blue SEPL-branded printable page:

- Header: SE logo box, company name, Head Office / Corporate Office / Website.
- Client + quotation meta grid (Name/Client, Address, GSTIN; Date, Quotation No, Prep By = SEPL, Revision No = 0, Ref Indent).
- **CHAPTER-1 : SUPPLY** table — S.No / Description / Unit / Qty / Rate / Amount + TOTAL. Lines with no previous BOQ rate are tagged " (no previous BOQ rate — fill manually)".
- **BASIC AMOUNT** total + "Taxes extra as per actual government rates."
- A full **Terms & Conditions** list (validity 10 days, 50/40/10 payment terms, warranty, completion 90 days, scope splits, etc.).
- Footer tagline and "For SECURED ENGINEERS PVT. LTD."

A toolbar (hidden on print) offers **Back** and **Print / Save PDF**; an amber banner warns if any line was priced at ₹0.

API endpoint

Method & path	Purpose
GET /procurement/indents/:id/quotation	Auto-quotation data for an Extra indent

DB tables (read)

- `indents` , `indent_items` — the indent and its lines (with `po_item_id` , `item_type` , `source`).
- `po_items` — previous BOQ lines (the rate source for exact-name matching).
- `business_book` , `order_planning` — to resolve the client.

3.6 Relationship: Quotation → Business Book order

The quotation modules sit at the **front** of the order lifecycle, and several lines of evidence link a quotation to a booked order:

1. **Lead → BOQ → Quotation**. A quotation references both a `lead_id` and a `boq_id` . The lead is the prospect; the BOQ is the scoped item list; the quotation is the priced offer (`quotations.quotation_number` , e.g. `QTN-0001`).
2. **Quotation → Purchase Order**. When the client accepts, a **Purchase Order** is recorded that references the quotation (`purchase_orders.quotation_id`). The delete guard on `DELETE /quotations/:id` (409 if any PO references it) enforces this dependency.
3. **PO/Order → Business Book**. The accepted order is booked into the **Business Book** (`business_book`) — SEPL's master "New Business Booked" sheet — which then drives `order_planning` , indents and downstream procurement. The Extra-indent auto-quotation (§3.5) closes the loop: it reads back from `business_book` / `order_planning` (to identify the client) and from previous BOQs (`po_items`) to price extra work.
4. **Pricing memory loops back**. Rates quoted in BOQs are written to `item_price_history` and `item_master.current_price` , so the AI rate suggestion and the Estimator's PP column reflect the most recent real quotes — every order's pricing informs the next quotation.

In short: **Lead → BOQ → Quotation → (accepted) → Purchase Order → Business Book order → Order Planning / Indents**, with the Estimator and PO/FOC modules supplying the sale-price intelligence that feeds the quotations at the start of that chain.

4. Procurement

The Procurement module is the largest functional area of the SEPL ERP. It covers the full life of a material requirement — from defining what an item is (Item Master), to discovering its price (RFQ Queue), to who can supply it (Vendors), to the end-to-end purchasing engine (**Indent → Dispatch**), and finally to the planning and timing layers (**Order to Planning** and the **Schedule / Gantt**).

Frontend pages live under `client/src/pages/`; backend routes under `server/routes/`. The two heaviest files are `client/src/pages/Procurement.jsx` (485 KB) and `server/routes/procurement.js` (300 KB) — both implement the Indent → Dispatch flow.

Concern	Frontend page	Backend route file	API base
Items (Item Master)	<code>ItemMaster.jsx</code>	<code>itemMaster.js</code> , <code>pipeweights.js</code>	<code>/api/item-master</code> , <code>/api/pipe-weights</code>
RFQ Queue (Price Required)	<code>PriceRequired.jsx</code>	<code>pricerequests.js</code>	<code>/api/price-requests</code>
Vendors	<code>Vendors.jsx</code>	<code>procurement.js</code> (vendor sub-routes)	<code>/api/procurement/vendors</code>
Indent → Dispatch	<code>Procurement.jsx</code>	<code>procurement.js</code>	<code>/api/procurement/*</code>
Order to Planning	<code>Orders.jsx</code>	<code>orders.js</code>	<code>/api/orders/*</code>
Schedule (Gantt)	<code>ProcurementSchedule.jsx</code>	<code>procurementSchedule.js</code>	<code>/api/procurement-schedule/*</code>

Note on the canonical process documents: the repo ships two living specs for the purchasing engine — `INDENT-TO-DISPATCH.md` (the 18-step automation status grid) and `INDENT-TO-DISPATCH-PROCESS.md` (the stage-by-stage "how it works today" walk-through). This section consolidates and expands on both.

4.1 Items — Item Master

Business purpose

The Item Master is the single catalogue of every purchasable / issuable thing in the business: hydrant valves, cables, pipes, tools, etc. Every BOQ line, indent line, vendor PO line, and sales-bill line ultimately resolves back to an Item Master record (its name, unit, GST slab, HSN/spec, and current price). It is the source of truth for unit of measure, tax slab, and the latest known rate.

Who uses it

- **Purchase team / Item-master maintainers** — create and edit items, set GST slab, set the current price, maintain the Pipe Weight conversion master.
- **Site engineers** — read-only consumers (item dropdowns when raising indents); they request *new* items via the RFQ Queue rather than creating them directly.

Access is gated by the permissions system: `requirePermission('item_master', 'view' | 'create' | 'edit' | 'delete')` on the backend.

Main screen, columns and fields

The list groups items by **department** and shows: item code, department, item name, specification, size, UOM, GST slab, type, and current price. A row's identity is de-duplicated on `MIN(id)` so duplicate codes collapse to one display row.

Key fields on the item form (`ItemMaster.jsx`):

Field	Notes
Item code	Auto-prefixed from department (first 3 letters, e.g. <code>FF0100</code>) on create if blank
Department	Free category used for grouping (FF, ELE, LV, HVAC, Plumbing, INTERIOR, ...)
Item name / Specification / Size	Descriptive
UOM	Defaults to <code>PCS</code>
GST slab	Dropdown: <code>0%</code> · <code>5%</code> · <code>12%</code> · <code>18%</code> · <code>28%</code> (defaults to <code>18%</code>); a non-standard saved value is preserved as an extra option
Type	Dropdown: <code>PO</code> · <code>FOC</code> · <code>RGP</code> · <code>RENTAL</code> . <code>PO</code> = bought on a purchase order; <code>FOC</code> = free-of-cost (no rate/GST line); <code>RGP</code> = Returnable Gate Pass (contractor's own tools sent to site and returned); <code>RENTAL</code> = rented tool
Make / Model number	Brand + model
Source type	<code>PO</code> · <code>Quote</code> · <code>Manual</code> · <code>Online</code> — where the price came from
Current price	Latest rate; has a dedicated price-history trail
Pipe weight (kg / meter)	Optional, pipes only — see Pipe MTR→KG below

Pipe MTR → KG conversion

Pipes are indented in **meters** but bought from vendors by **weight (kg)**. The Item Master form has an optional **Pipe weight (kg/meter)** field. There is a separate **Pipe Weights master** (the 🍷 *Pipe Weights* button, backed by `pipeweights.js`) that stores standard `pipe_class × size → kg_per_meter` rows. Picking a class/size in the item form auto-fills `weight_per_meter` . When the item is set with kg/m, an item indented in MTR is converted to KG for the **vendor enquiry and the PO** ($qty_kg = mtr \times kg/m$), and both meters and kg are shown.

Open decision (per project memory): whether the *client* Sales Bill / Delivery Note should print in meters or kg is still being decided by management; the vendor side is already kg.

Bulk upload

Items can be bulk-created from an Excel/CSV template (sample column order: `item_code`, `department`, `item_name`, `specification`, `size`, `uom`, `gst`, `type`, `make`, `price`, `vendor`, `source_type`, `bill_po_number`, `bill_po_date`). Backend route `POST /item-master/bulk` .

API endpoints — Item Master

Method & path	Purpose
<code>GET /api/item-master</code>	List (filter by <code>department</code> , <code>type</code> , <code>search</code> , <code>status</code>)
<code>GET /api/item-master/dropdown</code>	Lightweight list for selects
<code>GET /api/item-master/:id</code>	Single item
<code>GET /api/item-master/:id/price-history</code>	Price-change trail
<code>PATCH /api/item-master/:id/price</code>	Update current price
<code>POST /api/item-master</code>	Create
<code>PUT /api/item-master/:id</code>	Edit
<code>DELETE /api/item-master/:id</code>	Delete
<code>POST /api/item-master/bulk</code>	Bulk create from template

API endpoints — Pipe Weights

Method & path	Purpose
GET /api/pipe-weights/lookup	Class/size/kg-per-meter list for the item-form dropdown
GET /api/pipe-weights	List master rows
POST /api/pipe-weights	Add a row
PUT /api/pipe-weights/:id	Edit a row
DELETE /api/pipe-weights/:id	Delete a row

DB tables

- `item_master` — columns include `item_code`, `department`, `item_name`, `specification`, `size`, `uom`, `gst`, `type`, `make`, `model_number`, `current_price`, `weight_per_meter`.
- `pipe_weights` — `pipe_class`, `size`, `kg_per_meter`.

4.2 RFQ Queue — Price Required

Business purpose

A lightweight queue for **items that are not yet in the Item Master** and have no known price. A site engineer raises a "I need a price for this" request; the purchase team collects up to three vendor quotes, picks a winner, and the system then **auto-creates the Item Master entry** with that finalised rate. It is the on-ramp that keeps the Item Master clean and priced.

Who uses it

- Site engineers / requesters** — raise requests (the *Raise* tab).
- Quoters** (admins, or anyone with approve rights on `procurement` or `item_master`) — see the *Quotes* tab, enter vendor rates and finalise.

Screen / tabs

Title: **Price Required**. Two tabs:

- Quotes** (quoters only) — grouped cards of merged requests, each with up to three vendor rate inputs and a finalise action.
- Raise / All requests** — requesters raise a new item and watch its status; an Excel template download + bulk upload is available to raise many at once.

Step-by-step workflow

- Site engineer raises a request for a new item (name, size, spec, make, UOM, type).
- Identical requests merge** on `(name + size + spec + make + uom + type)` so the same need from two sites becomes one card.
- Purchase team enters **up to 3 vendor rates** per merged item.
- Purchase team **finalises** the chosen vendor + rate.
- The system **auto-creates an Item Master entry** with that rate and links the request to it (status becomes `added`).

Statuses

`pending` → (rates being entered) → `final_rate` chosen → `added` (Item Master row created and linked).

API endpoints — Price Requests

Method & path	Purpose
GET /api/price-requests	List all requests
GET /api/price-requests/grouped	Merged groups for the Quotes tab
POST /api/price-requests	Raise a request
PUT /api/price-requests/:id	Edit a request
DELETE /api/price-requests/:id	Delete
PUT /api/price-requests/:id/rate	Enter / update a vendor rate
POST /api/price-requests/:id/finalize	Pick winner → auto-create Item Master row
GET /api/price-requests/template	Blank Excel template
POST /api/price-requests/bulk-upload	Bulk-raise from filled template

4.3 Vendors

Business purpose

The vendor master — the directory of suppliers, who they are, what trades and brands they cover, their tax identity, and their default commercial terms. Vendors are referenced when entering quotes, finalising rates, and raising POs.

Who uses it

Purchase team and admins (vendor master maintenance).

Categories and key fields

Vendor **categories** are a fixed list (`Vendors.jsx`): `FF`, `ELE`, `LV`, `Solar`, `HVAC`, `**Plumbing**`, `INTERIOR`, `OTHER` — each colour-coded in the list.

Key fields:

Field	Notes
Vendor code / Name / Firm name	Identity
Category (required)	One of the fixed list above
Sub category	Optional
Deals in / Type	Free description
Make / Brand	Repeatable — up to ~10 brands this vendor deals in, stored in <code>vendors.makes</code> (CSV)
GSTIN	Validated against the GSTIN regex; state code + PAN are parsed out of it to pre-fill
District / State	Address
Authorized dealer / Turnover	Qualification info
Payment Terms (required) + Credit days	Default commercial terms (credit days optional)
Phone / Email	Contact

GSTIN parsing: the format `2-digit state + 10-char PAN + entity + Z + checksum` is validated, and the state/PAN are extracted to reduce manual entry.

Rate comparison

Vendors feed the **Vendor Rates** comparison in the Indent → Dispatch flow (up to three quotes side by side); their default **payment terms** are surfaced there as a suggestion.

API endpoints — Vendors (under procurement)

Method & path	Purpose
GET /api/procurement/vendors	List vendors
POST /api/procurement/vendors	Create
PUT /api/procurement/vendors/:id	Edit
DELETE /api/procurement/vendors/:id	Delete

DB tables

- `vendors` — `vendor_code`, `name`, `firm_name`, `category`, `sub_category`, `deals_in`, `type`, `makes`, `gst_in`, `district`, `state`, `authorized_dealer`, `turnover`, `payment_terms`, `credit_days`, `phone`, `email`.

4.4 Indent → Dispatch (the purchasing engine)

This is the core of the module: the single page `Procurement.jsx` with a tab strip, and the route file `procurement.js`. It takes an approved requirement and drives it through quoting, PO creation, payment, billing, dispatch/receiving, and debit notes — with auto-numbering and notifications along the way.

Document numbers you will see throughout: Indent `IND-####` · Stock Issue Note `SI/YYYY/####` · Delivery Challan `DC/YYYY/####` · RGP gate pass `RG/YYYY/####` · Vendor PO `VP0/YYYY/####` · Debit Note `DBN/YYYY/####` · Sales Bill (client invoice) `INV/YYYY/####` (current series `GST/26-26/##`).

4.4.1 Tabs (top-level navigation)

Defined in `Procurement.jsx`:

Tab id	Label	Visible to
<code>indents</code>	Raise Indent	users who can raise indents
<code>rates</code>	Vendor Rates	purchase-ops users
<code>vendorpo</code>	Vendor PO	purchase-ops users
<code>payment</code>	Payment	purchase-ops users
<code>bills</code>	Purchase Bills	purchase-ops users
<code>delivery</code>	Dispatch & Receiving	purchase-ops users
<code>debitnotes</code>	Debit Notes	purchase-ops users
<code>pipeline</code>	PO Pipeline	purchase-ops users

A single **Export Excel** button (top right) exports the *current tab's* data as CSV (see Export section).

4.4.2 Indent categories

When raising an indent the user picks a **category**, which controls BOQ caps, stock behaviour, and the approval policy:

Category	Meaning
Material	BOQ items (PO + FOC). RGP hidden.
RGP	Returnable Gate Pass — no BOQ; picked directly from Item Master where <code>type = RGP</code> .
Extra · Schedule	BOQ item exists but the qty cap is removed (over-BOQ). Client-billable.
Extra · Non-Schedule	Item outside the BOQ — picked freely from Item Master (PO + FOC). Client-billable.
Rental	Rented tool — <code>Days × Rate/Day</code> . Blocks if renting $\geq$ buying outright (the 2× buy-vs-rent guard).

4.4.3 Approval levels (L1 / L2 / CRM)

The approval policy is decided at indent creation (`procurement.js`, `POST /indents`):

- **Cutoff:** indents raised **on or after 2026-05-25** use the two-level chain (`approval_policy = 'two_level'`); older indents stay on the legacy `single` policy.
- **Material / RGP / Rental** → `two_level : L1 → L2` (the same person cannot do both). (RGP was briefly a single-HR sign-off `hr_single`, then reverted to the normal L1→L2 chain per management on 2026-06-06.)
- **Extra · Schedule / Extra · Non-Schedule** (client-billable) → `crm_two_level : CRM approves first` (revenue gatekeeper), then **L1 → L2**.

Status progression for two-level: `submitted → l1_approved → approved`. For the CRM chain: `submitted → crm_approved → l1_approved → approved`. Each step fires approval emails/events (`fireIndent( ... )`).

FREE-STOCK CHECK + ALLOCATE (AT APPROVAL)

At the approval screen the approver can **adjust quantity per line** and **allocate "From Store"**: each item shows "📦 **N available**" office stock with a one-tap *use* button; the remainder shows as "**To Procure**". When store stock is used:

- a **Stock Issue Note** `SI/YYYY/####` and a **store Delivery Challan** `DC/...` are created and **office stock is reduced**;
- **RGP** items get an auto **gate-pass challan** `RG//YYYY/####` and stock is reduced;
- **billable extra** items create a billing line and update the CRM funnel.
- Only the **shortfall** continues into purchasing.

This is the "free stock check + allocate" behaviour (step S2 in the automation grid) — it *suggests* availability and the approver decides; it does not auto-allocate.

4.4.4 The tab flow, step by step

STAGE 1 — RAISE INDENT (TAB: RAISE INDENT)

1. Pick a site / planning, select BOQ items + quantity, choose category, submit.
2. An indent number `IND-####` is created (`nextSequence( ... 'IND-', pad:4)`); status = **Submitted**; an "indent raised" email goes out; it routes by the policy above.
3. Quantities are checked against the BOQ cap (except Extra-Schedule, which lifts the cap).

STAGE 2 — APPROVAL (TAB: RAISE INDENT → OPEN THE INDENT)

L1 clicks **Approve L1**, then L2 clicks **Approve L2** (CRM first for extra items). The From-Store/To-Procure allocation and stock/SI/RGP side-effects (Stage 1 above) happen here.

STAGE 3 — VENDOR RATES & FINALISE (TAB: VENDOR RATES)

1. Purchase team picks an approved indent and enters **up to 3 vendor quotes** per item (name, rate, terms = Advance/Credit, credit days).
2. **Finalise** the chosen vendor + rate per line (`POST /item-rates/:id/finalize`). Finalisation is blocked if the parent indent is not fully approved (`assertIndentApprovedByRate`).
3. (Not yet built: auto-emailed RFQs and auto-highlight of the cheapest vendor — quotes are typed in and the winner is chosen by hand.)

STAGE 4 — CREATE VENDOR PO (TAB: VENDOR PO)

1. Pick the indent + finalised items, optionally attach the signed PO, click **Create Vendor PO**.
2. A PO number `VPO/YYYY/####` is created (`nextSequence( ... 'VPO/{yr}', pad:4)`); total is computed from the finalised rates; indent status → **PO Sent**.
3. The PO can be **printed to PDF** (`GET /vendor-po/:id/print`). A PO can be cancelled / uncanceled. (Auto-emailing the PO PDF to the vendor is planned, not built.)

STAGE 5 — VENDOR PAYMENT / ADVANCE (TAB: PAYMENT)

For advance-required POs a payment request runs through the standard approval chain (HR/Accounts → Billing Engineer → Release). The tab buckets POs by urgency:

Bucket	Meaning
🔥 Payment Urgent	Vendor won't ship — Accounts must clear advance / old dues
✓ Recently Cleared	Payment done — Purchase team can chase the bill in the next tab

Each PO carries a payment posture chosen at creation: **No advance** (ships on credit — default), **Advance required** (vendor wants ₹X first), or **Old payment hold** (old dues must clear before shipment). `PATCH /vendor-po/:id/clear-payment` records the clearance. (Auto-pay below a threshold is intentionally manual — no payment gateway.)

STAGE 6 — PURCHASE BILL + MTC (TAB: PURCHASE BILLS)

- When material arrives, upload the vendor's **bill (PDF/photo)**, enter bill no / date / amount / GST, and mark material **Approved / Reject** (`POST /purchase-bills`).
- If no challan exists yet, a placeholder `DC/YYYY/####` is auto-created so the row shows in Dispatch.
- The screen shows an **item-wise variance** (ordered vs received). If a line is short or at an extra rate, a **short-supply / extra-rate debit note** `DBN/YYYY/####` is auto-raised and the vendor can be emailed. (*OCR pre-fill of the bill and auto-booking of matched bills are not built — fields are typed.*)

STAGE 7 — DISPATCH & RECEIVING (TAB: DISPATCH & RECEIVING)

7a. Dispatch to site — create a **Delivery Challan** `DC/YYYY/####` with vehicle / driver / LR details (`POST /delivery-notes`). (*SMS/WhatsApp to the transporter is not built.*)

7b. Receive (GRN) — `PATCH /delivery-notes/:id/receive` :

- Requires a **receiver name** and a **photo of the stamped/signed GRN** (both mandatory; the receipt photo is saved under `/uploads`).
- Accepts **per-line received quantity** (`items_received` JSON: `vendor_po_item_id`, `ordered_qty`, `received_qty`, `short_reason`); received qty (not ordered qty) is what credits stock, so partial receipts don't over-credit.
- If a **warehouse** is selected, items **auto-land as stock IN** (`stock_movements` , `reference_type='RECEIVE'` , idempotent on `DN-<id>`).
- Short supply** → **auto debit note**: if any `received_qty < ordered_qty` , a `short_supply` debit note `DBN/YYYY/####` is auto-raised (value = shortfall × PO rate), one per delivery note (guarded by a `[DN-<id>]` marker in the reason).
- Engineer alert on mismatch (\$16)**: *only* when a short-supply debit was raised, the **site engineer (the indent raiser)** is notified by WhatsApp + SMS + email, all best-effort (never blocks the receipt). A clean, fully-matched receipt sends nothing.
- Auto Sales Bill on receive**: when a **challan** with billable `PO` -type items is received and the PO has no Sales Bill yet, an **INV/** Sales Bill is auto-generated from the BOQ selling rates. It is flagged **draft** when the client GSTIN or any selling rate is missing. FOC/RGP-only challans get **no** sales bill (the challan is the delivery note).

STAGE 8 — DEBIT / CREDIT NOTES (TAB: DEBIT NOTES)

Review auto-raised short-supply debits or create one by hand. Debit **types** include `short_supply` , rejected, and extra-rate. Each is numbered `DBN/YYYY/####` , can be emailed to the vendor, printed (`GET /debit-notes/:id/print`), and settled. (*A credit-note type — money owed back to the vendor — is not built.*)

STAGE 9 — CLIENT SALES BILL & COLLECTIONS (AUTO ON RECEIVE)

The Sales Bill `INV/...` is generated automatically on receive (Stage 7b) from BOQ selling rates and flagged **draft** if the client GSTIN or rates are missing. Staff complete / print the invoice; the DSO dashboard shows outstanding amounts and ageing per site. (Detailed billing logic lives in the Sales Billing section.) A `POST /auto-sales-bills/sweep` endpoint can back-fill any received challans that didn't get a bill. (*Automatic client payment-reminder emails are not built.*)

STAGE 10 — PO PIPELINE (TAB: PO PIPELINE)

A single read-only tracking view of all POs across their stages (`GET /po-pipeline`).

4.4.5 Auto-numbering

All document numbers come from the shared `nextSequence(db, table, column, prefix, {startFrom, pad})` helper, giving a single authoritative source per series:

Document	Series	Where generated
Indent	<code>IND-####</code>	<code>POST /indents</code>
Stock Issue Note	<code>SI/YYYY/####</code>	indent approval (store issue)
RGP gate pass	<code>RGP/YYYY/####</code>	indent approval (RGP items)
Vendor PO	<code>VPO/YYYY/####</code>	<code>POST /vendor-po</code>
Delivery Challan	<code>DC/YYYY/####</code>	<code>POST /delivery-notes</code> , auto on bill, store issue
Debit Note	<code>DBN/YYYY/####</code>	bill entry + receive (auto) and manual
Sales Bill (invoice)	<code>INV/YYYY/####</code> → current <code>GST/26-26/##</code> (started at 61, 2026-06-15)	auto on receive

4.4.6 Short-supply debit notes + engineer alerts (summary)

Two automatic triggers raise a `short_supply` `DBN/...` :

- At bill entry** (Stage 6) — when item-wise variance shows received < ordered.

2. **On receive / GRN** (Stage 7b) — when per-line `received_qty < ordered_qty` ; value = shortfall × PO rate; guarded by `[DN-<id>]` so it fires once.

On the receive trigger only, the **site engineer who raised the indent** is alerted via WhatsApp + SMS + email. The notify path resolves the engineer through `delivery_notes → vendor_pos → indents → users (created_by)` and is wrapped in try/catch so it never blocks the receipt; it silently skips if Twilio/email is not configured.

4.4.7 Prints

Document	Endpoint
Indent	GET <code>/api/procurement/indents/:id/print</code>
Vendor PO (SEPL-format PDF)	GET <code>/api/procurement/vendor-po/:id/print</code>
Delivery Note / Challan / Sales Bill	GET <code>/api/procurement/delivery-notes/:id/print</code>
Debit Note	GET <code>/api/procurement/debit-notes/:id/print</code>
Delivery-note data (for the dispatch view)	GET <code>/api/procurement/vendor-po/:id/delivery-note-data</code>

4.4.8 Export (Excel / CSV)

The single **Export Excel** button exports whatever tab is active, via the `exportCsv(name, headers, rows)` utility (`client/src/utils/exportCsv`):

Active tab	File / columns
Raise Indent	<code>indents</code> — Indent No, Date, Site, Raised By, Status, Items
Vendor PO	<code>vendor-pos</code> — PO Number, PO Date, Vendor, Amount, Status
Purchase Bills	<code>purchase-bills</code> — Bill No, Vendor, Date, Amount, GST, Total, Payment
Dispatch	<code>dispatch</code> — ID, Type, Doc No, PO, Date, Received By, Received On, Status
Vendor Rates	<code>vendor-rates</code> — Item, Vendor 1/Rate 1 ... Vendor 3/Rate 3, Final

4.4.9 API endpoints — Indent → Dispatch (selected)

All under `/api/procurement` .

Method & path	Purpose
GET /vendors , POST /vendors , PUT /vendors/:id , DELETE /vendors/:id	Vendor master
GET /sites	Sites/planning list for raising indents
GET /boq-items , GET /boq-items-by-bb	BOQ lines (by site / business book)
GET /indents	List indents
POST /indents	Raise indent (sets approval policy)
PUT /indents/:id	Approve (L1/L2/CRM), reject, edit, allocate stock
GET /indents/:id / GET /indents/:id/print	View / print
POST /indents/:id/reset-store-issue	Undo a store allocation
DELETE /indents/:id	Delete
GET /indents/:id/items-for-po	Finalised lines ready for a PO
GET /indents/:id/quotation	Auto-quotation feed
GET /vendor-rates , POST /vendor-rates , DELETE /vendor-rates/:id , PUT /vendor-rates/:id/approve	Vendor quote rows
GET /item-rates , POST /item-rates , POST /item-rates/:id/finalize , DELETE /item-rates/:rate_id	Per-item rates + finalise winner
GET /pending-po-items	Items finalised but not yet on a PO
GET /vendor-po	List POs
POST /vendor-po	Create PO (VPO/...)
PUT /vendor-po/:id , DELETE /vendor-po/:id	Edit / delete
POST /vendor-po/:id/cancel , POST /vendor-po/:id/uncancel	Cancel toggle
PATCH /vendor-po/:id/clear-payment	Mark advance/dues cleared
GET /vendor-po/:id/print	PO PDF
GET /vendor-po/:id/with-items , GET /vendor-po/:id/bill-items , GET /vendor-po/:id/debit-source	PO detail feeds
GET /purchase-bills , POST /purchase-bills , DELETE /purchase-bills/:id	Purchase bills (+ auto debit)
GET /delivery-notes , POST /delivery-notes , PUT /delivery-notes/:id , DELETE /delivery-notes/:id	Challans / dispatches
PATCH /delivery-notes/:id/receive	GRN: receive, stock-IN, auto debit, auto sales bill, engineer alert
POST /delivery-notes/:id/sales-bill , POST /delivery-notes/:id/generate-sales-bill	Attach / generate sales bill
GET /delivery-notes/:id/print	Challan / DN / sales-bill print
POST /auto-sales-bills/sweep	Back-fill missing sales bills
GET /debit-notes , POST /debit-notes , PATCH /debit-notes/:id , DELETE /debit-notes/:id , GET /debit-notes/:id/print	Debit notes
GET /po-pipeline	Cross-stage PO overview
GET /sales-bills , POST /sales-bills , DELETE /sales-bills/:id	Sales-bill rows
POST /admin/wipe-indents-pos	Admin clean-slate wipe (double-confirmed)

4.4.10 DB tables (Indent → Dispatch)

- indents** — header: indent_number, status, planning_id, site_name, raised_by_name, client_name, created_by, approval_policy, l1_status/l1_by/l1_at, l2_status/l2_by/l2_at, crm_status, indent_category .
- indent_items** — lines: description, unit, item_type (PO/FOC/RGP/RENTAL), item_master_id, po_item_id, indent_item_id linkage .

- `indent_item_rates` — per-item vendor quotes + finalised winner (`final_rate`, `final_vendor_name`, `final_terms`, `final_credit_days`, `status='finalized'`).
- `vendor_pos` / `vendor_po_items` — PO header + lines (`po_number`, `vendor_id`, `total_amount`, `status`).
- `purchase_bills` — vendor bills.
- `delivery_notes` — multi-purpose: challan / RGP gate pass / sales bill (distinguished by `document_type`), plus receive fields (`received_by_name`, `received_at`, `receipt_file_path`, `items_json`, `warehouse_id`, `sales_bill_pending`, `sales_bill_number`).
- `debit_notes` — `dn_number`, `type`, `vendor_po_id`, `vendor_id`, `amount`, `reason`, `items_json`, `status` .
- `stock_issue_notes` — SI/... headers for store issues.
- `stock_movements` — inventory ledger (auto-IN on receive).

4.5 Order to Planning

Business purpose

The bridge between a won deal (Business Book) and execution. It records the **Client Purchase Order** (with payment-term milestones) and the **Order Planning** that links a PO to a delivery window — the data that downstream procurement (BOQ, schedule) builds on.

Who uses it

CRM / project planners and admins (`requirePermission('orders', ...)`).

Screen / tabs

Page title with two tabs (`Orders.jsx`):

1. **Client Purchase Orders** (`po`) — capture the client PO.
2. **Order Planning** (`planning`) — create a plan linking a PO/business-book to a window.

Key fields

Client PO form: business book entry, PO number, PO date, total amount, advance amount, PO copy link, BOQ file link, **payment-term milestones** (`pt_advance`, `pt_delivery`, `pt_installation`, `pt_commissioning`, `pt_retention`), site engineer(s), CRM name, and status (`received`, `booked`, `planning`, `in_progress`, `completed`). Selecting a business book pre-fills the amount from `po_amount` / `sale_amount_without_gst` .

Order Planning form: PO, business book, planned start / end, notes.

BOQ upload

A Client PO's BOQ can be uploaded from Excel (`po-template` download + `po-upload-excel`), and labour rates separately (`labour-upload-excel`). These BOQ items (`po_items`) are what the Schedule/Gantt and indents draw from.

API endpoints — Orders

Method & path	Purpose
GET <code>/api/orders/business-book-entries</code>	Won deals to attach a PO to
GET <code>/api/orders/po</code> , POST <code>/api/orders/po</code> , PUT <code>/api/orders/po/:id</code> , DELETE <code>/api/orders/po/:id</code>	Client POs
GET <code>/api/orders/po/:id/items</code> , POST <code>/api/orders/po/:id/items</code>	PO BOQ items
GET <code>/api/orders/po/:id/boq-items</code>	BOQ feed
POST <code>/api/orders/po/:id/labour-rates</code> , POST <code>/api/orders/po/:id/labour-rates/reset</code>	Labour rates
GET <code>/api/orders/po-template</code> , POST <code>/api/orders/po-upload-excel</code>	BOQ Excel template / upload
POST <code>/api/orders/labour-upload-excel</code>	Labour Excel upload
GET <code>/api/orders/planning</code> , POST <code>/api/orders/planning</code> , PUT <code>/api/orders/planning/:id</code> , DELETE <code>/api/orders/planning/:id</code>	Order planning
GET <code>/api/orders/bb/:bbId/items</code>	Items for a business book

DB tables

- `purchase_orders` — client PO header (incl. `pt_*` milestones, `business_book_id`).
- `po_items` — BOQ lines (`description`, `unit`, `quantity`, `rate`, `item_master_id`, `hsn_code`).
- `order_planning` — links PO/business book to a delivery window.

4.6 Schedule (Gantt) — Procurement Schedule

Business purpose

A **backward-pass scheduler** that answers the single most important procurement question: "*By what date must I raise the indent for each item so the project still finishes on time?*" Starting from the project's committed completion date and working **backwards** through six phases (install → receive → dispatch → PO → quotes → indent), subtracting each phase's lead time in **business days**, it produces a Gantt and the must-raise-indent date per BOQ item. AI predicts the per-item trade and dispatch lead time; a human reviews and approves before the Gantt is generated.

Who uses it

Project planners / procurement leads (`requirePermission('procurement_schedule', ...)`).

Screen / tabs

Page title **Procurement Schedule** (`ProcurementSchedule.jsx`). A custom SVG Gantt (no chart library) with two-tier rows (a trade row, expandable to its items), bars coloured by phase, and a red "Today" line. Tabs include **Schedule (Gantt)** and an AI/Setup area.

The six phases and colours: `indent` (amber) → `quotes` → `po` → `dispatch` → `receive` → `install` .

Backward-pass logic

`POST /:project_id/regenerate` (`procurementSchedule.js`):

1. Anchor on an end date — body `end_date` → saved meta → business-book `committed_completion_date` (user override always wins).
2. Pull BOQ items from `purchase_orders` → `po_items` for the project.
3. Load **holidays** and the **phase lead-time rules** map (per trade, in business days).
4. For each item, optionally apply AI overrides (`trade` , `dispatch_days` , `reasoning`), then compute backwards with `subBusinessDays( ... )` (skips weekends + holidays):
 - `installEnd = completion_date`
 - `installStart = subBusinessDays(installEnd, install_days)`
 - `receiveEnd = installStart - 1`, `receiveStart = subBusinessDays(receiveEnd, receive_days)`
 - `dispatchEnd = receiveStart - 1`, `dispatchStart = subBusinessDays(dispatchEnd, dispatch_days)`
 - `poEnd = dispatchStart - 1`, `poStart = subBusinessDays(poEnd, po_days)`
 - `quotesEnd = poStart - 1`, `quotesStart = subBusinessDays(quotesEnd, quotes_days)`
 - `indentEnd = quotesStart - 1`, `indentStart = subBusinessDays(indentEnd, indent_days)`
5. Wipe old rows for the project and insert the six phase rows per item; the **earliest indent.start_date** is the headline "raise indent by" date.
6. A **snapshot** of the full row dump + meta is saved each time (browse history / restore without re-running AI).

The AI step (`POST /:project_id/ai-suggest`) calls Claude with the BOQ (and any attached drawings) to predict each item's trade + dispatch lead time + reasoning, which the user reviews before regenerating.

1-day-before reminder cron

A weekday cron (`server/scripts/procurementReminderCron.js` , wired up in `server/index.js` , ~09:00 plus a boot catch-up):

1. Computes **tomorrow's business day**.
2. Scans `procurement_schedule` for live `phase = 'indent'` rows whose `end_date` equals that day.
3. Posts an **announcement + push notification** per item: " 📢 Raise indent tomorrow: {item} ... vendor lead time builds in from tomorrow."
4. Dedups via a `procurement_schedule_reminders` table so nothing double-posts. Disabled with `ERP_DISABLE_PROCSCH_REMINDER=1` ; an admin can also trigger the run on demand.

API endpoints — Procurement Schedule

All under `/api/procurement-schedule`.

Method & path	Purpose
GET <code>/phase-rules</code> , PUT <code>/phase-rules</code>	Per-trade lead-time table
GET <code>/holidays</code> , POST <code>/holidays</code> , DELETE <code>/holidays/:id</code>	Holiday list (business-day math)
GET <code>/projects</code>	Projects with a schedule
GET <code>/:project_id</code>	A project's schedule rows
GET <code>/:project_id/meta</code> , PUT <code>/:project_id/meta</code>	Setup card (start/end, client requirements)
POST <code>/:project_id/drawings</code> , GET <code>/drawing/:fileId</code> , DELETE <code>/drawing/:fileId</code>	Drawing attachments for AI
POST <code>/:project_id/ai-suggest</code>	Claude predicts trade + dispatch lead time per item
POST <code>/:project_id/regenerate</code>	Run the backward pass + write rows + snapshot
GET <code>/:project_id/snapshots</code> , GET <code>/snapshot/:id</code> , DELETE <code>/snapshot/:id</code>	Snapshot history

DB tables

- `procurement_schedule` — `project_id`, `item_id`, `trade`, `phase`, `start_date`, `end_date`, `status`, `lead_days`, `ai_reasoning`.
- `procurement_schedule_meta` — `project_id`, `start_date`, `end_date`, `client_requirements`.
- `procurement_schedule_snapshots` — full approved-Gantt dumps for history/restore.
- `procurement_schedule_reminders` — dedup ledger for the 1-day-before cron.
- Phase-rule and holiday tables back `/phase-rules` and `/holidays`.

4.7 Automation status (cross-reference)

Per `INDENT-TO-DISPATCH.md`, the 18-step automation grid currently stands roughly at:

- Built / reused:** free-stock check + allocate (S2), L1/L2/CRM approvals (S3/S4), PO auto-number (S7 partial), auto delivery challan (S12), auto sales bill on receive (S14), GRN photo (S15), short-supply debit + engineer mismatch alert (S11/S16), PO Pipeline overview.
- Partial:** vendor-rate side-by-side without auto-rank (S6), PO email not sent (S7), bill upload without OCR (S9), variance shown but no auto-book (S10), DSO dashboard without client reminders (S17).
- Not built (by design or future):** low-stock auto-indent (S1), auto-RFQ email (S5), transporter SMS/WhatsApp (S8), RA-bill (S13, awaiting definition), credit-note type (S11 credit), auto-pay (S18 — kept manual, no payment gateway).

Integrations the flow leans on: **SMTP email** (RFQ/PO/reminders/alerts), **Twilio** (SMS/WhatsApp engineer + transporter alerts), and **Claude AI** (Gantt lead-time prediction, planned OCR).

5. Projects

The **Projects** domain of the SEPL ERP covers everything that happens once a sales order is won and the company has to *execute* and *bill* the work on site. It is a cluster of five inter-related modules that together carry a project from the moment labour is paid on the ground, through the daily site report, the snag list, the statutory Fire-NOC renewal funnel, and finally the sequence of client sales bills that turn the executed work into money received.

Module	Page	Route	Server route file
Indent Labour Payment	<code>client/src/pages/IndentLabourPayment.jsx</code>	<code>/indent-labour-payment</code>	<code>server/routes/indentLabourPayment.js</code>
Daily Reports (DPR)	<code>client/src/pages/DPR.jsx</code>	<code>/dpr</code>	<code>server/routes/dpr.js</code>
Snags	<code>client/src/pages/Snags.jsx</code>	<code>/snags</code>	<code>server/routes/snags.js</code>
Fire NOC Renewal	<code>client/src/pages/FireNoc.jsx</code>	<code>/fire-noc</code>	<code>server/routes/fireNoc.js</code>
Sales Billing (4-type)	<code>client/src/pages/SalesBilling.jsx</code>	<code>/installation</code>	<code>server/routes/salesBilling.js</code>

Two sales-bill flows exist. The 4-type sequential module above is the planned per-order billing chain. A second, **delivery-note-driven** sales-bill flow lives in `server/routes/procurement.js` and produces the actual printable SEPL Tax Invoice (with CGST/SGST/IGST split and the Against-Delivery "Payment Due" line). Both write to the shared `sales_bills` table. Both are documented in §5.5.

5.1 Indent Labour Payment (Project Execution & Billing)

Business purpose

Track, per project, every rupee of **labour** spent on site, split across the three ways SEPL pays for labour:

- **L1 — Salary** (own staff on the company payroll)
- **L2 — Daily Wages** (daily-rated workers, `per_day_rate × days`)
- **L3 — Sub-contract Work Orders** (work given to sub-contractors, paid against a Work-Order value)

The project **Budget** is simply the running sum of all three streams. From there the module is designed to grow into the full execution-to-billing chain (Muster Roll → DPR link → Measurement Book / CDPR → Contractor RA Bill → Client RA Bill → Payment Received), but only the **Projects** tab is fully built today; **MB/CDPR**, **RA Bills** and **Dashboard** are phase stubs.

This module **coexists** with the simpler `labour_payment_indents` module (the older `/labour-payment` indent screen, table `labour_payment_indents`). The two do not share data; the new module uses the `proj_*` tables.

Who uses it

Project / commercial team (default owner shown as **Aanchal**). Gated by the `indent_labour_payment` permission module (`view` / `create` / `edit`).

Main screen / tabs

The page (`IndentLabourPayment.jsx`) has four top tabs, each labelled with its build phase (P1, P5, P6):

Tab	Phase	State
Projects	P1	Built — list + per-project Salary / Daily Wages / Work Orders
MB / CDPR	P5	Phase stub
RA Bills	P6	Phase stub
Dashboard	P6	Phase stub

The **Projects** list shows, per project: name, owner, **L1 / L2 / L3** sub-totals, work-order count, and the derived **Budget = L1 + L2 + L3**.

Key fields

- **Project** (`proj_projects`): `unique_name` , `owner` (default Aanchal), `notes` . Projects are entered **manually** — they are *not* derived from the Business Book (a deliberate Phase-1 amendment because auto-derivation was creating wrong projects).
- **Salary entry** (`proj_salary_entries`): `kind` = `legacy` (one bulk pre-ERP amount captured at kickoff) or `monthly` (per-month rows with `employee_name` + `period_month` YYYY-MM), `amount` .
- **Daily-wage entry** (`proj_daily_wage_entries`): `kind` = `legacy` or `entry` ; for `entry` , `total_amount` = `per_day_rate` × `days_required` .
- **Work Order** (`proj_work_orders`): `wo_number` , `sub_contractor_id` / `_name` , `scope` , `planned_value` , `amount_paid` (running), `work_order_file_url` (uploaded WO doc), `status` (`draft` / `active` / `closed` / `cancelled`). **Balance is derived:** `planned_value` - `amount_paid` . The WO count per project is dynamic (never hard-coded).

Step-by-step workflow

1. Create a **Project** (unique name; duplicate names are rejected case-insensitively).
2. Add **Salary** entries (L1) — a legacy bulk row and/or monthly rows.
3. Add **Daily Wages** entries (L2) — legacy bulk and/or `rate` × `days` entries.
4. Create **Work Orders** (L3), attach the WO file, set `planned_value` , and post payments which accumulate into `amount_paid` .
5. The Budget tile updates live as the sum of all three streams.

Formulas

- **Project Budget** = $\sum \text{proj_salary_entries.amount} + \sum \text{proj_daily_wage_entries.total_amount} + \sum \text{proj_work_orders.amount_paid}$
- **Daily-wage entry total** = `per_day_rate` × `days_required`
- **Work-order balance** = `planned_value` - `amount_paid`

Automations / statuses

No cron. Statuses are on the work order only (`draft` / `active` / `closed` / `cancelled`).

API endpoints

All under `/api/indent-labour-payment` , all guarded by `indent_labour_payment` .

Method	Path	Purpose
GET	<code>/projects</code>	List projects with L1/L2/L3 + budget; <code>?q=</code> , <code>?owner=</code> filters
GET	<code>/projects/:id</code>	One project with rollout
POST	<code>/projects</code>	Create project (unique name)
PUT	<code>/projects/:id</code>	Rename / change owner / notes
DELETE	<code>/projects/:id</code>	Delete project
GET	<code>/owners</code>	Distinct owner list
GET	<code>/projects/:pid/salary</code>	List salary entries
POST	<code>/projects/:pid/salary</code>	Add salary entry
DELETE	<code>/salary/:id</code>	Delete salary entry
GET	<code>/projects/:pid/daily-wages</code>	List daily-wage entries
POST	<code>/projects/:pid/daily-wages</code>	Add daily-wage entry
DELETE	<code>/daily-wages/:id</code>	Delete daily-wage entry
GET	<code>/projects/:pid/work-orders</code>	List work orders
GET	<code>/active-work-orders</code>	All active WOs across projects (feeds DPR)
GET	<code>/work-orders/:id/dpr-items</code>	DPR items linked to a WO (progress rollout)
POST	<code>/projects/:pid/work-orders</code>	Create WO
PUT	<code>/work-orders/:id</code>	Update WO (incl. running <code>amount_paid</code>)
DELETE	<code>/work-orders/:id</code>	Delete WO

DB tables

proj_projects , proj_salary_entries , proj_daily_wage_entries , proj_work_orders , plus the phase-stub tables proj_budgets , proj_muster_roll , proj_mb_sheets , proj_mb_lines (and the RA-bill tables seeded for later phases). The active-WO endpoint joins into dpr_work_items.work_order_id .

5.2 Daily Reports (DPR)

Business purpose

The **Daily Progress Report** is the core site-execution document. Each day, the site engineer logs what was installed (**Table A**), what it cost in labour/staff/rental (**Table B**), the manpower deployed, material consumed, machinery used, safety observations, hindrances, and a next-day plan. The system derives a per-day **Profit / Loss** for the site, surfaces loss reasons to management, scores engineer compliance, and marks a DPR as `billing_ready` so the Installation (Type-3) sales bill can be raised against it.

Who uses it

- **Site engineers** submit DPRs and plan the week.
- **Project managers / admin** approve DPRs and flag `billing_ready` .
- **Management** reads the Dashboard, Loss Reasons, and Engineer Compliance views.

Gated by the `dpr` permission module (`view` / `create` / `approve`).

Main screen / tabs

The DPR page toolbar exposes five tabs:

Tab	Content
Dashboard	KPI tiles — Active Sites, DPR Today, Pending approvals, BOQ progress; each tile drills into the matching list
Daily Reports	The DPR list (filterable by date / pending), submit modal, approve, view detail
Engineer Compliance	Shared <code>EngineerPerformance</code> component (also reachable from HR → Performance)
Sites	Site master (create / edit / delete)
Loss Reasons	Management view of DPRs with <code>profit_loss < 0</code> , grouped by category/reason

Manpower Plan tab — required vs actual manpower (required from the order's value slab, actual from DPR `dpr_manpower`) is presented in the **HR System** area, not on the DPR page itself. DPR also has a **Plan-Week** feature (`/dpr/week-view` , `/dpr/plan-week`) that lets an engineer lay out a 7-day plan; planned-only days are stored as stub DPR rows with `is_planned_template=1` .

Key fields

DPR header (`dpr`): `site_id` , `report_date` , `shift` (day/night), `weather` , `overall_status` (`on_track` / `delayed` / `ahead` / `blocked`), `system_type` (MEPF system — Fire Fighting, Electrical, Low Voltage, Plumbing, HVAC, Solar), `floor_zone` , `mb_sheet_no` , safety flags (`safety_toolbox_talk` , `safety_ppe_compliance` , `safety_incidents`), `next_day_plan` , `hindrances` , `remarks` , `grand_total_a` , `grand_total_b` , `profit_loss` , `billing_ready` , `approval_status` (`pending` / `approved` / `rejected`).

Table A — Installation work items (`dpr_work_items`): from the site's PO/BOQ. Per row: `description` , `unit` , `floor_zone` , `boq_qty` , `rate` , `amount` , `planned_qty` , `actual_qty` , `cumulative_qty` , `variance_pct` , optional `work_order_id` link (feeds the Indent Labour Payment contractor rollout).

Table B — Costs: Skilled Manpower @ ₹800/qty and Helper @ ₹500/qty (fixed company rates), Rental Cost, **Staff Cost** (auto-pulled from the site's PO engineers — sum of monthly salary ÷ 30, never exposing individual salaries to the client), and **TA/DA** (auto). Stored via `dpr_manpower` (trade / required / deployed / shortage) and the cost rows.

Supporting child tables: `dpr_contractors` (up to 5+ sub-contractors per day, name + manpower), `dpr_material` (consumed today / cumulative / balance), `dpr_machinery` (equipment, qty, hours, condition).

The DPR Table A labour rate — 11% of SITC

The PO/BOQ rate is the **full SITC** value (Supply + Installation + Testing & Commissioning) and already includes labour. The DPR Table A must carry **only the labour portion**, which is taken as **11% of the SITC rate**:

```
LABOUR_RATE_PCT = 0.11
Table-A row rate = round(SITC_rate × 0.11, 2) // e.g. 1810 → 199.1
```

This 11% is the agreed placeholder until real labour rates are collected. The server-side note in `dpr.js` confirms the rates the app sends are already the labour portion. Historical DPRs were backfilled to this rule via a guarded migration.

Profit / Loss formula

```
Grand Total (A) = Σ Table-A amounts // labour value of work installed
Grand Total (B) = Σ Table-B cost amounts // skilled + helper + rental + staff + TA/DA
Profit / Loss = Grand Total (A) - Grand Total (B)
```

When `Profit/Loss < 0`, the submit form makes the **loss Category and Reason mandatory** so management always knows why a site lost money that day.

Step-by-step workflow

1. (Optional) Engineer plans the week via Plan-Week — stub rows with `is_planned_template=1`.
2. Engineer opens the **Submit DPR** modal for a site/date.
3. Table A auto-fills installation items from the site's PO; rate is set to **11% of SITC**; engineer enters `actual_qty`.
4. Table B fills skilled/helper qty; Staff Cost and TA/DA auto-compute from the site's PO engineers.
5. Engineer logs manpower, material, machinery, contractors, safety, next-day plan, hindrances. If it's a loss day, category + reason are required.
6. Submit → DPR saved with computed `grand_total_a/b` and `profit_loss`.
7. Approver opens the DPR, sets `approval_status` = approved/rejected and may flip `billing_ready = 1`.
8. A `billing_ready` DPR becomes the reference for the **Type-3 Installation** sales bill (\$5.5).

Automations / crons / statuses

- **DPR auto-prompt cron** nudges engineers to submit (the `/dpr/admin/trigger-prompt` endpoint can fire it on demand).
- Statuses: `approval_status` (pending / approved / rejected), `billing_ready` (0/1), `overall_status`.
- **Loss dashboard** aggregates `profit_loss < 0`; `/dpr/:id/loss-addressed` lets management mark a loss as addressed.
- **Engineer compliance** (`/dpr/engineer-compliance`) scores on-time / complete submissions per engineer.

Print outputs

The DPR detail modal renders the SEPL DPR format (Table A / Table B / Profit-Loss banner). There is no separate standalone PDF route; printing is via the browser from the detail view.

API endpoints (selected)

All under `/api/dpr`.

Method	Path	Purpose
GET	/sites	Site list
POST	/sites	Create site
PUT	/sites/:id	Update site
DELETE	/sites/:id	Delete site
GET	/sites/:site_id/po-items	PO/BOQ items for Table A (with SITC rates)
GET	/sites/:site_id/staff-cost	Auto staff-cost (Σ engineer salary $\div$ 30)
GET	/sites/:site_id/ta-da-cost	Auto TA/DA cost
GET	/	DPR list (date / filter params)
GET	/summary	Dashboard KPIs
GET	/week-view	7-day plan + actuals for a site (<code>site_id</code> , <code>week_start</code>)
POST	/plan-week	Upsert the 7-day plan (stub rows)
POST	/	Submit a DPR (header + work_items + manpower + machinery + contractors)
GET	/:id	One DPR with children
PUT	/:id/approve	Approve/reject + set <code>billing_ready</code>
DELETE	/:id	Delete DPR
GET	/progress	BOQ progress rollup
GET	/loss-dashboard	Loss-making DPRs
PATCH	/:id/loss-addressed	Mark loss addressed
GET	/engineer-compliance	Engineer compliance scores
GET	/payment-check/:site_id	Payment check for a site
POST	/admin/trigger-prompt	Fire the auto-prompt manually

DB tables

`sites` , `dpr` , `dpr_work_items` (Table A), `dpr_manpower` (Table B / trade), `dpr_material` , `dpr_machinery` , `dpr_contractors` .

5.3 Snags

Business purpose

A **snag** is a defect or pending item found during inspection/handover that must be fixed and proven fixed before sign-off. The module is a maker-checker punch-list: someone **raises** a snag with a photo, the **assignee** fixes it and **uploads proof**, and an **approver** approves or rejects the proof.

Who uses it

- **Raiser** (anyone with `snags.create`) — logs the snag with site, location, description, photo, priority, assignee, target date.
- **Assignee** — uploads the fix proof.
- **Approver** (`snags.approve` permission / admin) — approves or rejects the proof.

Gated by the `snags` module (`view` / `create` / `edit` / `delete`).

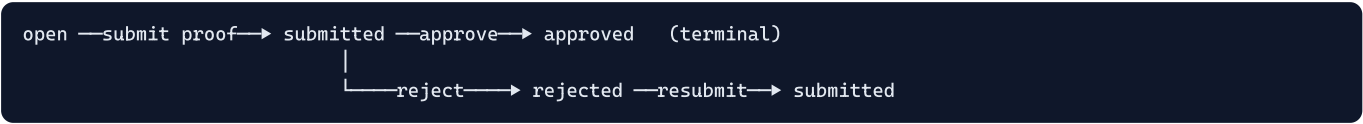
Main screen / columns

The Snags page is a single filterable list (filters: status, priority, search, scope). Columns/CSV: `snag_no` , site name, location, description, **priority**, **status**, raised-by, assigned-to, target date, raised-at. Action buttons appear conditionally based on the viewer's role and the snag status.

Key fields (snags)

snag_no (SNAG-YYYY-####), site_id + site_name (snapshot), location , description , photo_url (raised photo), priority (low / medium / high / critical), status , assigned_to (+name), raised_by / raised_at , target_date , proof_url / proof_notes / proof_submitted_at / _by , approved_by / approved_at , reject_reason / rejected_at .

Status lifecycle



- **open** — raised, awaiting a fix.
- **submitted** — proof uploaded, awaiting approval.
- **approved** — proof accepted; snag closed.
- **rejected** — proof rejected (with reject_reason); assignee can resubmit.

Step-by-step workflow

1. Raiser creates the snag (site, location, description, photo, priority, assignee, target date) → status **open**.
2. Assignee (or approver/admin) uploads fix proof (photo/PDF + notes) → **submitted**.
3. Approver reviews:
 - **Approve** → **approved**, stamps approved_by / approved_at .
 - **Reject** → **rejected** with reason; goes back for **resubmit** → **submitted**.

Automations / statuses

No cron. The submit / approve / reject endpoints carry their own in-handler permission checks (they are not wrapped in requirePermission because the actor may be the assignee rather than a permission-holder). A /stats endpoint feeds counts.

API endpoints

All under /api/snags .

Method	Path	Purpose
GET	/	List snags (status/priority/search/scope filters)
GET	/stats	Counts by status/priority
POST	/	Raise a snag
PUT	/:id	Edit snag (while not approved)
POST	/:id/submit	Upload fix proof → submitted
POST	/:id/approve	Approve proof → approved
POST	/:id/reject	Reject proof (reason) → rejected
DELETE	/:id	Delete snag

DB tables

snags (single table; FKs to sites and users).

5.4 Fire NOC Renewal

Business purpose

A full T-180 → T+30 funnel that auto-pilots the statutory **Fire No-Objection Certificate renewal** for every building SEPL services. Each building's NOC has an expiry; the system computes how many days remain, walks a renewal cycle through a fixed stage sequence (alert → qualify → quote → site visit → PO → dept filing → inspection → NOC issued → final pay → upsell → close), and advances stages automatically as the calendar moves — so the team only has to act, not remember.

Spec: docs/FIRE_NOC.md (mam's brief, 2026-05-16). Money is in **rupees REAL** (column amount , not paise) to match the other ERP modules.

Who uses it

Sales / Sales-Head team. RBAC adds two permission modules: `fire_noc` (view / create=advance-stage / edit / approve=approve-quote) and `fire_noc_master_db` (view / create=ingest). Default mapping: Admin → all; Sales Head → most; Sales → view/edit/advance/master-db.view.

Main screen / tabs

Tab	Content
Dashboard	KPI tiles + per-stage funnel counts + next-7-days expiries
Cycles	Searchable cycle list with state / stage / status filters; create + import
State Rules	The state × building-type × cycle-years renewal-period lookup

Cycle list columns/CSV: building name, customer, state, building type, expiry date, days-to-expiry, current stage, status. A cycle drawer shows stage history and lets a user **advance the stage** or add a **free-text note**.

The stage state-machine

Internal stage codes (DB enum) with human labels (mam: "*t is time, t-30 means required 30 days — don't show t-30*", so labels read "30 days before · ..."):

```
T-180 (180 days before · Auto Alert)
T-150 (150 days before · Qualify)
T-120 (120 days before · Quote v1)
RESPONSE_CHECK · REENGAGE
T-90 (90 days before · Site Visit)
CONVERT_CHECK · LOST_POOL (Lost · Win-Back)
T-60 (60 days before · PO + 30%)
T-45 (45 days before · Dept Filing)
T-30 (30 days before · Inspection)
INSPECTION_CHECK · COMPLIANCE_FIX
T-15 (15 days before · NOC Issued)
T-0 (Expiry day · Final Pay)
T+30 (30 days after · Upsell)
CYCLE_CLOSE
```

Renewal-period lookup (formula)

The cycle length (how often a NOC must be renewed) comes from `fire_noc_state_cycle_rule`, resolved **most-specific-first**:

1. match `(state, building_type)`,
2. else `(state, NULL)`,
3. else the `__DEFAULT__` row (5 years, low-risk).

Seeded rules: UP & Maharashtra hospital/school = **1 year**; Karnataka = **2 years**; Delhi / Gujarat / Tamil Nadu = **3 years**; default = **5 years**.

Key fields

- **Property** (`fire_noc_property`): building registry (FK customers), inline `decision_maker_*` contact columns.
- **Cycle** (`fire_noc_cycle`): `current_stage`, `status` (`active` → `lapsed`), `expiry_date`, `days_to_expiry`. Indexed on `expiry/stage/status`.
- **Stage history** (`fire_noc_stage_history`): every transition with `triggered_by`; `UNIQUE(cycle_id, to_stage, entered_at)` so the cron can re-fire idempotently.
- **Master DB** (`master_noc_database`): RTI / past-client / broker / field-scrape lead pool (sources: `rti`, `past_client`, `broker`, `field_scrape`, `manual`).

Step-by-step workflow

1. Create a cycle (state, district, building type/name, address, pincode, expiry date, source) — or **bulk import** via CSV (with Levenshtein-style dedup helper).
2. The cycle enters at the stage matching its days-to-expiry.
3. Each hour the **auto-pilot cron** recomputes `days_to_expiry` and advances the stage when a threshold is crossed, logging the change with `trigger='hourly_cron'`.

4. The team works the funnel: quote (maker-checker), site visit, PO + 30% advance, department filing, inspection (pass → NOC issued; fail → compliance-fix tickets), final payment, then **T+30 upsell** (AMC / Annual Audit / Refilling / Training).
5. When expiry passes, status flips `active` → `lapsed`. Cycle ends at `CYCLE_CLOSE`.

Automations / crons / statuses

- `server/scripts/fireNocCron.js` — **hourly auto-pilot**. First run 60 s after boot, then every hour. For every non-terminal cycle it recomputes days-to-expiry, advances the stage across thresholds, flips `active` → `lapsed` on expiry, and logs to `fire_noc_stage_history`. Disable with `ERP_DISABLE_FIRE_NOC_CRON=1`.
- **Boot-time backfill** (`backfillOnceOnBoot`) runs the sync once for the 100+ rows imported before auto-sync existed (guarded by `app_settings.fire_noc_autosync_backfilled_v1`).
- The real advancement logic lives in `server/lib/fireNocSync.js` (`syncAllActiveCycles`).

Print outputs

Quotes print as an HTML view → browser **Save-as-PDF** (same pattern as Salary Slip / Vendor PO), per `docs/FIRE_NOC.md` (`/fire-noc/quote/:id/print`).

API endpoints

All under `/api/fire-noc`, guarded by `fire_noc`.

Method	Path	Purpose
GET	<code>/dashboard</code>	KPI tiles + funnel + upcoming expiries
GET	<code>/cycles</code>	Cycle list (state/stage/status/q filters)
GET	<code>/cycles/:id</code>	One cycle + stage history
POST	<code>/cycles</code>	Create a cycle
GET	<code>/cycles/import/template</code>	CSV import template
POST	<code>/cycles/import</code>	Bulk import (multipart upload)
POST	<code>/cycles/:id/advance</code>	Manually advance a cycle to a stage
PATCH	<code>/cycles/:id</code>	Edit cycle fields
POST	<code>/cycles/:id/note</code>	Add a free-text note
GET	<code>/state-rules</code>	State × building-type × cycle-years rules

DB tables

11 tables (idempotent `CREATE TABLE IF NOT EXISTS`), defined in `server/db/fireNocSchema.js`: `fire_noc_property`, `fire_noc_cycle`, `fire_noc_stage_history`, `fire_noc_outreach`, `fire_noc_quote`, `fire_noc_document`, `fire_noc_inspection`, `fire_noc_compliance_ticket`, `fire_noc_upsell`, `master_noc_database`, `fire_noc_state_cycle_rule`.

5.5 Sales Billing

There are **two** sales-bill flows, both writing to `sales_bills`. They are described separately below.

5.5.1 The 4-type sequential module (`/installation`)

BUSINESS PURPOSE

Generate client sales bills in **four sequential stages** that mirror the real project workflow, ending in a Final bill against which payment is received. The order is picked from the **Business Book** (customer + items auto-fill as reference); the **amounts are typed manually** on each bill. Design source: `SALES-BILLING-MODULE.md`.

The page **replaces the old Installation tracker** and is mounted at the `/installation` route (`App.jsx` renders `<SalesBilling/>` for `module="installation"`), so it reuses the `installation` permission for RBAC (Admin + Accounts: anyone with installation create/edit, or admin).

THE 4 BILL TYPES

#	Name	Trigger	Auto-filled (reference)	You type	Status set
1	Sales Order Bill	Order booked	Customer, items, ordered qty, rate (Business Book)	Bill amount, GST, date	ORDER BOOKED
2	Material Delivery Bill	Material delivered	Type-1 items + delivery challan	Delivered qty, amount (≤ Type 1), GST	MATERIAL DELIVERED
3	Installation Bill	DPR approved (<code>billing_ready</code>)	The approved DPR ref	Installation charges, GST	INSTALLATION COMPLETE
4	Final / Commissioning Bill	Testing & commissioning signed off	Testing/handover ref	Commissioning charges, GST, final total	READY FOR PAYMENT

In-module sequence is 1 → 3 → 4 . The create endpoint explicitly **rejects Type 2** with "*Type 2 (material delivery) is billed in Dispatch, not here.*" — so in this page material-delivery billing is handled by the Dispatch/delivery-note flow (§5.5.2), and the chain enforced here is 1 → 3 → 4. Each type requires every earlier type in the sequence to exist, and links to the previous via `previous_bill_id` .

PAGE TABS

Dashboard , Sales Order Bills , Material · PO vs Bill , DPR / Installation Bills . The DPR tab has a **Generate Installation Bills** button (creates Type-3 bills from approved DPRs). The list shows bill number, type, customer, project, amount, GST (`gst_amount @ gst_rate%`), total, status, approval status.

NUMBERING

SEPL/SB/26-27/001

Company format, **financial-year aware** (26-27 = FY Apr 2026 – Mar 2027), sequence **resets per FY**, zero-padded to 3. Generated by `nextBillNumber()` which scans existing SEPL/SB/<FY>/% numbers and increments.

GST FORMULA (SINGLE RATE PER BILL)

```
gst_amount = round2(amount × gst_rate / 100)
total_amount = round2(amount + gst_amount)
```

One GST % per bill (default 18). The auto-generated installation bills hard-code `gst_rate = 18` (installation service GST).

TYPE-4 AUTO-SUM

When creating the **Type-4 Final** bill, the amount is **pre-filled with the sum of the prior bills in the chain** (T1+T3) and the user adds the commissioning charge on top — the field stays **editable**.

PAYMENT → RECEIVABLES

When a **Type-4** bill is approved, a `receivables` row is auto-created (invoice = Type-4 total, `invoice_number` = the bill number, deduped against existing). Payments are then entered against it and tracked by the existing receivables ageing screen — no new payment screen. Payment entry is only enabled on Type 4.

VALIDATION RULES

- Can't create Type N without the earlier types in the `1→3→4` sequence.
- Type 2 is rejected here (Dispatch handles it).
- A bill can't be deleted if a later-type bill in its chain exists.
- A bill is `draft → approved` (single-step, Admin + Accounts).

STEP-BY-STEP WORKFLOW

1. **New Sales Bill** → pick a Business Book order → the system offers the next allowed type in that order's chain.
2. Reference fields auto-fill from the order/DPR; you type amount + GST + date.
3. Save → bill created as `draft` with computed GST/total and a status-log row.
4. Admin/Accounts **approve** (`draft → approved`).
5. For Type-3, DPRs flagged `billing_ready` can be swept into bills via **Generate Installation Bills** (also runs fortnightly via cron, below).

6. On Type-4 approval → receivables row created → payments tracked downstream.

AUTOMATIONS / CRONS

- `server/scripts/installationBillingCron.js` — **fortnightly (1st & 16th)**. Auto-generates Type-3 installation bills from approved/ `billing_ready` DPRs. Disable with `ERP_DISABLE_INSTALL_BILLING=1`.

API ENDPOINTS

All under `/api/sales-billing` (guarded by `installation`).

Method	Path	Purpose
GET	<code>/orders</code>	Business Book orders pickable for billing
GET	<code>/orders/:bbId</code>	One order (items/rates for reference)
GET	<code>/</code>	Bill list
GET	<code>/pending</code>	Pending bills
GET	<code>/material</code>	Material PO-vs-bill view
GET	<code>/:id</code>	One bill
POST	<code>/</code>	Create a bill (type 1/3/4; chain-validated)
PUT	<code>/:id/approve</code>	Approve (<code>draft</code> → <code>approved</code>); Type-4 spawns receivable
DELETE	<code>/:id</code>	Delete bill (blocked if a later type exists)
PUT	<code>/:id/sent</code>	Mark bill sent
POST	<code>/:id/payment</code>	Record payment (Type-4)
POST	<code>/generate-installation</code>	Sweep approved DPRs into Type-3 bills

DB TABLES

`sales_bills` (master — the 4-type columns `bill_type`, `business_book_id`, `customer_name`, `bill_status`, `previous_bill_id`, `reference_doc_type/no`, `approval_status`, `payment_status`, `project_name`, `gst_rate` are added by migration so legacy rows still work), `sales_bill_items` (line snapshot), `sales_bill_status_log` (audit trail). Reuses `business_book` + `po_items`, `delivery_notes`, `dpr` / `billing_ready`, `receivables` + `payments`.

5.5.2 The delivery-note-driven Sales Bill / Tax Invoice (procurement.js)

BUSINESS PURPOSE

This is the flow that produces the **actual printable SEPL Tax Invoice** to the client. A delivery note is converted into a sales bill; the printable invoice carries the GST split (CGST+SGST vs IGST) and the **Against-Delivery "Payment Due"** line. Recent work made the print **recover the client from the Business Book order via PO items** for the BILL-TO / SHIP-TO block and dropped the redundant "Taxable Value" column.

GST SPLIT FORMULA (CGST+SGST VS IGST)

The split is decided by **place of supply vs SEPL's Punjab GSTIN**:

```
intra-state (client state blank or Punjab) → CGST 9% + SGST/UTGST 9%, IGST 0
inter-state (a known other state)         → CGST 0 + SGST 0, IGST 18%
```

(SEPL GSTIN `03AASCS7836D2Z3` — state code 03 = Punjab.) For sales bills the percent is **recomputed at print time** from the client state so only the correct lines show:

```
cgst = subtotal × cgstPct / 100
sgst = subtotal × sgstPct / 100
igst = subtotal × igstPct / 100
grandTotal = subtotal + cgst + sgst + igst + freight + roundOff
```

AGAINST-DELIVERY "PAYMENT DUE" FORMULA

The order's **Against-Delivery** % comes from `business_book.payment_against_delivery` (parsed to a number). The print adds a highlighted line:

Payment Due (Against Delivery <pct>%) = Grand Total × pct / 100

For generated sales-bill line rates, mam's rule (2026-06-15) is "**BOQ item rate × against-delivery terms %**" when generating against delivery; the MD later directed that the printed sales bill bills the **full BOQ rate** (the Against-Delivery % then only drives the Payment-Due line, not the line rate). The client + Against-Delivery % are recovered from the **Business Book order** that the delivery note's PO items belong to.

KEY ENDPOINTS (UNDER `/API/PROCUREMENT`)

Method	Path	Purpose
GET	<code>/delivery-notes</code>	Delivery-note list
POST	<code>/delivery-notes</code>	Create a delivery note (multipart)
POST	<code>/delivery-notes/:id/sales-bill</code>	Attach/raise a sales bill on a DN
POST	<code>/delivery-notes/:id/generate-sales-bill</code>	Generate a sales bill from a DN
POST	<code>/auto-sales-bills/sweep</code>	Bulk auto-create sales bills
GET	<code>/delivery-notes/:id/print</code>	Printable Delivery Note / Sales Bill (Tax Invoice) HTML
GET	<code>/sales-bills</code>	Sales-bill list
POST	<code>/sales-bills</code>	Create a sales bill

PRINT OUTPUT

`GET /api/procurement/delivery-notes/:id/print` renders the full SEPL HTML invoice: GSTIN/PAN header, company block (Ludhiana HO + Noida + pan-India presence), BILL-TO / SHIP-TO recovered from the Business Book order, line items, subtotal, the conditional CGST/SGST/IGST rows, freight, round-off, Grand Total, and the Against-Delivery **Payment Due** line. Output to PDF via the browser.

DB TABLES / COLUMNS

Shares `sales_bills` (the procurement flow uses columns such as `cgst_pct` , `sgst_pct` , `igst_pct` , `freight_amount` , `round_off_amount` , `place_of_supply` , `state_code` , `reverse_charge` , `subtotal` , `grand_total_amount` , `items_json` , `sales_bill_number` , `sales_bill_pending` , added by migration), plus `delivery_notes` and `business_book / po_items` for the BILL-TO/SHIP-TO and Against-Delivery lookup.

6. Finance

The Finance domain of the SEPL ERP covers everything that moves money in or out of the company and everything that tracks money the company is owed or owes. It is implemented as six largely independent modules, each with its own React page, Express route file, and database tables, but they are wired together by a small set of shared helpers (`server/lib/cashSync.js`) and crons so that one user action (e.g. recording a client collection) ripples into the receivables ledger, the sales-bill payment status, and the daily cash-flow ledger without any manual reconciliation.

The six modules documented here are:

#	Module	React page	Route file	Primary tables
6.1	Cheques (Cheque FMS)	<code>client/src/pages/ChequeFMS.jsx</code>	<code>server/routes/cheques.js</code>	<code>cheques</code> , <code>cheque_actions</code>
6.2	Payables (Payment Required)	<code>client/src/pages/PaymentRequired.jsx</code>	<code>server/routes/paymentrequired.js</code>	<code>payment_requests</code> , <code>payment_approvals</code> , <code>payment_approval_overrides</code>
6.3	Collections (Receivables)	<code>client/src/pages/Collections.jsx</code> , <code>client/src/pages/admin/CollectionsMD.jsx</code>	<code>server/routes/collections.js</code>	<code>receivables</code> , <code>collections</code> , <code>collection_follow_ups</code>
6.4	Invoices / Billing	<code>client/src/pages/Billing.jsx</code>	<code>server/routes/installation.js</code> (+ <code>salesBilling.js</code> for sales bills)	<code>ra_bills</code> , <code>mb_bills</code> , <code>installation_bills</code> , <code>sales_bills</code>
6.5	Cash Flow	<code>client/src/pages/CashFlow.jsx</code>	<code>server/routes/cashflow.js</code>	<code>cash_flow_daily</code> , <code>cash_flow_entries</code> , <code>project_finance</code>
6.6	Expenses	<code>client/src/pages/Expenses.jsx</code>	(expenses route)	<code>expenses</code>

All Express routers in this domain mount `authMiddleware` (every endpoint requires a valid session) and most enforce a per-module `requirePermission(module, action)` check, where `action` is one of `view` / `create` / `edit` / `approve` / `delete` . The module keys used by the permission system are `cheques` , `payment_required` , `collections` , `cashflow` , `billing` , `installation` , and `expenses` .

A cross-cutting design principle visible throughout this domain is **audit fidelity**: rows that have downstream effects (a cheque with a logged action, a receivable with a recorded collection, an installation with bills) cannot be casually deleted — the API returns a `409 Conflict` and tells the user to cancel/reverse instead. Money amounts are stored in **raw rupees** (no lakh conversion) after the `pf_amounts_raw_rupees_v1` migration; only the display layer divides by 100000 when it wants "lakhs".

6.1 Cheques (Cheque FMS)

Business purpose

The Cheque Financial Management System tracks the full life-cycle of every physical cheque the company issues — from the moment it is raised, through the date it is presented, to its final outcome (cleared, bounced, stopped) or its post-dated hold and follow-up. It gives Finance a single register of cheques in flight, with action-due alerts so nothing is forgotten on the day a cheque becomes presentable.

Who uses it

Finance / accounts staff with the `cheques` permission. `view` to read the register, `create` to issue a cheque, `edit` to log actions, `delete` only for cheques that have no action history.

The 3-stage workflow

The module is built around a deliberate three-stage flow (documented at the top of both the route file and the page):

- Stage 1 — Raise / Issue.** Capture the cheque's core details: cheque number, payee, bank, cheque date, amount, optional photo of the cheque. The cheque enters `current_status = 'pending'` (or `'cancel'` if issued cancelled).
- Stage 2 — Action on/after the cheque date.** Once `cheque_date ≤ today` , the user logs an outcome via a dropdown: `clear` , `hold` , `bounce` , `stopped` (also `cancel` , `re_issue`). Every action requires **remarks**. A `hold` action additionally requires a **Next Date** (`next_date`), which is written to `cheques.hold_until` .

- Stage 3 — Follow-up on/after the hold date.** For held cheques, when `hold_until ≤ today` the cheque re-appears as "action due" and the user logs the next action (clear it, hold again, bounce, etc.).

Statuses

current_status	Meaning	Terminal?
pending	Issued, awaiting its cheque date / presentation	No
hold	Post-dated / held until <code>hold_until</code>	No
clear	Cleared by the bank	Yes
bounce	Bounced	Yes
stopped	Payment stopped	Yes
cancel	Cancelled	Yes

Once a cheque is in a terminal state (`clear` / `bounce` / `stopped` / `cancel`) the `/:id/action` endpoint refuses further actions. `re_issue` re-opens a held cheque back to `pending` (rare).

Key fields

- `cheque_number` , `payee_to` , `cheque_date` , `amount` (all required at creation).
- `bank_name` — constrained to an allow-list: `HDFC` , `ICICI` , `SBI` , `PNB CC` , `PNB Saving` , `Other` ; free-text `bank_other` when "Other".
- `photo_url` — optional uploaded image, stored under the shared `/data/uploads` dir (same one POs and Sales Bills use), served at `/uploads/ ...` .
- `issue_status` — `approved` (default) or `cancel` .
- `hold_until` — set when an action is `hold` , cleared otherwise.
- `raised_by` — FK to `users` .

Duplicate guard

On create, the module calls `utils/duplicateGuard` to block re-entry of the same physical cheque: a duplicate is defined as **same cheque number + same bank**. If a match is found the API returns a duplicate error instead of inserting.

Screen (ChequeFMS.jsx)

Filter tabs / status tiles at the top: **All**, **Action Due** (red — pending whose cheque date has arrived OR hold whose hold date has arrived), **Pending**, **On Hold**, **Cleared**, **Bounced**. Each tile shows a count; the "Total Value" tile changes to the action-due amount sum when Action Due is selected. A colour-coded `StatusBadge` renders each cheque's status (with the due date appended). The "+ Issue Cheque" button opens the Stage-1 form; a per-row action button opens the Stage-2/3 action modal (dropdown + remarks + conditional next-date).

Automations / computed fields

- `action_due` flag (computed in SQL on every list query): `1` when a `pending` cheque's `cheque_date ≤ today` OR a `hold` cheque's `hold_until ≤ today` . Drives the red "Action Due" highlight.
- `action_count` per cheque (sub-select count of `cheque_actions`).
- The audit report (`auditReport.js`) counts open cheques (`pending` + `hold`) as a finance exception metric.

Endpoints

Method	Path	Permission	Purpose
GET	<code>/api/cheques</code>	<code>cheques.view</code>	List with filters: <code>status</code> , <code>bank</code> , <code>search</code> , <code>action_due=1</code> . Returns <code>action_due</code> flag + <code>action_count</code> .
GET	<code>/api/cheques/:id</code>	<code>cheques.view</code>	One cheque with its full <code>cheque_actions</code> history.
POST	<code>/api/cheques</code>	<code>cheques.create</code>	Stage 1 — raise/issue (multipart, optional photo). Duplicate-guarded.
PUT	<code>/api/cheques/:id</code>	<code>cheques.edit</code>	Edit Stage-1 details — only while no action is logged (audit integrity).
DELETE	<code>/api/cheques/:id</code>	<code>cheques.delete</code>	Delete — only while no action is logged ; else returns 400 (cancel instead).
POST	<code>/api/cheques/:id/action</code>	<code>cheques.edit</code>	Stage 2 + Stage 3 — log an action (clear/hold/bounce/stopped/cancel/re_issue). Remarks required; hold requires <code>next_date</code> .
GET	<code>/api/cheques/stats/summary</code>	<code>cheques.view</code>	Dashboard cards: per-status count + amount totals, plus action-due count and amount.

DB tables

- **cheques** — `id`, `cheque_number`, `payee_to`, `bank_name`, `bank_other`, `cheque_date`, `amount`, `photo_url`, `issue_status`, `current_status`, `hold_until`, `raised_by`, `created_at` (`raised_at`), `updated_at` .
- **cheque_actions** — `id`, `cheque_id` (FK), `action`, `remarks`, `next_date`, `action_by` (FK), `action_at` . One row per logged action; provides the immutable audit trail.

6.2 Payables — Payment Required

Business purpose

Payment Required is the company's outbound-payment approval engine. Any employee who needs the company to pay money — a vendor purchase, a labour bill, transport, a salary advance, TA/DA travel reimbursement, a compliance fee — raises a **payment request** here. The request then walks a fixed multi-level approval chain. Only after the final release step is the payment booked as a cash-flow outflow.

Who uses it

- **Requesters** — any employee with `payment_required.create` (site engineers, purchase staff, etc.). A requester who is not an approver sees only their own requests.
- **Approvers** — the four named/role-based approvers in the standard flow (see below), who have `payment_required.approve` .
- **Admin** — sees everything and can override routing.

Scope rule: a user "sees all" requests iff they are admin OR one of their roles has `can_approve=1` or `can_see_all=1` on `payment_required` ; otherwise the list, stats, and single-GET are filtered to `created_by = self` .

The standard approval flow (mam 2026-06-11)

As of June 2026 **every category uses one standard chain** (the old per-category chains — HR / Purchase-head / Site-engineer plus an automatic Velocity Check and Billing Engineer — were retired). The chain is:

Step	Stage name	Approver	Type
1	L1 Approval (Accountant)	anyone holding the Accountant role	role-based
2	L2 Approval (Nitin Jain)	Nitin Jain	named person
3	L3 Approval (MD – Ankur Kaplesh)	Ankur Kaplesh (MD)	named person
5	Payment Release (Aanchal)	Aanchal	named person

Step numbers are deliberately 1, 2, 3, 5 (not 1,2,3,4) so requests already in flight on the legacy steps keep flowing; the retired Billing-Engineer step 4 was migrated to step 5. The same `STANDARD_FLOW` array is assigned to every category in the `WORKFLOW` map: `TA/DA`, `Purchase`, `Labour`, `Transport`, `Salary`, `Compliance` .

Named approvers are resolved to a live `users` row by name (exact match first, then a loose `LIKE`) via `resolveUserByName()` , so the flow survives across the local and production databases without hard-coded user ids.

A guarded one-time migration `pr_standard_flow_v1` runs at module load: it clears all old per-category routing overrides (so they don't shadow the new named approvers) and moves any request parked on retired step 4 to step 5. It is recorded in `app_migrations` so it executes exactly once.

Per-step routing overrides

Admins can re-route any (`category`, `step`) to a specific user via the **Approval Routing** matrix (a table at the bottom of the page). Overrides live in `payment_approval_overrides` (`category`, `step`, `user_id`) . When an override exists, **only** that user (or admin) can approve that step — there is no fallback to the role. Clearing an override (`user_id = null`) returns the step to its role/named default.

The function `canUserApproveStep(db, userId, category, step)` decides authority in this order:

1. Admin → always allowed.
2. Explicit override set → only the assigned user.
3. Named approver on the step (`approver_name`) → only the resolved user.
4. Otherwise role-based → any user whose role matches `approver_role` .

Step-by-step workflow

1. **Create** (POST /): requester fills the form. Required: employee name, category, amount, purpose. Category-specific mandatory fields/proofs are enforced:
 - **Purchase** → `vendor_name` required; `quotation_link` (Quotation / PO) required.
 - **TA/DA** → if mode is Bus/Train/Flight, a travel ticket upload is required; if Car/Bike, both Start KM and End KM photos are required.
 - **Required By Date** (if given) must be at least **today + 5 days** (no same-day payouts).
 - A request number `PR-<year>-NNNN` is generated via `nextSequence`. Push notifications fire to all approvers; an email event `payment.requested` is fired.
2. **Approve** (PUT /:id/approve): the current-step approver supplies **remarks (min 5 chars)** and optionally an **approved_amount**. The amount is **decrease-only** — it must be > 0 and $\leq$ `original requested amount` (an approver can pay less, never more). Each approval writes a `payment_approvals` row with `step_amount` (what this step agreed) and updates `payment_requests.approved_amount` to the latest agreed figure.
3. **Advance** (`advanceToNextStep`): on approval the request moves to the next step in the chain. When the **last** step (Payment Release) is approved, `status` becomes `final_approved` AND a cash-flow **outflow** entry is auto-created in `cash_flow_daily` / `cash_flow_entries` for `COALESCE(approved_amount, amount)` — i.e. the latest approver-agreed figure, not the original request.
4. **Reject** (PUT /:id/reject): any current-step approver can reject with remarks; `status` → `rejected`, with `rejected_by` / `rejected_at` / `rejection_remarks` recorded. An email event `payment.rejected` fires.

Email events `payment.approved` are fired per step. All emails route to the requester and the director (best-effort recipient resolution).

Status column

Pending → Approved → Paid as seen by the user, mapped from the underlying state:

- **Pending** — `status NOT IN ('final_approved', 'rejected')`, i.e. still walking the chain (the Step column shows which level it is parked on).
- **Approved / Paid** — `status = 'final_approved'` (all four sign-offs done; outflow booked).
- **Rejected** — `status = 'rejected'`.

Per-level "Approved by L1 / L2 / L3" views (mam 2026-06-15)

Beyond the standard tabs (**Dashboard, All Requests, Pending, Approved, Rejected**), the page offers per-level filters **Approved by L1 / L2 / L3**. Selecting one shows the requests that level has signed off, and the **Approval Amt** column shows the specific amount **that level** approved (from `payment_approvals.step_amount`, keyed by step). The list responses enrich each row with a `step_amounts` map `{ step: amount }`. The Approval Amt column shows the agreed amount once any level approves, otherwise the requested amount greyed out.

The Dashboard tab always renders the full standard flow (L1 → L2 → L3 → Release) as stages, even a stage with zero rows, so L3 (Ankur Kaplesh) is never hidden.

My Inbox

`GET /my-inbox` returns the pending requests whose **current step's approver is the logged-in user** (via override or role/named match), enriched with the workflow progress. `GET /my-inbox-count` returns just `{ count }` and is polled every 60s by the header bell badge. (The Inbox tab itself was removed from the page UI but the endpoints remain.)

Vendor pickers & request detail fields

The create form carries category-specific fields stored on `payment_requests`: `travel` (`travel_from_to`, `travel_dates`, `mode_of_travel`, `stay_details`, `ticket_upload`, `start_km` / `end_km` + photos), `purchase` (`indent_number`, `item_description`, `vendor_name`, `quotation_link`), `labour` (`labour_type`, `number_of_workers`, `work_duration`, `site_engineer_name`), `transport` (`vehicle_type`, `from_to_location`, `material_description`, `driver_vendor_name`), plus `site_id` / `site_name`, `department`, `contact_number`, `payment_mode`. Proofs missed at submit time can be attached later via `PATCH /:id/proof` (fields: `ticket_upload`, `km_photo`, `end_km_photo`, `quotation_link`, `attachment_link`).

Cash-flow integration & velocity helper

The route file also contains `isInTop3Velocity()` (cash-velocity ranking of projects) used historically by the retired auto Velocity Check; with the standard flow there is no longer any auto-advance or auto-reject. The cashflow tracker reads "Purchase value" from `payment_requests WHERE category='Purchase'` for a site (see 6.5).

Endpoints

Method	Path	Permission	Purpose
GET	/api/payment-required	view	List w/ filters: <code>status</code> , <code>category</code> , <code>step</code> , <code>search</code> , <code>date_from</code> , <code>date_to</code> . Scope-filtered. Enriched with <code>current_step_name</code> , <code>next_approver_name/role</code> , <code>last_approved_*</code> , <code>approvals_count/total</code> , <code>step_amounts</code> .
GET	/api/payment-required/stats	view	Counts + amounts (total / pending / approved / rejected, by category, by step). Scope-filtered.
GET	/api/payment-required/my-inbox	view	Requests awaiting this user's action.
GET	/api/payment-required/my-inbox-count	view	{ <code>count</code> } for the bell badge (60s poll).
GET	/api/payment-required/:id	view	Single request + <code>approvals[]</code> + <code>workflow</code> + <code>can_approve_current</code> . Ownership-checked.
POST	/api/payment-required	create	Create a request (validates proofs, vendor, +5-day rule).
PUT	/api/payment-required/:id/approve	approve	Approve current step (remarks ≥5 chars; optional decrease-only <code>approved_amount</code>).
PUT	/api/payment-required/:id/reject	approve	Reject current step (remarks required).
DELETE	/api/payment-required/:id	delete	Delete request + its approvals.
PATCH	/api/payment-required/:id/proof	view	Attach a proof URL after the fact.
GET	/api/payment-required/approval-routing	(auth)	Routing matrix: every category × step with default + override assignee.
PUT	/api/payment-required/approval-routing	admin only	Set/clear a (<code>category</code> , <code>step</code>) override.

DB tables

- **payment_requests** — the request header: `request_no`, `employee_name`, `site_id/site_name`, `department`, `category`, `amount`, `approved_amount`, `purpose`, `payment_mode`, `required_by_date`, `current_step`, `status` (`pending...` / `final_approved` / `rejected`), `rejection_remarks` / `rejected_by` / `rejected_at`, `created_by`, plus the category-specific columns listed above.
- **payment_approvals** — one row per approval/rejection: `request_id`, `step`, `step_name`, `action` (`approved` / `rejected`), `remarks`, `step_amount`, `approved_by`, `approved_at`. The per-step audit trail (Original 600 → L1 500 → ...).
- **payment_approval_overrides** — (`category`, `step`) PK → `user_id`, `updated_at`, `updated_by`. Routing overrides.
- **app_migrations** — `key` PK, `applied_at`. Guards one-time migrations like `pr_standard_flow_v1`.

6.3 Collections (Receivables)

Business purpose

Collections manages money the company is **owed** by clients — the accounts-receivable ledger. Each receivable is an outstanding invoice (or advance) against a client/site. The module tracks the billed amount, what has been received, what remains outstanding, how overdue it is (ageing), and the CRM follow-up activity behind each unpaid balance. Recording a collection here automatically updates the receivable, syncs the linked sales bill's payment status, and posts an inflow to cash flow.

Who uses it

- **CRM / collections staff** with the `collections` permission — create receivables, edit them, log follow-ups, record collections.
- **MD / management** — the read-only **Collections MD** dashboard (`CollectionsMD.jsx`) correlates outstanding money with chasing activity per site.

Main screen (Collections.jsx)

- **Ageing-bucket summary tiles** across the top — count + total per bucket.
- **Payment Target vs Received (with ageing)** panel — per-bucket Target / Received / Outstanding / Collection % (from `/target-summary`).
- **Receivables table** — columns: Site/Client, CRM, Invoice #, Invoice Date, Invoice Amount, Received, Outstanding, Due Date, Ageing (days), Bucket, Status, Follow-up Status, Owner. Each row exposes its individual collection installments (`payments[]`) and a `pms_tasks_count`.
- **Refresh Ageing** button → `POST /refresh-ageing` (same code path as the 01:00 cron).
- **Follow-up** modal — log a contact attempt; **Collect** modal — record a payment.

Ageing & status formulas (`server/lib/cashSync.js`)

Ageing (`calculateAgeing(dueDate)`):

```
days = max(0, floor((today - due_date) / 1 day))
bucket = '0-30'   if days ≤ 30
          '31-60'  if 30 < days ≤ 60
          '61-90'  if 60 < days ≤ 90
          '90+'    if days > 90
```

(No due date → `{ days: 0, bucket: '0-30' }`.)

Status colour (`getStatusColor(outstanding, ageingDays)`):

```
green  if outstanding ≤ 0           (fully collected)
red    if ageingDays > 60
yellow if ageingDays > 30
green  otherwise
```

Collection % (per bucket and overall): `received / target × 100`, where `target = SUM(invoice_amount)`, `received = SUM(received_amount)`.

Step-by-step: recording a collection (`POST /:id/collect`)

1. Insert a `collections` row (amount, date, payment_mode, transaction_ref, notes, collected_by).
2. Update the parent `receivables` row: `received_amount += amount`, `outstanding_amount = invoice_amount - received`, and recompute ageing/bucket/status.
3. **A9 sync** — if `receivables.invoice_number` matches a `sales_bills.bill_number`, recompute that bill's `payment_status` (`pending` / `partial` / `paid`) from received vs total.
4. **A14 cash-flow link** — ensure today's `cash_flow_daily` row exists (opening = yesterday's closing), then insert an **inflow** `cash_flow_entries` row tagged `reference_type='collection'`, and recompute the day's totals/closing balance.

Follow-ups

`POST /:id/follow-up` records a `collection_follow_ups` row (date, contact method, response, promised date/amount) and bumps the receivable's follow-up status to `contacted`. `GET /:id/follow-ups` lists them.

Collections MD dashboard (CollectionsMD.jsx + `/md-dashboard`)

One row **per site**, sorted by outstanding DESC, joining money with activity:

- target / received / outstanding / invoice_count / oldest_ageing,
- `pms_tasks_count` (CRM follow-up tasks raised for the site),
- `location_pings_7d` (GPS pings to the site in the last 7 days — proxy for "is anyone visiting"),
- `last_follow_up`, `next_planned_date`, `last_discussion`,
- `indents_count` / `indents_30d`, `materials_value_sent`, `dpr_count_30d`.

Silent Overdue Sites callout — flags rows where `outstanding > 0 AND oldest_ageing > 30 AND pms_tasks_count = 0 AND location_pings_7d = 0` (significant overdue money with zero recent chasing). MD can click the tile to filter to just those sites. Overall **Collection %** = `100 × received / target`.

DSO (Days Sales Outstanding)

DSO is surfaced on the CMD/Finance dashboards (`DashboardCMD*.jsx` , fed by `computeKpiPayload` in `auditReport.js`), not on the Collections page itself:

```
DSO = round( AR_outstanding / sales_in_window × window_days )
```

where `AR_outstanding` = `SUM(receivables.outstanding_amount)` and `sales_in_window` = `SUM(sales_bills.total_amount)` over the window (default 30 days, clamped 7–365). It feeds the Cash Conversion Cycle `CCC = DSO + DIO - DP0` . The CMD War Room shows DSO as a lever: "Drop DSO to 60d → estimated cash unlock ...".

Receivable sources & pickers

The Add/Edit modal's site dropdown lists **unique project names from the Business Book** (`/sites`), since "site name is project name". Creating a receivable auto-derives `client_name` from `site_name` when missing. Editing recomputes outstanding/ageing/status when amount or due date changes, and defensively coerces stale `owner_id` / `site_id` FKs to NULL to avoid FK-constraint failures on save.

Payment Advice print output

`GET /payment-advice?client=<name>&bbid=<id>` builds an **outstanding statement** for one client — every invoice with billed / received / pending, plus totals, and the client's company/address/GSTIN/state pulled from `business_book` . Rendered by the `PaymentAdvicePrint` page. (A separate **Debit Note** / Short Supply Notice print exists under Procurement — `DebitNotePrint.jsx` , `/debit-note/:id/print` — for rejected material / excess rate / short supply; it is procurement-owned, not part of the receivables ledger.)

Endpoints

Method	Path	Permission	Purpose
GET	<code>/api/collections</code>	view	Receivables list (filters: <code>status</code> , <code>ageing_bucket</code> , <code>client</code> , <code>search</code>) + per-row <code>payments[]</code> + <code>pms_tasks_count</code> .
GET	<code>/api/collections/md-dashboard</code>	view	Per-site money ↔ activity dashboard + silent-overdue flag.
GET	<code>/api/collections/sites</code>	view	Unique Business-Book project names for the site dropdown.
GET	<code>/api/collections/summary</code>	view	Totals: outstanding, by bucket, by status, top 10 clients, overdue (>30d).
GET	<code>/api/collections/payment-advice</code>	view	Client outstanding statement (Payment Advice print).
POST	<code>/api/collections</code>	create	Create a receivable.
PUT	<code>/api/collections/:id</code>	edit	Quick or full edit (recomputes ageing/outstanding).
GET	<code>/api/collections/target-summary</code>	view	Target vs Received vs Outstanding + Collection %, overall and per bucket.
DELETE	<code>/api/collections/:id</code>	delete	Delete — blocked (409) if any collection has been recorded.
POST	<code>/api/collections/:id/follow-up</code>	edit	Add a follow-up.
GET	<code>/api/collections/:id/follow-ups</code>	view	List follow-ups.
POST	<code>/api/collections/:id/collect</code>	edit	Record a payment received (updates receivable, sales bill, cash flow).
POST	<code>/api/collections/refresh-ageing</code>	edit	Recompute ageing for all outstanding receivables (manual trigger of the daily cron).

DB tables

- receivables** — `id` , `client_name` , `project_name` , `po_id` , `business_book_id` , `invoice_number` , `invoice_date` , `invoice_amount` , `received_amount` , `outstanding_amount` , `due_date` , `ageing_days` , `ageing_bucket` , `status` (colour), `follow_up_status` / `follow_up_date` / `follow_up_notes` , `escalation_level` , `owner_id` , `site_id` , `site_name` , `crm_name` , `next_planned_date` , `last_discussion` , `created_by` , timestamps.
- collections** — `id` , `receivable_id` (FK), `amount` , `collection_date` , `payment_mode` , `transaction_ref` , `notes` , `collected_by` , `created_at` . One row per installment received.

- `collection_follow_ups` — `id`, `receivable_id` (FK), `follow_up_date`, `contact_method`, `response`, `promised_date`, `promised_amount`, `followed_by`, `created_at`.

6.4 Invoices / Billing

Business purpose

The Billing page is the construction/installation billing workflow: it tracks the progressive bills raised against an installation job (RA → MB → Installation bills) and the outbound Sales Bills issued to the client. It is the bridge between "work done on site" and "money invoiced to the client". The legacy Installations page was superseded by the dedicated Sales Billing module (4-type sequential bills), but the RA/MB/Installation-bill scaffolding still lives in `installation.js` and is surfaced in the Billing page's tabs.

Who uses it

Billing / projects staff with `billing` / `installation` permissions.

Tabs (Billing.jsx)

Tab	Source	Key columns
Sales Bills	<code>/procurement/sales-bills</code>	Bill No, PO, Date, Amount, GST , Total , Payment status
RA Bills	<code>/installation/ra-bills</code>	Bill No, Date, Work Done, Previous, Current, Status
MB Bills	<code>/installation/mb-bills</code>	Bill No, Amount, Status
Installation Bills	<code>/installation/inst-bills</code>	Bill No, Amount, Payment status

The progressive-bill chain

These bills form a referential hierarchy enforced by the delete guards in `installation.js`:

```
Installation (job)
├── RA Bill      (Running Account - work_done / previous / current amounts)
│   └── MB Bill (Measurement Book - measurements + total)
│       └── Installation Bill (final amount)
```

- An **RA Bill** captures running-account progress: `work_done_amount`, `previous_amount`, `current_amount` (the new claim this period). Status is updatable.
- An **MB Bill** references an RA bill and records `measurements` + `total_amount`.
- An **Installation Bill** references an MB bill and carries the final `amount`.
- **Sales Bills** (issued to the client) carry `amount`, `gst_amount`, `total_amount`, `payment_status` and are produced by the Sales Billing module (`salesBilling.js`); the Billing page reads them for display/export.

GST split

Sales Bills store amount, GST, and total as separate columns (`amount` / `gst_amount` / `total_amount`). Across the ERP the standard GST rate is **18%** (the Business Book and Cash Flow tracker compute PO/Sale "with GST" as `Sale × 1.18`). For the receivables link, a sales bill is marked `paid` / `partial` / `pending` by comparing collected receipts to `total_amount` (the GST-inclusive figure).

Bill numbering

Bill numbers for complaints and handover certificates use the shared `nextSequence` helper (`CMP-NNNN`, `HC-NNNN`). RA/MB/Installation bill numbers are supplied by the caller on create; Sales Bills are numbered by the Sales Billing module (4-type sequential bill numbering — see the Sales Billing module docs).

Delete safety

Deletes cascade-guard up the chain (all return 409 when referenced):

- An installation cannot be deleted if RA/MB bills or handover certificates reference it.
- An RA bill cannot be deleted if MB bills reference it.
- An MB bill cannot be deleted if installation bills reference it.

Related sub-modules in `installation.js`

The same router also serves **Testing & Commissioning** (`/testing`), **Complaints** (`/complaints` , numbered `CMP-`), **Handover Certificates** (`/handover` , numbered `HC-`), and a generic **Payments** ledger (`/payments` — `type` , `reference_type` , `reference_id` , `amount` , `mode` , `ref`).

Automation — fortnightly installation billing cron

`server/scripts/installationBillingCron.js` (scheduled from `index.js`) runs on the **1st and 16th** of each month to auto-generate Type-3 installation Sales Bills, so progressive installation billing happens on a fixed fortnightly cadence without manual triggering.

Endpoints (`installation.js`)

Method	Path	Purpose
GET/POST/PUT/DELETE	<code>/api/installation</code>	Installations (jobs).
GET/POST/PUT/DELETE	<code>/api/installation/ra-bills</code>	RA bills.
GET/POST/PUT/DELETE	<code>/api/installation/mb-bills</code>	MB bills.
GET/POST/DELETE	<code>/api/installation/inst-bills</code>	Installation bills.
GET/POST/DELETE	<code>/api/installation/testing</code>	Testing & commissioning records.
GET/POST/PUT	<code>/api/installation/complaints</code>	Complaints (<code>CMP-</code> numbered).
GET/POST/PUT/DELETE	<code>/api/installation/handover</code>	Handover certificates (<code>HC-</code> numbered).
GET/POST	<code>/api/installation/payments</code>	Generic payments ledger.

Sales Bills are served by `/api/sales-billing` and read via `/api/procurement/sales-bills` .

DB tables

- `installations` — `po_id` , `site_address` , `start_date` , `end_date` , `assigned_to` , `status` , `notes` .
- `ra_bills` — `installation_id` , `bill_number` , `bill_date` , `work_done_amount` , `previous_amount` , `current_amount` , `status` .
- `mb_bills` — `ra_bill_id` , `installation_id` , `bill_number` , `measurements` , `total_amount` , `status` .
- `installation_bills` — `installation_id` , `mb_bill_id` , `bill_number` , `amount` , `payment_status` .
- `sales_bills` — `bill_number` , `po_id` , `bill_date` , `amount` , `gst_amount` , `total_amount` , `payment_status` (synced from collections).
- `payments` — generic `type` , `reference_type` , `reference_id` , `amount` , `payment_date` , `payment_mode` , `transaction_ref` , `notes` , `created_by` .
- Plus `testing_commissioning` , `complaints` , `handover_certificates` .

6.5 Cash Flow

Business purpose

The Cash Flow module has two faces: a **daily cash ledger** (opening/inflows/outflows/closing per day, like a running cash book) and a **project financial tracker** (per-project sale value, received, purchase, cash velocity, payment days, and the locked last-payment-date). Together they answer "how much cash do we have, how is it moving day-to-day, and which projects are draining vs filling the tank".

Who uses it

Finance staff and management with the `cashflow` permission. Non-admin users see only the projects where they are the assigned CRM, **unless** their role has `can_approve=1` or `can_see_all=1` on `cashflow` (then they see all, like Accountant/Auditor).

Tabs (`CashFlow.jsx`)

1. **Project Finance** — the per-project tracker (default).
2. **Daily Cash Flow** — the daily ledger.

Daily Cash Flow

Top cards for the selected date: **Opening** , **Inflows** (green, +), **Outflows** (red, -), **Closing** (purple). A last-7-days table and a per-date entries list. Add-entry form: type (inflow/outflow), category, description, amount, `payment_mode` , and **party_name** (**mandatory** — every

entry must be tied to a counterparty for audit/BB linking).

Rollover logic (`cash_flow_daily`): each day's row carries `opening_balance` , `total_inflows` , `total_outflows` , `closing_balance` . When a date has no row, the system derives `opening` = closing of the most recent prior day . Adding or deleting an entry recomputes `closing` = `opening` + `inflows` - `outflows` for that day.

```
closing_balance = opening_balance + total_inflows - total_outflows
```

Entries arrive from three sources: manual entry, collections (inflows, tagged `reference_type='collection'`), and Payment Required final-release (outflows, paid at `COALESCE(approved_amount, amount)`).

Project Finance tracker

One row **per project** (grouped by `business_book.company_name`), with columns: Sr, Project, CRM, **Sale ₹ (with GST)**, **Received ₹, Milestone, AR Cleared ₹, Aanchal ₹, Purchase ₹, Velocity, Live** (today), **Inv Days, Compl., Pmt, Total, Last Pmt Date**.

Sources & formulas:

- **Sale (with GST)** = `bb.po_amount` (enforced as `Sale × 1.18`), summed across all BB rows of the project. (A critical fix dropped a `sites` JOIN that was inflating SUMs when a BB row had >1 site.)
- **Received** = sum of `cash_flow_entries` inflows whose `party_name` matches the client (or manual `amount_received`).
- **Purchase** = sum of `payment_requests` WHERE `category='Purchase'` for the site (or a manual override `manual_purchase_value`).
- **Aanchal / AR Cleared / Milestone / Inv Days / Completion / Payment days** = manual fields stored in `project_finance` (raw rupees, no lakh conversion).
- **Total days (R)** = effective completion days (P) + payment days (Q).
- **Cash Velocity (M)** = `(Aanchal - Purchase) / Total Days / 100000` (displayed in lakhs/day). The page's column tooltip frames velocity as "received ÷ purchase, ≥1 = cash-positive".
- **Last Pmt Date — Option A locked target**: computed once as `today + total_days` and written to `project_finance.last_payment_target_date` ; it does **not** drift as the calendar moves. It is re-locked only when the user saves new days. Legacy rows are backfilled once on first read.

The page also exposes a **project breakdown** diagnostic (`/project-breakdown?company_name=...`) listing exactly which BB rows roll up into one project's totals, so management can audit a surprising sum (distinct clients sharing one company name).

Runway formula

Runway is shown on the CMD/War Room dashboards (computed in `server/utils/cmdDashboard.js`), not on the Cash Flow page itself:

```
dailyBurn    = SUM(outflows over last 30 days) / 30
runway_days  = round(cashOnHand / dailyBurn)      (null if burn = 0)
```

`cashOnHand` is the latest `cash_flow_daily.closing_balance` . The War Room colours the cash light green if runway > 90d, amber 30–90d, red < 30d.

Automations / crons

`server/scripts/cashFidelityCron.js` (audit items A7 + A14; disable with `ERP_DISABLE_CASH_CRON=1`) runs two daily jobs via drift-corrected `setTimeout` :

- **00:00 — cash_flow_daily rollover** (`ensureTodayCashFlowDaily`): create today's row with `opening` = yesterday's `closing` , so runway/dashboards don't show wrong numbers on zero-collection days.
- **01:00 — receivables ageing refresh** (`refreshAllAgeing`): recompute `ageing_days` / `bucket` / `status` for every outstanding receivable (same code as the on-demand Refresh Ageing button).

Endpoints

Method	Path	Permission	Purpose
GET	/api/cashflow/projects	view	Per-project tracker (CRM-scoped unless can_see_all). Returns projects[] + summary KPIs.
POST	/api/cashflow/projects/:id/update	edit	Save manual project-finance fields; re-locks last-payment date.
GET	/api/cashflow/project-breakdown	view	Which BB rows feed one project's roll-up.
GET	/api/cashflow/daily	(auth)	Daily ledger rows (date range filter).
GET	/api/cashflow/today	(auth)	Today's row (auto-creates with rolled-over opening).
GET	/api/cashflow/summary	(auth)	Selected-date opening/in/out/closing, last 7 days, MTD inflow/outflow.
POST	/api/cashflow/entry	(auth)	Add an inflow/outflow entry (party_name mandatory).
GET	/api/cashflow/entries/:date	(auth)	Entries for a date.
DELETE	/api/cashflow/entry/:id	(auth)	Delete an entry (recomputes the day).
POST	/api/cashflow/opening-balance	(auth)	Set/override a day's opening balance.

DB tables

- cash_flow_daily** — id, date (unique), opening_balance, total_inflows, total_outflows, closing_balance, created_by, timestamps. One row per day.
- cash_flow_entries** — id, daily_id (FK), date, type (inflow / outflow), category, description, amount, payment_mode, party_name, reference_type, reference_id, created_by, created_at.
- project_finance** — business_book_id (FK), amount_received, milestone_name, ar_cleared_value, aanchal_value, payment_investment_days, payment_days, manual_purchase_value, manual_completion_days, last_payment_target_date, updated_at. Manual per-project overrides (idempotent ALTER TABLE ... ADD COLUMN migrations add late columns).

6.6 Expenses

Business purpose

Expenses is a lightweight petty-cash / operational-expense register — small spends (cylinder purchase, site sundries, etc.) recorded with a category, amount, date, and an approval status. It is distinct from Payment Required (which is the formal multi-level outbound-payment chain); Expenses is for already-incurred or low-ceremony spends.

Who uses it

Staff with the `expenses` permission record their own spends; the page shows who submitted each and its status.

Main screen (Expenses.jsx)

- Status summary tiles** at the top, each summing the amount of expenses in that status (the page filters `expenses` by `status` and sums `amount`).
- Table** (bounded scroll, sticky header) — columns: Description, Category, Amount, Date, Submitted By, Status, Actions.
- Add/Edit modal** — fields: Category (required, from a dropdown), Description/Title (e.g. "Manga – Cylinder purchase"), Amount (₹, required), Date (required).

Key fields

`title / description`, `category`, `amount`, `expense_date`, `status`, `submitted_by` (resolved to `submitted_by_name` for display).

Workflow

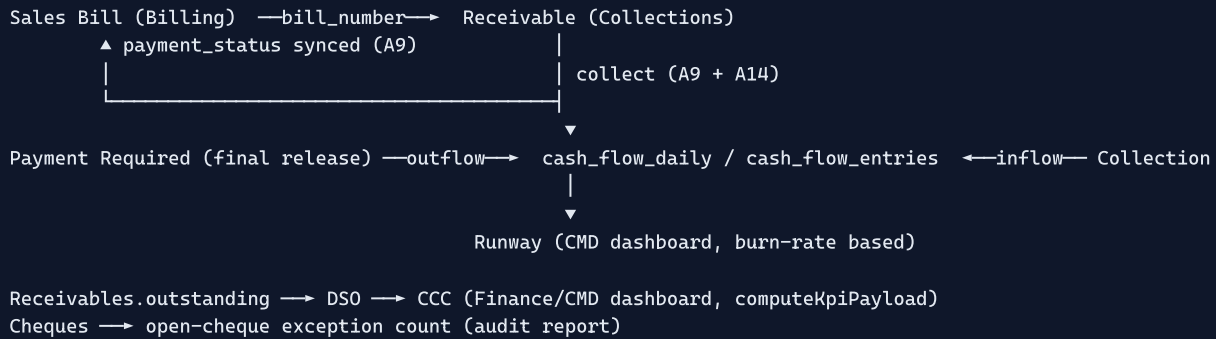
- Submit an expense (category + amount + date required).
- The expense appears in the register with a status; status tiles aggregate amounts per state.
- Expenses can be edited; export to CSV is available (Description, Category, Amount, Date, Submitted By, Status).

DB table

- expenses** — id, title / description, category, amount, expense_date, status, submitted_by, timestamps.

6.7 How the Finance modules interconnect

The chain below (mam's audit asks A7 / A9 / A14, implemented in `server/lib/cashSync.js`) makes the modules a single ledger rather than six silos:



- **Recording a collection** updates the receivable, syncs the matching sales bill's `payment_status`, and posts a cash-flow **inflow** — in one transaction.
- **Final-approving a Payment Required** posts a cash-flow **outflow** at the approver-agreed amount.
- The **00:00 cron** rolls `cash_flow_daily` forward (so runway is correct on quiet days); the **01:00 cron** re-ages all receivables (so DSO/ageing buckets are correct).
- **Runway** (burn-rate) reads the latest cash-flow closing balance; **DSO/CCC** read receivables outstanding vs sales-bill turnover; **open cheques** feed the daily audit exception report.

7. HRMS (Human Resources)

The HRMS is the people-side of the SEPL ERP. It runs end-to-end: raise a hiring requirement, source and screen candidates, interview, generate an offer, onboard, induct, train, track attendance on-site by GPS geofence, run monthly payroll with attendance-driven day rules, manage the employee master, score weekly performance (MIS scorecards), and maintain the sub-contractor hiring workflow + sub-contractor master.

There are **two parallel hiring stacks** in the codebase:

Stack	Frontend	Backend	Mount	Status
HR (Phase-1 ATS) — the live, full-featured hiring/onboarding/training system	<code>client/src/pages/HR.jsx</code>	<code>server/routes/hr.js</code>	<code>/api/hr</code>	Active, complete
HR System (legacy funnel) — an earlier candidate-funnel prototype	<code>client/src/pages/HRSystem.jsx</code>	<code>server/routes/hrSystem.js</code>	<code>/api/hr-system</code>	Partial (onboarding + training tabs are stubs)

This section documents both but treats **HR.jsx / hr.js** as the canonical hiring system. Each module below is its own `##` heading.

Source files referenced:

- Hiring (ATS): `client/src/pages/HR.jsx` (115 KB), ~~`server/routes/hr.js`~~ (137 KB)
- Hiring (legacy funnel): `client/src/pages/HRSystem.jsx`, `server/routes/hrSystem.js`
- Public offer accept: `server/routes/publicHr.js`, `client/src/pages/PublicOffer.jsx`
- Print: `client/src/pages/OfferLetterPrint.jsx`, `NDAPrint.jsx`, `EmploymentAgreementPrint.jsx`
- Sub-contractor hiring: `client/src/pages/SubconHiring.jsx`, `server/routes/subconHiring.js`
- Onboarding: `client/src/pages/Induction.jsx` (+ Induction tab in HR.jsx)
- Training: `client/src/pages/Training.jsx` (+ Training tab in HR.jsx)
- Attendance: `client/src/pages/Attendance.jsx` (95 KB), ~~`server/routes/attendance.js`~~ (51 KB)
- Payroll: `client/src/pages/Payroll.jsx`, `server/routes/payroll.js`, `client/src/pages/SalarySlipPrint.jsx`
- Employees: `client/src/pages/Employees.jsx`
- Performance: `client/src/pages/Scorecard.jsx`, `server/routes/scoring.js`, `server/db/seedScoring.js`
- Sub-contractor master: `client/src/pages/SubContractors.jsx`, `server/routes/subcontractors.js`
- HR automations cron: `server/scripts/hrAutomationsCron.js`

7.1 Hiring — Applicant Tracking System (ATS)

Business purpose

Run the full recruitment funnel in one place: a manager raises a hiring request, HR approves it and opens the position, candidates are added and screened (with optional auto-reject rules), interviewed by an employee panel, sent to an MD final round, and finally issued an auto-generated offer letter the candidate can accept online without logging in.

Who uses it

- Hiring managers** — raise hiring requests.
- HR** — approve requests, manage candidates, schedule interviews, run screening, generate offers, manage JD/screening/final-round content.
- Interviewers (employees)** — receive scheduled interviews and submit scorecards.
- MD** — runs the final round and the shortlist/reject + offer decision.
- Candidates** — receive offer link, accept/decline via public page.

Main screen & tabs

`HR.jsx` renders a top tab bar (all server-backed):

Tab	Purpose
Dashboard	Funnel counts, open positions, pipeline summary
Manpower Plan	Required vs actual manpower per project (see 7.1.7)
Candidates (ATS)	Kanban-style pipeline by stage
Hiring Requests	Manager-raised requirements + approvals
Job Descriptions	JD templates + per-position JDs
Screening Qs	Per-position / global screening question bank
Final-Round Qs	MD final-round question bank
Induction Content	Onboarding content manager (see 7.4)
Training Library	Training videos + assignments (see 7.5)

The **Candidates** tab groups candidates into pipeline columns:

Column	Label
applied	1 · APPLIED
screening	2 · SCREENING
interview	3 · INTERVIEW
final_round	4 · FINAL ROUND
selected	5 · SELECTED
rejected	REJECTED
on_hold	ON HOLD

7.1.1 Candidate stages (status machine)

The `candidates.status` column is a CHECK-constrained enum. The HR.jsx status badges map raw status → human label:

<code>status</code> value	UI label	Next action
lead	New Lead	schedule_interview
called	Called	screening
interview_scheduled	Interview Scheduled	interview_done
interview_done	Interview Done · pending decision	interview_decision
qualified (no MD date)	Shortlisted · MD round pending	schedule_md
qualified (MD date set)	MD Interview Scheduled	md_decision
offer_sent	Offer Sent	finalize
accepted	Offer Accepted	finalize
onboarded	Onboarded ✓	—
rejected	Rejected	—

An overlay flag `is_on_hold` (+ `hold_reason`) can be toggled at **any** pipeline stage without changing `status`.

7.1.2 Step-by-step hiring workflow

- Raise hiring request** — manager `POST /hr/hiring-requests` with department, position_title, num_openings, salary range, experience, employment_type, hiring_deadline, reporting_manager_id, JD blurb. Status starts `pending`.
- HR approves / rejects** — `POST /hr/hiring-requests/:id/approve` (or `/reject`). Separation of duties: the cron will not nudge the requester to approve their own request.
- Add candidate** — `POST /hr/candidates` (optionally linked back to a `hiring_request_id`). Resume can be auto-parsed first via `POST /hr/candidates/parse-resume` (file upload → text extraction → pre-filled fields; file is saved even if parse fails).
- Screening** — HR submits screening answers `POST /hr/candidates/:id/screening-answers`. Auto-reject rules evaluate each answer; a tripped rule sets `eligibility_status` / `eligibility_reason` and can flag the candidate as ineligible.

5. **Schedule interview** — POST `/hr/candidates/:id/schedule-interview` with `interviewer_id` (an employee) + `interview_date`. Status → `interview_scheduled`. Writes a `candidate_events` timeline row.
6. **Interview decision** — POST `/hr/candidates/:id/interview-done` with decision:
 - `shortlisted` → status `qualified` (waiting for MD round)
 - `rejected` → status `rejected`
 - `on_hold` → status `interview_done` (HR returns later)
 - Interviewer may also file a scorecard (see 7.1.4).
7. **Schedule MD round** — POST `/hr/candidates/:id/schedule-md-interview` sets `md_interview_date`; status stays `qualified` (the date being set marks the final round).
8. **MD decision** — POST `/hr/candidates/:id/md-decision` with decision:
 - `shortlisted` → status `offer_sent`; the offer letter is **auto-generated** from `offered_position` (required), `offered_salary`, `joining_date`, `reporting_to`, optional `salary_breakup` JSON. A single-use **offer_token** (32 random bytes, base64url ≈ 43 chars) is generated and `offer_sent_at` is stamped. Re-saving the decision preserves the existing token so already-shared accept links keep working. There is **no upload path** — letters are always system-generated.
 - `rejected` → status `rejected`.
9. **Candidate accepts/declines** — via the public link (see 7.2), or HR marks final state via POST `/hr/candidates/:id/finalize` (`accepted` / `onboarded` / `rejected`).

Every transition writes to `candidate_events` (GET `/hr/candidates/:id/timeline` returns the chronological audit log). Free-form CSV tags (PUT `/hr/candidates/:id/tags`) and the hold toggle (POST `/hr/candidates/:id/hold`) are also logged.

7.1.3 Hiring requests & approvals

`hiring_requests` holds: `department`, `position_title`, `num_openings`, `salary_min/max`, `experience_required`, `employment_type` (`full_time` | `part_time` | `contract` | `internship` | `freelance`), `hiring_deadline`, `reporting_manager_id`, `job_description`, `status` (`pending` | `approved` | `rejected` | `closed`), `approval_notes`, `requested_by` (+ denormalised `requested_by_name`), `approved_by/approved_at`, `closed_at`.

Approval is a single HR/admin action — there is no multi-level chain on hiring requests (unlike Payment Required, which uses L1→L2→L3).

7.1.4 Job descriptions, screening, scorecards, final-round bank

- **JD templates** (`jd_templates`) — reusable skeletons (Site Engineer, Sales Executive, etc.), `template_content` is a JSON blob, `is_default`.
- **Job descriptions** (`job_descriptions`) — one JD per position, can be derived from a template and linked to the hiring_request. Stores two flavours side by side: `internal_jd` (full HR/manager detail) and `public_job_post` (sanitised for Naukri / LinkedIn). Status: `draft` | `published` | `archived`.
- **Screening questions** (`screening_questions`) — per-position (`hiring_request_id` set) or global (NULL). Types: `mcq` | `descriptive` | `yes_no` | `number`. Optional **auto-reject rules engine**: `auto_reject_op` ∈ `gt`, `lt`, `gte`, `lte`, `eq`, `neq`, `contains`, `not_contains`, `in`, `not_in`, with `auto_reject_value` and `auto_reject_reason`. A NULL op means info-only. Submitting a screening form deletes-and-reinserts the candidate's answers so re-screening doesn't double-count.
- **Interview scorecards** (`interview_scorecards`) — one row per (candidate × interviewer × stage). Stage = `first` (interviewer round) or `final` (MD round). Four 1–5 dimension scores (technical, communication, culture_fit, problem_solving) plus an explicit verdict `overall_recommend` ∈ `strong_yes`, `yes`, `maybe`, `no`, `strong_no`, and strengths/weaknesses/overall_feedback.
- **Final-round question bank** (`final_round_questions`) — curated MD/panel questions by category (Leadership / Ownership / Decision Making / Conflict Management / Team Handling), `for_role` free-text tag, difficulty (`easy` | `medium` | `hard`). GET `/hr/final-round-questions/pick` pulls a set.

7.1.5 Pre-onboarding document checklist

`candidate_docs` tracks one row per (candidate × doc_type). Standard doc_type values: `aadhaar`, `pan`, `resume`, `experience`, `bank`, `photo`, `education`, `other` (free-text, so admin can add custom). Status: `pending` | `received` | `verified` | `rejected`, with `verified_at/verified_by`. Managed via GET/POST `/hr/candidates/:id/docs`, PUT/DELETE `/hr/docs/:id`.

7.1.6 Notifications

`notifications` rows are created both by direct admin actions and by the HR automations cron (7.1.8). Fields: `user_id` (recipient), type (`interview_reminder` | `offer_expiry` | `approval_pending` | `training_assigned` | `generic`), title, body, link_url, channel_sent (CSV `in_app,email`), dedupe_key, read_at. Read/clear via GET `/hr/my-notifications`, PUT `/hr/notifications/:id/read`, POST `/hr/notifications/mark-all-read`.

7.1.7 Manpower Plan tab

GET /hr/manpower-plan joins business_book (projects), sites , and DPR manpower (dpr_contractors.manpower , falling back to legacy dpr.contractor_manpower) to show **required vs actual** manpower per project. Required is driven by a per-project value slab; admins can override required (PUT /hr/manpower-plan/required) and category (PUT /hr/manpower-plan/category). Stored in manpower_required_overrides and manpower_project_settings .

7.1.8 Automation — HR automations cron (every 30 min)

server/scripts/hrAutomationsCron.js runs 10 s after boot then every **30 minutes** (setInterval , 30-60-1000 ms). Disabled via ERP_DISABLE_HR_CRON=1 . Each run scans three triggers and creates in-app notifications (+ best-effort email via server/lib/email.js), with a dedupe_key so re-runs don't spam:

Scanner	Trigger	Recipient	dedupe_key
Interview Reminder	candidate interview_scheduled with interview_date in next 24 h	the interviewer (employee→user)	interview_reminder:<cand>:<date>
Offer Expiry Reminder	offer_sent , offer_sent_at ≥ 7 days old, no accept/decline	all HR users (re-triggers daily)	offer_expiry:<cand>:<dayBucket>
Approval Pending Alert	hiring_requests still pending > 24 h	all HR users except the requester	approval_pending:<req>:<dayBucket>

"HR users" = active users where role is admin , or department/role name contains hr . Email send failures never block the in-app notification.

7.1.9 ATS API endpoints (/api/hr)

Method	Path	Purpose
GET	/manpower-plan	Required vs actual manpower
PUT	/manpower-plan/required	Override required count
PUT	/manpower-plan/category	Override category settings
POST	/candidates/parse-resume	Upload + parse resume
GET	/candidates	List candidates
POST	/candidates/check-duplicates	Dedupe check
POST	/candidates	Create candidate
PUT/DELETE	/candidates/:id	Edit / delete
POST	/candidates/:id/schedule-interview	Stage 1 interview
POST	/candidates/:id/interview-done	Interview decision
POST	/candidates/:id/schedule-md-interview	Schedule MD round
POST	/candidates/:id/md-decision	MD decision + auto-offer
GET	/candidates/:id	Candidate detail (offer print)
POST	/candidates/:id/finalize	Final accepted/onboarded/rejected
GET	/candidates/:id/timeline	Audit timeline
PUT	/candidates/:id/tags	Tags
POST	/candidates/:id/hold	Toggle on-hold
GET	/candidates/stats	Funnel stats
GET/POST	/hiring-requests	List / create
GET	/hiring-requests/:id	Detail
PUT/DELETE	/hiring-requests/:id	Edit / delete
POST	/hiring-requests/:id/approve	Approve
POST	/hiring-requests/:id/reject	Reject
POST	/hiring-requests/:id/close	Close position
GET/POST/PUT/DELETE	/jd-templates[:id]	JD templates
GET/POST/PUT/DELETE	/job-descriptions[:id]	Job descriptions
GET/POST	/candidates/:id/scorecards · /scorecard	Interview scorecards
DELETE	/scorecards/:id	Remove scorecard
GET	/final-round-questions · /pick	Final-round bank
POST/PUT/DELETE	/final-round-questions[:id]	Manage bank
GET/POST/PUT/DELETE	/screening-questions[:id]	Screening bank
POST/GET	/candidates/:id/screening-answers	Submit / read answers
GET/POST	/candidates/:id/docs	Doc checklist
PUT/DELETE	/docs/:id	Update / delete doc
GET	/my-notifications	My notifications
PUT	/notifications/:id/read	Mark read
POST	/notifications/mark-all-read	Mark all read
GET	/dashboard	HR dashboard counts

(Note: `hr.js` also hosts employees, sub-contractors, expenses, and checklists routes — covered in their own sections.)

7.1.10 ATS DB tables

`candidates`, `candidate_events`, `candidate_docs`, `hiring_requests`, `jd_templates`, `job_descriptions`, `interview_scorecards`, `screening_questions`, `screening_answers`, `final_round_questions`, `notifications`. (Plus `induction_items`, `training_videos`, `training_assignments` — see 7.4 / 7.5.)

Key `candidates` columns (base + migrations): `name`, `phone`, `email`, `source` (`facebook` | `naukri` | `linkedin` | `reference` | `other`), `position`, `status`, `resume_file`, `address`, `linkedin_url`, `tags`, `is_on_hold`/`hold_reason`, `hiring_request_id`, `interviewer_id`, `interview_date`/`notes`/`decision`, `md_interview_date`/`notes`/`decision`, `eligibility_status`/`reason`/`screened_at`, `offered_position`, `offered_salary`, `joining_date`, `reporting_to`, `salary_breakup`, `offer_letter_file`, `offer_sent_at`, `offer_token`, `offer_accepted_at`, `offer_declined_at`, `offer_response_note`.

7.2 Public offer-accept (no-login) & print outputs

Public offer flow

When the MD shortlists a candidate, a single-use `offer_token` is minted. The candidate receives a link to `/offer/:token` (rendered by `PublicOffer.jsx`). This path is served by `server/routes/publicHr.js`, mounted at `/api/public/offer/*` **before** the global auth middleware, so it needs no SEPL login.

Security model:

- Token = 32 random bytes (base64url, $\approx$ 256 bits) → not brute-forceable.
- Single-use: response only allowed while status is `offer_sent`; once accepted/declined the POST returns 409 with the prior decision.
- Token length must be $\geq$ 16 chars to be accepted.
- Sensitive offer fields (salary, joining, breakup) ARE returned — that's the point — but no other candidate's data is exposed.

Method	Path	Body	Effect
GET	<code>/api/public/offer/:token</code>	—	Returns offer for inline render
POST	<code>/api/public/offer/:token/respond</code>	<code>{decision: 'accept' 'decline', note?}</code>	<code>accept</code> → status <code>accepted</code> + <code>offer_accepted_at</code> ; <code>decline</code> → status <code>rejected</code> + <code>offer_declined_at</code> ; logs a <code>candidate_events</code> row (<code>user_name</code> = candidate via link)

`hrSystem.js` has its own variant: `POST /api/hr-system/offers/:id/send` (stamps `offer_sent_at` + token) and `POST /api/hr-system/offers/accept/:token`.

Print pages

Page	Source	Notes
Offer Letter	<code>OfferLetterPrint.jsx</code>	Auto-generated from candidate fields. SEPL template: subject line, body, and a CTC table (Basic Pay / Conveyance Allowance / HRA / ...). <code>salary_breakup</code> JSON overrides the default split; otherwise Basic = total and allowances render as "As per actual". Reads <code>GET /hr/candidates/:id</code> .
NDA	<code>NDAPrint.jsx</code>	Standard non-disclosure agreement print.
Employment Agreement	<code>EmploymentAgreementPrint.jsx</code>	Standard employment agreement print.

These are print-ready (browser print → PDF) standalone pages.

7.3 Sub-contractor Hiring (14-step / 2-phase tracker)

Business purpose

Track, per site, the journey of awarding work to a sub-contractor: from project kickoff and BOQ scope split, through vendor sourcing, pre-qualify, RFQ/negotiate, award, to onboarding (KYC, MSA+NDA, safety induction, work order, mobilization, site entry). A manual workflow tracker with per-step file uploads, a vendor candidate list, an award action, and two "PASS" gates with loop-back.

Who uses it

- **Phase 1 (Pre-Award, steps 1–7)** — owner: **PM + Procurement**.
- **Phase 2 (Onboarding, steps 8–14)** — owner: **Legal + HR + PM**.

The 14 steps / 2 phases (STEP_META)

#	Phase	Step	Gate
1	pre_award	Project Kickoff	
2	pre_award	BOQ Scope Split	
3	pre_award	Source Vendors	
4	pre_award	Pre-Qualify	gate: prequalify — vendor score ≥ 7 → step 5, else loop to 3
5	pre_award	RFQ & Negotiate	
6	pre_award	Award Decision	
7	pre_award	LOI to Vendor	→ triggers Phase 2
8	onboarding	KYC & Vendor Master	
9	onboarding	MSA + NDA	
10	onboarding	Safety Induction	
11	onboarding	Mobilization Plan	gate: docs — docs complete → 12, else loop to 8
12	onboarding	Issue Work Order	
13	onboarding	Mobilization Advance	
14	onboarding	Site Entry & Setup	

phase is derived: step ≤ 7 = pre_award , else onboarding . A workflow's phase auto-recomputes from current_step .

Workflow steps

1. **Create workflow** — POST /api/subcon-hiring with site_id + scope_description. Starts at step 1, phase pre_award , status active .
2. **Advance a step** — POST /:id/step/:no to set per-step status (pending | in_progress | done | blocked), notes, and a numeric decision_value (e.g. the Step-4 vendor score).
3. **Add vendor candidates** — POST /:id/candidate (vendor from the sub-contractor master, quote_amount, qualification_score 0–10). Edit/remove via PATCH/DELETE /candidate/:cid .
4. **Run a gate** — POST /:id/gate/:gate (prequalify or docs). Pass advances; fail loops back per the table above.
5. **Upload files per step** — POST /:id/step/:no/upload (max 20 MB, stored under data/uploads/subcon-hiring/). List/serve/delete via /file/:fileId .
6. **Award** — POST /:id/award/:cid sets awarded_vendor_id and marks the winning candidate awarded .
7. **Delete** — DELETE /:id .

API endpoints (/api/subcon-hiring)

Method	Path	Purpose
GET	/	List workflows (with site + awarded vendor names)
GET	/steps-meta	STEP_META for UI labelling
GET	/:id	Detail (workflow + steps + candidates + files)
POST	/	Create workflow
POST	/:id/step/:no	Update a step
POST	/:id/gate/:gate	Run a gate (prequalify/docs)
POST	/:id/step/:no/upload	Upload a step file
GET/DELETE	/file/:fileId	Serve / delete a file
POST	/:id/candidate	Add vendor candidate
PATCH/DELETE	/candidate/:cid	Edit / remove candidate
POST	/:id/award/:cid	Award to a candidate
DELETE	/:id	Delete workflow

All routes require `subcon_hiring` permission (view/create/edit/delete).

DB tables

`subcon_hiring` (site_id, scope_description, current_step, phase, status, awarded_vendor_id), `subcon_hiring_steps` (step_no, status, decision_value, completed_by/at), `subcon_hiring_files` (per step file metadata), `subcon_hiring_candidates` (vendor_id, quote_amount, qualification_score 0–10, status `shortlisted` | `rejected` | `awarded`).

7.4 Onboarding / Induction

Business purpose

Give new joiners a structured induction: founder message, company culture, HR policies, IT-security, and SOPs, delivered as text blocks, videos, PDFs, or links. Admin curates the content; new employees consume it.

Screen

`Induction.jsx` (standalone page) plus the **Induction Content** tab inside `HR.jsx`. Content is grouped by `section` : `founder` | `culture` | `hr_policies` | `it_security` | `sop` (free-text, so custom sections are possible).

Key fields (`induction_items`)

`section`, `title`, `content_type` (`text` | `video` | `pdf` | `link`), `content_url` (for video/pdf/link), `content_text` (body for text), `order_index`, `is_active`, `created_by`.

API (/api/hr)

Method	Path
GET	/induction
POST	/induction
PUT	/induction/:id
DELETE	/induction/:id

Note: the legacy `HRSYSTEM.jsx` Onboarding tab is a stub ("Document collection, BG verification, joining checklist — wiring in the next sprint"). The live onboarding doc-collection is the `candidate_docs` checklist (7.1.5) plus this induction content.

7.5 Training (LMS)

Business purpose

A lightweight learning library: admin uploads training videos categorised by type, optionally targeting departments/roles and marking some mandatory; videos are assigned to employees and completion is tracked.

Screen

`Training.jsx` + the **Training Library** tab in `HR.jsx`. "My Training" view shows an employee's assigned videos.

Key fields

- **training_videos** — title, description, video_url (YouTube/Vimeo embed or direct file), training_type (`product` | `process` | `communication` | `sop` | `other`), duration_minutes, target_dept (CSV, NULL = any), target_role (CSV), is_mandatory, is_active, created_by.
- **training_assignments** — one row per (employee × video), UNIQUE. Status flow: assigned → started → completed (or skipped), with assigned_at / started_at / completed_at / completion_note / assigned_by.

Workflow

1. Admin adds a video (`POST /hr/training/videos`).
2. Admin assigns to an employee (`POST /hr/training/videos/:id/assign`) — may also create a `training_assigned` notification.
3. Employee opens "My Training" (`GET /hr/training/mine`), starts (`POST /hr/training/assignments/:id/start`) and completes (`POST /hr/training/assignments/:id/complete`).

API (`/api/hr`)

Method	Path
GET/POST	<code>/training/videos</code>
PUT/DELETE	<code>/training/videos/:id</code>
POST	<code>/training/videos/:id/assign</code>
GET	<code>/training/videos/:id/assignments</code>
DELETE	<code>/training/assignments/:id</code>
GET	<code>/training/mine</code>
POST	<code>/training/assignments/:id/start</code>
POST	<code>/training/assignments/:id/complete</code>

7.6 Attendance

Business purpose

GPS-geofenced, photo-stamped punch in/out for site staff, plus live location tracking throughout the day, leave requests/approvals, an admin attendance grid, and a geofence-violation audit. Attendance is the raw input the payroll engine consumes.

Who uses it

- **Site staff / engineers** — punch in/out from their assigned site; the app sends live location pings while open.
- **HR / admins** — admin-mark days, manage geofences, approve leaves, view the grid, dashboard, and geofence audit.

Punch-in rules (geofence)

`POST /attendance/punch-in` requires `latitude` / `longitude` (GPS must be on) and is **mandatory geofenced**:

1. At least one active `geofence_settings` row must exist (else punch is blocked with "Contact admin to add geofence areas").
2. The haversine distance from each active geofence centre is computed. **GPS-accuracy tolerance**: the reported `accuracy` is clamped to ≤ 500 m and subtracted from the raw distance — $\text{dist} - \text{accuracy} \leq \text{radius_meters}$ counts as on-site (default radius 200 m). This stops indoor/cloudy GPS noise from blocking genuinely-on-site staff.

3. If outside all geofences, the punch is rejected with the metres-to-nearest site and the radius, plus a hint to move outdoors if accuracy is poor.
4. **Late check** uses IST (UTC+5:30) + `payroll_settings.late_after_time` (default `09:46`). If late, the row's status is stored as `late`. (Fixes a UTC-vs-IST bug where 10:23 IST = 04:53 UTC looked not-late.)

Already punched in today → `400 Already punched in today`.

Punch-out rules (strict geofence)

`POST /attendance/punch-out` is **equally strict** (mam: out-punch must also obey geofencing). Same accuracy-buffered radius test; if off-site the punch-out is blocked with metres-from-site. There is no walk-away allowance — staff must punch out **first**, then leave. Total hours = (punch_out – punch_in); a day under **4 h** is downgraded to `half_day`.

Live location tracking (30-second pings)

The Attendance page calls `trackLocation` on a `setInterval` of **30 seconds** (`30 * 1000`). Each ping `POST /attendance/track-location`:

- Same accuracy buffer (clamped 500 m); if inside a geofence the ping is tagged with that `site_name`, else `Outside`.
- A `gps_off=true` heartbeat (page open, network alive, but no GPS fix) is stored with NULL lat/lng and `site_name='GPS_OFF'` so the admin tracking page shows it as a distinct red card.

Admin views history via `GET /attendance/track/:userId/:date`. Rows land in `location_tracking` (user_id, date, time, lat/lng, address, site_name).

Geofence settings & audit

- `GET/POST /attendance/geofence`, `PUT/DELETE /attendance/geofence/:id` — manage `geofence_settings` (site_id, site_name, latitude, longitude, radius_meters default 200, active).
- `GET /attendance/audit/geofence-violations` — surfaces punches/pings that fell outside, for the "are staff really at site?" audit.

Admin marking & grid

- `POST /attendance/admin-mark` and `/admin-mark-bulk` set a day's status directly (sets `admin_marked=1`, no punch time). Payroll honours this over punch logic.
- `GET /attendance/grid` — month grid for all users.
- `GET /attendance/`, `/dashboard`, `/report` — list, dashboard, reports.
- `GET /attendance/my-today`, `/my-month`, `/my-history` — self views.
- `POST /attendance/link-login` — links an attendance/employee record to a user login.

Leave rules

`POST /attendance/leave` creates a `leave_requests` row. Leave types: `casual` | `sick` | `earned` | `half_day` | `short_leave` | `comp_off`.

- **Multi-day leave** — `days = ceil(to_date - from_date) + 1`.
- **Short leave** — requires `from_time` / `to_time`; hours are computed; **monthly cap is 4 hours** (sum of non-rejected short leaves in the month). Exceeding it is rejected with remaining-hours feedback. `days = 0`.
- A `leave.requested` email event fires to the requester + director.

Approval: `PUT /attendance/leave/:id/approve` (`status` = approved/rejected + remarks) requires `attendance:approve`. Visibility scope: approvers/admins (or roles with `can_approve/can_see_all` on `attendance`) see all leaves; plain users see only their own.

API endpoints (/api/attendance)

Method	Path	Purpose
GET	/my-today /my-month /my-history	Self views
GET	/ /dashboard /grid /report	Admin views
POST	/punch-in /punch-out	Geofenced punch
POST	/track-location	Live 30 s ping
GET	/track/:userId/:date	Location history
POST	/admin-mark /admin-mark-bulk	Manual marking
POST	/link-login	Link record → user
GET/POST	/geofence	List / add geofence
PUT/DELETE	/geofence/:id	Edit / delete geofence
GET	/audit/geofence-violations	Violation audit
POST	/leave	Request leave
GET	/leaves	List leaves (scoped)
PUT	/leave/:id/approve	Approve / reject
PUT/DELETE	/leave/:id	Edit / delete leave
DELETE	/:id	Delete attendance row

DB tables

attendance (punch in/out time, lat/lng, address, photo, site, total_hours, status present | half_day | short_day | absent | late | leave | holiday, admin_marked), geofence_settings, location_tracking, leave_requests .

7.7 Payroll

Business purpose

Compute monthly salary per employee from attendance + leaves using admin-tunable day rules, split into SEPL salary components, apply late penalties / advances / food allowance / overtime, and lock a finalised snapshot so historical payslips don't drift when attendance is later edited.

Who uses it

- **Payroll admins** — review the monthly run, set advances/food/overrides, finalise the month, mark paid.
- **Employees** — view their salary slip.

Screen / tabs (Payroll.jsx)

- **Monthly** — per-employee salary table for a chosen YYYY-MM .
- **Leaves** — leave-balance view.
- **Settings** — admin-only editor for every payroll rule (single-row payroll_settings , id=1) grouped into: Attendance rules, Penalty, Working days/leaves, Overtime, and Salary component %s.

Settings (rules, all admin-tunable)

Setting	Default	Meaning
late_after_time	09:46	Punch after this = late mark (full day, counts toward grace)
half_day_after_time	10:00	Punch after this = half day, no grace
min_hours_half_day	4	Work $\geq$ this = full day; less = half day
min_hours_full_day	8	(legacy; current rule uses the 4 h floor)
skip_half_day_if_short_leave	1	Short leave that day cancels late/half-day docking
late_grace_count	3	First N late marks/month are free
late_per_minute_rate	20	After grace, ₹/min $\times$ minutes-late
lates_to_absent	0	Legacy "N lates = 1 absent" (0 disables)
basic_pct	56.5	Salary split — Basic
conveyance_pct	22.6	Conveyance
hra_pct	5.9	HRA
adhoc_pct	15.0	Adhoc
misc_pct	0	Misc (the five should sum to 100)
working_days_per_month	26	Shown as divisor (UI)
sundays_paid	1	Sundays paid for monthly staff
cl_per_month	1	Paid casual leave allowance
sl_per_month	1	Paid sick leave allowance
pl_per_month	1.5	Paid privilege/earned leave
short_leave_per_month	2	Allowed short-leave count
ot_threshold_hours	9	Hours/day before OT
ot_rate_multiplier	1	OT pay multiplier

Day rules (the core calculator — confirmed with mam)

`calculateForEmployee(db, settings, employee, month)` walks each day of the month (up to **today** for the current month — future days aren't counted as absent; uses IST to avoid the VPS-UTC off-by-5.5 h bug; future months return zero) and assigns each day a `pay` of 1 / 0.5 / 0:

- Sunday** (no leave, no punch) — paid 1 if `sundays_paid`, labelled `sunday_paid`; else `sunday_unpaid`.
- Approved leave that day:**
 - casual $\rightarrow$ paid while `cl_used` < `cl_per_month` (1/mo)
 - sick $\rightarrow$ paid while `sl_used` < `sl_per_month`
 - earned $\rightarrow$ paid while `pl_used` < `pl_per_month` (1.5/mo)
 - comp_off / half_day leave $\rightarrow$ always a full paid day
 - Beyond allowance $\rightarrow$ unpaid leave (pay 0).
- Admin-marked day** (admin_mark, no punch) — honoured directly over punch logic: present/late $\rightarrow$ 1; half_day/short_day $\rightarrow$ 0.5; leave/holiday $\rightarrow$ 1 (paid leave); absent/other $\rightarrow$ 0.
- Punched day:**
 - Half day (pay 0.5)** if punch-in is **after 10:00** (`half_day_after_time`, no grace) **or** total hours < 4 (`min_hours_half_day`) — *unless* a short leave was applied that day and `skip_half_day_if_short_leave` is on.
 - Otherwise **full day (pay 1)**. If punch-in is in the **09:46–10:00** window it's a **late mark** on an otherwise full day (recorded, not docked here). A 4–8 h day is a **full day** (mam dropped the old 8 h half-day docking).
 - If `ot_eligible` and hours > `ot_threshold_hours` (9), the excess accrues OT hours.
- No punch, no leave, not Sunday** $\rightarrow$ absent (pay 0).

Late penalty (current model): the first `late_grace_count` (3) late marks per month are free; every late day after that is charged `late_per_minute_rate` (₹20) $\times$ `minutes_late` (punch-in minus the 09:46 late-zone start). The legacy "N lates $\rightarrow$ 1 absent" model is off

by default.

Sandwich rule (revised): a weekly-off Sunday is paid **unless BOTH the Saturday before AND the Monday after are absent** — only then is the sandwiched Sunday deducted (`sunday_sandwich_break`). If either neighbour is worked/paid (full, half, or paid leave), the Sunday stays paid.

Sunday-worked bonus: if the person actually works a Sunday, they get an **extra** full (or half) day's pay on top of the already-paid weekly off, so net pay can exceed base salary.

Salary formula

- `per_day_rate = base_salary / total_days_in_month` (actual calendar days, 28–31 — Sundays are already paid via the sandwich rule, so the month is covered evenly).
- `gross_earned = per_day_rate × paid_days`.
- **Component split** of gross: basic = `gross×basic_pct/100`, conveyance, hra, adhoc, misc (matching SEPL's Tally slip).
- **OT pay** = `ot_hours × (base_salary / (total_days × ot_threshold)) × ot_rate_multiplier` ($\approx$ salary ÷ days ÷ 9 per hour at default settings).
- **Deductions** = late penalty (override-aware) + advance taken this month.
- `net_before_ot = gross - deductions + food`; `net_pay = gross + ot_pay - deductions + food`.

Manual monthly overrides & adjustments

Stored on `payroll_advances` (one row per month × employee): `amount` (advance, deducted), `food` (allowance, added), and three hand-set overrides — `paid_days_override`, `cl_override`, `late_penalty_override` (NULL = use auto). These let admin pay salary now while attendance is being corrected. Set via `PUT /payroll/advance/:employee_id`, `/food/:employee_id`, `/override/:employee_id`.

Salary-exempt employees (full salary)

When `employees.salary_exempt = 1` (mam's directive — e.g. Parul Goyal, Rajat Sir, Nitin Jain, Ankur Kaplesh, Pooja Kaplesh, D.S Kaplesh, Soma Kaplesh), the calculator **short-circuits**: it bypasses every attendance / late / leave deduction and returns the **full base salary** as gross (still split into Basic/Conveyance/HRA/Adhoc/Misc so the slip stays compliant). Advances are still deducted; future months still return 0.

Finalise & disburse

- `GET /payroll/calculate` (all) / `/calculate/:employee_id` (one) — live calc for a month.
- `POST /payroll/finalise` — snapshots the month into `payroll_runs` (status `draft` → `finalised`), locking historical payslips.
- `PUT /payroll/paid/:employee_id` — mark disbursed.
- `POST /payroll/unlock` (admin) — reopen a finalised month.
- Leave balances: `GET /payroll/leave-balances`, `PUT /payroll/leave-balance/:employee_id`, `POST /payroll/leave-balances/rollover`.

Print output

`SalarySlipPrint.jsx` — print-ready monthly salary slip showing the SEPL component breakdown (Basic / Conveyance / HRA / Adhoc / Misc), paid days, deductions, advance, food, OT, and net pay.

API endpoints (/api/payroll)

Method	Path	Purpose
GET	/settings	Read settings
PUT	/settings	Update settings (admin)
GET	/calculate	All-employee month calc
GET	/calculate/:employee_id	One-employee calc
POST	/finalise	Finalise month
PUT	/paid/:employee_id	Mark paid
POST	/unlock	Unlock month (admin)
PUT	/advance/:employee_id	Set advance (admin)
PUT	/food/:employee_id	Set food allowance (admin)
PUT	/override/:employee_id	Manual paid-days/CL/late overrides
GET	/leave-balances	Leave balances
PUT	/leave-balance/:employee_id	Set balance (admin)
POST	/leave-balances/rollover	Roll over (admin)

DB tables

payroll_settings (single row id=1), payroll_runs (finalised monthly snapshot, UNIQUE month+employee, status draft | finalised | disbursed, breakdown_json per-day), payroll_advances (advance/food/overrides per month × employee). Reads attendance, leave_requests, employees.

7.8 Employees (master)

Business purpose

The employee master: who works here, their designation/department, reporting line, join date, salary, and login linkage. It is the spine the rest of HRMS hangs off — payroll, scorecards, training assignments, DPR staff-cost auto-calc, and interview panels all reference employees.

Screen & columns

Employees.jsx — searchable table (by name/phone/email/designation/ department). Columns: **Name, Phone, Email, Designation, Department, Join Date, Linked User, [Salary], Status, Actions**. The Salary column is gated (only visible to roles that may see salary). A mobile card layout mirrors the table.

The **Linked User** column shows whether the employee row is connected to a login user — required for DPR Staff Cost auto-calc and for payroll attendance lookups. Where unlinked, payroll falls back to matching by name (case-insensitive) and persists the link once found.

Key fields (employees + migrations)

user_id (login link), name, phone, email, designation, department, join_date, salary, status (active | training | inactive | terminated), aadhar_file / pan_file / qualification_file, **salary_exempt** (full-salary flag), cl_eligible, **ot_eligible**, cl_opening_balance.

Bulk add and auto-link helpers exist (POST /hr/employees/bulk , /employees/auto-link).

Reporting-to / hierarchy

Reporting line is expressed through hiring (reporting_manager_id on hiring_requests , reporting_to on offers) and the org structure; the employee master itself stores designation + department as the primary grouping.

API endpoints (under /api/hr)

Method	Path	Permission
GET	/employees	open (dropdowns/dashboards)
POST	/employees	employees:create
POST	/employees/auto-link	employees:edit
POST	/employees/bulk	employees:create
PUT	/employees/:id	employees:edit
DELETE	/employees/:id	employees:delete

DB table

employees .

7.9 Performance — MIS Scorecards

Business purpose

A weekly MIS scorecard per role: each role has a template of weighted KPIs (summing to 100%), with weekly planned-vs-actual entries. Some KPIs pull their actuals automatically from other ERP modules (DPR, PMS, checklists, tickets, delegations, indents, stock, billing); the rest are entered manually. Produces a weekly score per user.

Who uses it

- **Admins / managers** — manage templates + KPIs, assign templates to users, set per-user KPI targets, review the weekly board.
- **Employees** — fill their weekly actuals/commitments.

Templates (20 seeded)

server/db/seedScoring.js seeds **20 role templates** into score_templates / score_kpis . The full set:

Site Engineer · Supervisor · Aanchal — Finance Executive · Monika — AI Implementation Head · Anmol — DPR / Score Card · Ankush — HR Ops + Marketing · Ajmer — Procurement Lead · Gaganpreet — Cash Flow Manager · Indresh — Billing Engineer · Lovely — Sales Coordinator · Nancy — Estimation & Costing Head · Nitin Sir — MD · Parul — Compliance & Tender · Pradeep Panda — Operations Lead · Raj Kumar — Procurement Manager · Rajeev Sood — Quotation · Riti — Sales Coordinator (Sales Side) · Ruksana — HR Hiring · Shubham — Accounts · Sushila — Sales Coordinator.

Each KPI (score_kpis) has: group_name (Basic | Weekly | Monthly), metric_name, weightage (0–100), direction (higher_better | lower_better), data_source (manual or auto:... such as auto:dpr_profit , auto:indents_in_week , auto:mb_signed , auto:pms , auto:checklists , auto:tickets , auto:delegations , auto:stock_at_site , ...), display_order, default_planned (fixed weekly target).

Assignment & per-user targets

- Each user is bound to one template via score_user_template .
- score_user_kpi_target overrides a KPI's default_planned **per user** (e.g. Ajmer's "Indent vs Bill" target = 5 while Aakash's = 3, same KPI). The scorecard reads the override first, then falls back to the KPI default.

Weekly entries & scoring

score_entries holds one row per (user, kpi, week_start = Monday): planned, actual, actual_pct, last_week_pct, total_uptodate, pending_uptodate, pending_work, pending_pct, commitment, notes.

GET /scoring/weekly walks active users and, for each, computes given-vs-done counts for the auto KPIs over the week window: delegations, PMS tasks, checklists (assigned × 6 weekdays vs completion rows), help tickets, plus DPR/indent/stock/billing-derived metrics. The weighted roll-up of KPI percentages × weightage produces the user's weekly score. GET /scoring/weekly/detail returns the per-KPI breakdown.

API endpoints (/api/scoring)

Method	Path	Purpose
GET	/templates /templates/:id	List / detail templates
POST/PUT/DELETE	/templates[:id]	Manage templates (admin)
POST	/templates/:id/kpis	Add KPI (admin)
PUT/DELETE	/kpis/:id	Edit / delete KPI (admin)
GET	/users/:user_id/kpi-targets	Per-user targets
PUT	/users/:user_id/kpi-targets/:kpi_id	Set target (admin)
GET	/assignments	Template assignments
PUT	/assignments/:user_id	Assign template (admin)
GET	/scorecard	A user's scorecard
PUT	/scorecard/entry	Save a weekly entry
GET	/weekly	Weekly board (all users)
GET	/weekly/detail	Per-KPI weekly detail

DB tables

score_templates , score_kpis , score_user_template , score_user_kpi_target , score_entries .

7.10 Sub-contractor Master

Business purpose

The persistent directory of sub-contractors (mam's "Sub-Contractor Form" Google-Form workflow brought into the ERP — 47+ entries), searchable and filterable alongside the rest of the data, and used as the vendor pool for the sub-contractor hiring tracker (7.3) and the DPR contractor picker.

Screen & columns

SubContractors.jsx — table columns: **Name, Type, Contact, Location, Exp., Manpower, Tools, GST, Rate vs Budget, Start (days), Status, Actions.** Filters by search text, state, and contractor_type; defaults to active-only. A "+ Add Sub-Contractor" modal captures the full form.

Key fields (sub_contractors + migrations)

name, phone, email, **contractor_type**, state / district / location_extra, **experience_years**, **manpower** (strength), with_tools, has_gst / gst_number, rate_in_budget, start_within_days, specialization, rate / rate_unit, status (**qualified** | **negotiation** | **onboarded** | **active** | **inactive**), active flag, work_order_file, notes, created_by/updated_at.

Lookup endpoint

GET /api/sub-contractors/lookup is open to any authenticated user (site engineers don't get full master access) and returns a deduped id/name/type/district list for dropdowns — it GROUPs BY name (NOCASE, keeping the lowest id) so duplicate-named gangs collapse to one picker entry.

API endpoints (/api/sub-contractors)

Method	Path	Permission
GET	/lookup	any authenticated
GET	/	sub_contractors:view (filters: q, state, contractor_type, active)
GET	/:id	sub_contractors:view
POST	/	sub_contractors:create
PUT	/:id	sub_contractors:edit
PATCH	/:id/active	sub_contractors:edit (toggle active)
DELETE	/:id	sub_contractors:delete

(A simpler /hr/sub-contractors set also exists in hr.js against the same table.)

DB table

sub_contractors .

7.11 Permissions & cross-module notes

- HRMS modules are guarded by requirePermission(module, action) with modules: hr , hr_system , employees , expenses , attendance , payroll , scoring , subcon_hiring , sub_contractors . Settings/unlock endpoints add an extra admin gate.
- **Shared READ endpoints stay open** so dropdowns/dashboards keep working (e.g. GET /hr/employees , GET /sub-contractors/lookup).
- Payroll attendance lookups depend on the **employee→user** link; payroll name-matches and persists the link when employees.user_id is NULL.
- The only HRMS cron is the **HR automations cron (every 30 min)** in hrAutomationsCron.js (7.1.8). Attendance has no cron — geofence + the 30 s in-browser location ping handle live tracking.

Module map (quick reference)

Module	Frontend	Backend	Mount	Main tables
Hiring (ATS)	HR.jsx	hr.js	/api/hr	candidates, candidate_events, candidate_docs, hiring_requests, jd_templates, job_descriptions, interview_scorecards, screening_*, final_round_questions, notifications
Hiring (legacy)	HRSystem.jsx	hrSystem.js	/api/hr-system	(own funnel tables)
Public offer	PublicOffer.jsx	publicHr.js	/api/public/offer	candidates
Sub-con Hiring	SubconHiring.jsx	subconHiring.js	/api/subcon-hiring	subcon_hiring(+_steps/_files/_candidates)
Onboarding	Induction.jsx + HR tab	hr.js	/api/hr	induction_items
Training	Training.jsx + HR tab	hr.js	/api/hr	training_videos, training_assignments
Attendance	Attendance.jsx	attendance.js	/api/attendance	attendance, geofence_settings, location_tracking, leave_requests
Payroll	Payroll.jsx, SalarySlipPrint.jsx	payroll.js	/api/payroll	payroll_settings, payroll_runs, payroll_advances
Employees	Employees.jsx	hr.js	/api/hr	employees
Performance	Scorecard.jsx	scoring.js	/api/scoring	score_templates, score_kpis, score_user_template, score_user_kpi_target, score_entries
Sub-con Master	SubContractors.jsx	subcontractors.js	/api/sub-contractors	sub_contractors

8. Inventory & Assets

This section documents the five physical-stock and equipment modules of the SEPL ERP:

Module	Page	Route prefix	What it tracks
Tool Rentals	RentalTools.jsx	/api/rental-tools	Renting in tools/machinery from outside vendors (scissor lifts, drills, etc.) through a 4-stage enquiry → rate → received → returned workflow with an auto-generated PO
Company Assets	CompanyAssets.jsx	/api/company-assets	IT / office equipment register (laptops, phones, SIMs, monitors), issued to employees
Inventory	Inventory.jsx	/api/inventory	Warehouse / site-store consumable stock with IN/OUT/transfer movements, valuation, and barcode scanning
Tools	Tools.jsx	/api/tools	Returnable company-owned tool catalog (serial, calibration, condition) issued to sites or people
Room Rentals	Rentals.jsx	/api/rentals	Staff/labour accommodation: rented properties/rooms, bookings, and the "Raise Rent" payment-request workflow

All five are permission-gated through the standard `requirePermission(module, action)` middleware (`view` / `create` / `edit` / `delete` , and `approve` where applicable). Source files:

- `client/src/pages/RentalTools.jsx` , `client/src/pages/RentalPOPrint.jsx` , `server/routes/rentalTools.js`
- `client/src/pages/CompanyAssets.jsx` , `server/routes/companyAssets.js`
- `client/src/pages/Inventory.jsx` , `client/src/components/BarcodeScanner.jsx` , `server/routes/inventory.js`
- `client/src/pages/Tools.jsx` , `server/routes/tools.js`
- `client/src/pages/Rentals.jsx` , `server/routes/rentals.js`

8.1 Tool Rentals

Business purpose

Tool Rentals manages **renting equipment IN from external vendors** — scissor lifts, drilling rigs, welding sets and similar machinery the company needs on site for a fixed number of days but does not own. It is a SLA-tracked, four-stage workflow (mam's spec, 2026-05-16) that ends with the rented item being returned to the vendor. A vendor **Purchase Order is auto-created** the moment the rate is finalised, so the rental itself becomes the procurement artifact.

Who uses it

- **Site engineers** raise the enquiry (Stage 1) and confirm material received on site (Stage 3) with a live photo + GPS.
- **Ajmer** (the designated rental approver) finalises the vendor rate (Stage 2, which auto-creates the PO) and signs off the return (Stage 4).
- The approver is identified by `app_settings.rental_approver_user_id` , set once by an admin from the page's top-right gear. If unset, any user whose role has `can_approve = 1` on the `rental_tools` module may perform the approver-gated steps (`canApprove()` in `rentalTools.js`).

The four stages

The internal DB enum is `enquiry` → `rate_finalised` → `material_received` → `returned` (plus `cancelled`). The display labels are admin-renamable (stored as a single JSON blob in `app_settings.rental_tools_stage_labels` , defaults below):

Stage enum	Default label	Owner	SLA target
<code>enquiry</code>	Stage 1 — Enquiry Raised	Site engineer	<code>created_at + 5 business hours</code> (<code>stage1_target_at</code>)
<code>rate_finalised</code>	Stage 2 — Rate Finalised	Ajmer (approver)	<code>date_of_requirement + 1 business day</code> (<code>stage2_target_at</code>)
<code>material_received</code>	Stage 3 — Material at Site	Site engineer	<code>material_received + days_required</code> business days (<code>return_target_date</code>)
<code>returned</code>	Stage 4 — Returned · Closed	Ajmer (approver)	— (closes the enquiry)
<code>cancelled</code>	Cancelled	any editor	—

SLA breaches are flagged per stage on transition (`stage1_breached` , `stage2_breached` , `stage3_breached`), and the dashboard counts open overdues per stage. Business-hour/day maths use `server/lib/businessHours.js` (`addBusinessHours` , `addBusinessDays` , `Sunday-skipped`).

Main screen / tabs

`RentalTools.jsx` uses `useUrlTab('dashboard')` . The dashboard shows per-stage chip counts, breach counts, open total, total in-flight value (`SUM(vendor_rate * days_required)`), and this-month enquiry count. The enquiry list is filterable by `stage` , `status` , and free-text `q` (site / enquiry no / tool / vendor). List columns: **Enquiry · Site · Tool · Days · Site Eng · Stage · Status · Vendor · Rate · PO**.

Key fields (table `rental_tool_enquiry`)

- `enquiry_no` — auto-numbered `RT-YYYY-####`
- `site_id` / `site_name` , `tool_description` , `date_of_requirement` , `days_required`
- `site_engineer_id` / `site_engineer_name`
- `vendor_id` , `vendor_name` , `vendor_rate` , `vendor_rate_unit` (default `per_day`)
- `po_id` , `po_number` (links the auto-created `purchase_orders` row)
- `material_received_photo` , `material_received_lat` , `material_received_lng` , `return_target_date`
- `current_stage` , `status` (`open` / `closed` / `cancelled`), breach flags, audit timestamps, `created_by` / `rate_finalised_by` / `return_signed_by`

Step-by-step workflow

1. **Raise enquiry** — `POST /enquiries` . Requires `site_name` , `date_of_requirement` , `days_required` (`> 0`) . Sets `current_stage='enquiry'` , computes `stage1_target_at = now + 5 business hours` , writes a `rental_tool_history` row.
2. **Finalise rate** (Ajmer) — `POST /enquiries/:id/finalise-rate` . Requires `vendor_id` , `vendor_name` , `vendor_rate` , `po_number` , `po_date` , `total_amount` . In one transaction it **inserts a `purchase_orders` row** (`business_book_id` `NULL` — rentals are operational, not tied to a sale) and updates the enquiry to `rate_finalised` , locking vendor + rate + PO and computing `stage2_target_at` .
3. **Material received** (site engineer) — `POST /enquiries/:id/material-received` (multipart). Requires a live `photo` upload **and** `latitude` + `longitude` . Stores the photo under `/uploads/rental-tools/` , sets `return_target_date = today + days_required` business days, moves to `material_received` .
4. **Return** (Ajmer) — `POST /enquiries/:id/return` . Sets `returned_at` , `return_signed_by` , optional `return_notes` ; moves to `returned` , `status='closed'` , stamps `stage3_breached` .
5. **Cancel** — `POST /enquiries/:id/cancel` at any open stage sets `status='cancelled'` .

PO numbering & print

The PO number is **entered by the approver** at Finalise-Rate (not auto-sequenced here), along with `po_date` , `advance_amount` , and `total_amount` . The PO row is created directly in the shared `purchase_orders` table. `RentalPOPrint.jsx` (route `/rental-po/:enquiryId/print`) renders a print-styled vendor PO from a single `GET /enquiries/:id` fetch (which joins vendor-master fields + the linked `purchase_orders` row). It uses the same SEPL company header as `VendorPOPrint.jsx` with a slim rental-specific terms list, and relies on the browser's Print → Save-as-PDF (no server PDF library).

API endpoints — /api/rental-tools

Method	Path	Purpose	Permission
GET	/dashboard	Stage counts, breaches, value, approver, labels	view
GET	/settings/stage-labels	Defaults + current labels	view
PUT	/settings/stage-labels	Rename stage labels (admin only)	edit
PUT	/settings/approver	Set/clear rental_approver_user_id (admin only)	edit
GET	/enquiries	List (filters: stage, status, q)	view
GET	/enquiries/:id	Single enquiry + history + linked PO	view
POST	/enquiries	Raise enquiry (Stage 1)	create
POST	/enquiries/:id/finalise-rate	Lock rate + auto-create PO (Stage 2)	create + approver
POST	/enquiries/:id/material-received	Live photo + GPS (Stage 3)	edit
POST	/enquiries/:id/return	Sign return / close (Stage 4)	edit + approver
POST	/enquiries/:id/cancel	Cancel an open enquiry	edit

DB tables

- rental_tool_enquiry — the workflow record
- rental_tool_history — per-transition log (from_stage , to_stage , triggered_by , notes , entered_at)
- purchase_orders — the auto-created PO (shared table)
- app_settings — rental_approver_user_id , rental_tools_stage_labels

8.2 Company Assets

Business purpose

A register of **IT / office equipment** — laptops, desktops, mobiles, tablets, SIM cards, monitors, routers, printers, etc. — with full issue/return/maintenance/scrap movement history per asset for audit. SIMs and subscriptions carry a monthly_cost so the dashboard can show recurring spend.

Who uses it

IT / admin staff with the company_assets permission. Assets are issued to **employees** (users), not sites.

Main screen / columns

Single-table page (CompanyAssets.jsx). Category options: **Laptop, Desktop, Mobile, Tablet, SIM Card, Monitor, Router, Printer, Charger, Other** (and more). Stat cards include total value and **Monthly Recurring** (SIM/subscription). Table columns: **Asset No · Category · Name/Model · Serial/IMEI · SIM/Mobile · Issued To · Condition · Status · Value · Actions**. Filters: category, status, search (matches name/brand/model/serial/imei/ip/mobile/asset_no). The add form shows category-specific fields — IP address for Laptop/Desktop/Monitor/Router/Printer; SIM number/ICCID, mobile number, carrier, and monthly cost for SIM Card / Mobile.

Key fields (table company_assets)

asset_no (auto AST-YYYY-####), category , name , brand , model , serial_no , imei , ip_address , mobile_number , carrier , monthly_cost , purchase_date , purchase_price , vendor , warranty_till , condition (new / good / fair / poor / damaged / scrap), status (available / issued / maintenance / lost / scrapped), current_user_id / current_user_name , issued_at , returned_at , photo_url , notes .

There is **no automatic depreciation** — purchase_price is stored as-is and the dashboard sums it as total_value (excluding lost/scrapped). Depreciation is not computed.

Workflow & movements

1. **Create** (POST /) — auto-assigns asset_no . If created already issued with a current_user_id , an initial issue movement is logged.

2. **Issue** (POST `/:id/issue` , body `user_id` , `notes`) — must be `available` ; sets status `issued` , stamps `issued_at` , logs an `issue` movement (from→to user).
3. **Return** (POST `/:id/return` , body `notes` , `condition`) — must be `issued` ; sets `available` , clears user, stamps `returned_at` , optionally updates condition, logs a `return` movement.
4. **Maintenance** (POST `/:id/maintenance`) — sets `maintenance` , clears user, logs a `maintenance` movement.
5. **Scrap / Lost** (POST `/:id/scrap` , body `lost` boolean) — sets `scrapped` or `lost` , logs a `scrap` movement.

Movement types: `issue` , `return` , `maintenance` , `scrap` . Each row records `from_user_id` , `to_user_id` , `notes` , `performed_by` , `performed_at` in `company_asset_movements` .

API endpoints — `/api/company-assets`

Method	Path	Purpose	Permission
GET	<code>/</code>	List (filters: category, status, search, current_user_id)	view
GET	<code>/stats</code>	Counts by status/category, total value, monthly recurring	view
GET	<code>/:id</code>	Asset + movement history	view
POST	<code>/</code>	Create asset	create
PUT	<code>/:id</code>	Edit asset fields	edit
POST	<code>/:id/issue</code>	Issue to employee	edit
POST	<code>/:id/return</code>	Return to inventory	edit
POST	<code>/:id/maintenance</code>	Send for maintenance	edit
POST	<code>/:id/scrap</code>	Mark scrapped / lost	edit
DELETE	<code>/:id</code>	Delete asset	delete

DB tables

- `company_assets`
- `company_asset_movements`

8.3 Inventory

Business purpose

The core **consumable-stock** system. Tracks quantities and valuation of material across **warehouses**, supporting opening balances, GRN/purchase receipts (IN), site issues and inter-warehouse transfers (OUT), and a full movement journal. Stock valuation uses a **moving weighted average** on IN movements.

Who uses it

Store keepers / procurement / project staff with the `inventory` permission. The most important consumer of inventory data, however, is the **Procurement indent-approval flow** (see 8.3.7), which reads free office stock to allocate "from-store" against indented quantities.

Warehouse vs site stock

There are two warehouse `type` s:

- `office` — head-office store(s). This is the free stock the procurement approver can draw from.
- `site_store` — a per-site store, linked to a `sites` row via `site_id` (required on create).

The dashboard and per-line stock visibility split totals by `warehouse.type` into **Office stock** vs **Site stock**.

Main screen / tabs

`Inventory.jsx` (`useUrlTab('stock')`) exposes these tabs:

Tab	What it does
Stock	Current balances per (warehouse × item) with effective rate, value, condition; inline edit/delete
Opening Stock (item-wise)	Enter opening balances item-by-item (create-only)
Receive (IN)	Record a GRN/purchase/opening/adjust IN movement into one warehouse, multiple lines, with optional per-line photo and barcode scan
Issue / Transfer (OUT)	Issue stock to a site (consumption) or transfer to another warehouse, with barcode scan
Movements	Filterable journal of all IN/OUT movements
Reports & Valuation	Per-warehouse roll-up
Warehouses	Create / rename / activate-deactivate warehouses

The IN/OUT/opening tabs are gated on `create` permission; warehouse edit/create gated accordingly.

Stock balance, rate & valuation

Each `stock_balance` row is a unique (warehouse_id × item_master_id) with `quantity`, `avg_rate`, `reorder_level`, and a `condition` column (`Used` / `Unused` / `Scrap` ; added 2026-05-29 so balances can be flipped inline without forcing a movement). Legacy rows with no condition were one-time backfilled to `Used` (guarded by `app_settings.stock_condition_used_backfill_v1`).

Effective rate / value (in `GET /stock`): use `avg_rate` when > 0, otherwise fall back to `item_master.current_price`; `rate_source` is tagged `movements` / `master` / `none` so the UI can show a "from master" hint. `value = effective_rate × quantity`.

Stock movements (IN / OUT / transfer / issue / adjust)

All quantity changes go through `applyMovement()` inside a SQLite transaction so the balance and the journal stay consistent:

- **IN** — new qty = prev + qty; `avg_rate` recomputed as weighted average $((\text{prevQty} \cdot \text{prevRate}) + (\text{qty} \cdot \text{inRate})) / \text{newQty}$.
- **OUT** — decrement; **throws if it would go negative** (no negative stock allowed); `avg_rate` unchanged.

Reference types seen in `stock_movements.reference_type`: `OPENING`, `PURCHASE`, `GRN`, `ISSUE`, `TRANSFER`, `ADJUST`, `RGP`, `ISSUE_REVERSAL`.

Receive (POST /receive) — body { `warehouse_id`, `items:[{item_master_id, quantity, rate, photo_url?, item_condition?}]`, `reference_type`, `reference_id`, `notes` }. Each line becomes one IN movement.

Issue / Transfer (POST /issue) — body { `from_warehouse_id`, `items:[{item_master_id, quantity}]`, `destination_type` ('site'|'warehouse'), `destination_id`, `reference_type`, `reference_id`, `notes` }.

- **Issue to site** (`destination_type='site'`) — one OUT movement per line, `site_id` set, `reference_type='ISSUE'`.
- **Transfer** (`destination_type='warehouse'`) — writes **two** halves sharing one `reference_id` (`XFR-<timestamp>` if not supplied): an OUT from the source and an IN into the destination, both carrying the source's current avg rate. Source and destination must differ.

Manual adjust (PATCH /stock/:id) — edit qty, avg_rate, and/or condition for one balance row. A condition-only edit is handled inline. A qty change records an `ADJUST` movement (IN if up, OUT if down). **Delete (DELETE /stock/:id)** writes a final OUT `ADJUST` for the remaining qty, then removes the balance row.

Barcode scanning

`client/src/components/BarcodeScanner.jsx` is a camera modal wrapping `html5-qrcode` (lazy-imported to keep the page bundle small). It prefers the rear camera (`/back|rear|environment/`), scans at 10 fps, and on the first successful read calls `onScan(decoded)` and closes. The Receive and Issue tabs each render a per-line "Scan" button (`setScanFor(index)`); `onScanResult` matches the decoded text against `item_master.item_code` (case-insensitive) or the numeric id and auto-fills that line, toasting success or "No item with code ...". Falls back gracefully with an instructions panel when camera permission is denied.

Reorder / low stock

`reorder_level` per balance row drives the low-stock view (`GET /low-stock`, and `low_only=1` on `/stock`): rows where `reorder_level > 0 AND quantity ≤ reorder_level`, ordered by how far below the level they are. `PUT /reorder/:warehouse_id/:item_master_id` upserts the level even before any stock exists.

How Inventory free-stock feeds Procurement indent approval allocation

This is the key cross-module integration (in `server/routes/procurement.js`):

1. **Stock visibility at approval** — when an indent is opened for approval, for every line linked to an `item_master_id` the system sums `stock_balance.quantity` grouped by `warehouse.type` (active warehouses only) and attaches `office_stock` and `site_stock` to

the line. Office = head-office store; Site = total across **all** site stores (so the approver can see "send from another site"). Lines with no `item_master_id` get 0/0.

2. **Allocate "from store"** — at approval the approver can take part/all of a line's quantity from existing office stock. The line is split into a **parent** row (`source='procure'` , `qty` = approved – from_store, continues to vendor PO) and a **child** row (`source='store'` , `qty` = from_store), tied by `parent_item_id` .
3. **Atomic stock-out** — `from_store` is validated against `SUM(stock_balance.quantity)` across **office** warehouses. On approval the system greedy-decrements `stock_balance` across office warehouses (smallest warehouse id first / FIFO), writes one `stock_movements` OUT (`reference_type='ISSUE'`) per warehouse touched, and creates one `stock_issue_notes` header (`SI/YYYY/####`) the storekeeper can print as a challan.
4. **RGP** — approved RGP-from-store lines similarly greedy-decrement office stock (`reference_type='RGP'`), best-effort and non-blocking (never go negative / never block approval).
5. **Reversal** — un-approving / reversing an issue note reads the `ISSUE` OUT movements by `reference_id` and adds the qty back, logging `ISSUE_REVERSAL` IN movements.

So free **office** stock directly reduces the quantity that flows to a vendor PO, and the deduction is journaled as `ISSUE` movements with a printable SI note.

API endpoints — `/api/inventory`

Method	Path	Purpose	Permission
GET	<code>/warehouses</code>	List warehouses + item count + total value	view
POST	<code>/warehouses</code>	Create warehouse (office / site_store)	create
PUT	<code>/warehouses/:id</code>	Rename / activate / deactivate	edit
GET	<code>/stock</code>	Balances (filters: warehouse_id, search, low_only)	view
GET	<code>/summary</code>	Per-warehouse roll-up	view
GET	<code>/low-stock</code>	Items at/below reorder level	view
POST	<code>/receive</code>	IN movement (one warehouse, many items)	create
POST	<code>/issue</code>	OUT — issue to site or transfer to warehouse	create
GET	<code>/movements</code>	Journal (filters: warehouse, item, type, ref, dates, limit)	view
PUT	<code>/reorder/:warehouse_id/:item_master_id</code>	Set reorder level	edit
PATCH	<code>/stock/:id</code>	Adjust qty / avg_rate / condition	edit
DELETE	<code>/stock/:id</code>	Zero out + delete balance row	delete

DB tables

- `warehouses` (`type` = office / site_store , `site_id` , `active` , `in_charge` , `location`)
- `stock_balance` (per warehouse × item: `quantity` , `avg_rate` , `reorder_level` , `condition`)
- `stock_movements` (IN/OUT journal with `reference_type` , `reference_id` , `from_warehouse_id` , `to_warehouse_id` , `site_id` , `item_condition` , `photo_url`)
- `item_master` (catalog: `item_code` , `item_name` , `uom` , `current_price` , ...)
- `stock_issue_notes` (SI/YYYY/#### challan headers created by procurement allocation)
- `sites` (for site_store warehouses and issue destinations)

8.4 Tools

Business purpose

A catalog of **returnable, company-owned tools** — drills, ladders, multimeters, safety gear — tracked separately from consumable inventory. Each tool is a unique item with serial, condition, calibration dates, and a current location (a site **or** a person). Distinct from Company Assets (8.2), which is IT/office equipment issued only to employees; Tools can be issued to **sites**.

Who uses it

Site supervisors / stores staff with the `tools` permission. A weekly per-site **tool-count submission** feeds the Supervisor MIS KPI.

Main screen / columns

Tools.jsx single-table page. Categories: **Drilling, Cutting, Measurement, Safety, Power, Hand, Lifting, Electrical, Other**. Statuses: **available, in_use, maintenance, lost, scrapped**. Stat cards include **Calibration Due (30 days)** and total value. Columns: **Code · Name · Category · Brand/Model · Serial · Cond. · Status · Current Site/User · Actions**. Filters: category, status, site_id, user_id, search (name/tool_code/serial/brand).

Key fields (table tools)

tool_code (auto T-YYYY-####), name , category , brand , model , serial_no , purchase_date , purchase_price , condition (default good), status (default available), current_site_id , current_user_id , last_calibration_date , next_calibration_date , photo_url , notes .

Movements

Issue/return/transfer/scrap/maintenance all log a tool_movements row (action , from/to site + user, expected_return_date , actual_return_date , condition_at_action , notes , photo_url).

- 1. **Issue** (POST /:id/issue) — body must give to_site_id or to_user_id (plus optional expected return date, condition, notes, photo). Sets status in_use and the new location. A re-issue to a different site/user is effectively a **transfer** (the from_* is the previous location).
- 2. **Return** (POST /:id/return) — clears location, sets available , stamps actual_return_date , optionally updates condition.
- 3. **Scrap** (POST /:id/scrap) — status scrapped , condition scrap .
- 4. **Maintenance** (POST /:id/maintenance) — status maintenance .

Calibration

next_calibration_date drives the "Calibration Due (30 days)" stat (next_calibration_date ≤ date('now','+30 days')).

Weekly submissions (Supervisor KPI)

POST /submissions upserts (on site_id + submitted_by + week_start) a tools_list_submissions row holding tools_count (summed from a tools_json array), an optional photo, and notes — powering a supervisor MIS metric. GET /submissions/list lists them filtered by week/site/submitter.

API endpoints — /api/tools

Method	Path	Purpose	Permission
GET	/	List (filters: category, status, site_id, user_id, search)	view
GET	/stats	Counts by status/category, calibration-due, value	view
GET	/:id	Tool + movement history	view
POST	/	Create tool	create
PUT	/:id	Edit	edit
DELETE	/:id	Delete tool + its movements	delete
POST	/:id/issue	Issue / transfer to site or user	edit
POST	/:id/return	Return	edit
POST	/:id/scrap	Scrap	edit
POST	/:id/maintenance	Mark for maintenance	edit
GET	/submissions/list	Weekly submissions list	view
POST	/submissions	Upsert weekly tool count	create

DB tables

- tools (catalog)
- tool_movements (issue/return/transfer/scrap/maintenance log)
- tools_list_submissions (weekly per-site count, unique on site_id+submitted_by+week_start)
- sites , users (current location targets)

8.5 Room Rentals

Business purpose

Tracks **staff/labour accommodation** rented across project sites. Two layers exist in code:

1. A full **property** → **room** → **booking** → **payment** model (rented flats/houses, rooms within them, which employee stays where, and monthly rent paid to the landlord).
2. The **"Raise Rent" request workflow** (mam's primary flow) — a per-month rent request with approval and payment-release tracking. This is the workflow surfaced in the current UI.

Who uses it

Site/admin staff raise rent requests; users with `rentals.approve` (and admins) approve/reject; finance marks them paid. Anyone with `rentals` permission can view.

Main screen / tabs

`Rentals.jsx` (`useUrlTab('requests')`) currently exposes only **two** tabs in the UI:

- **Raise Rent** (`requests`) — the rent-request workflow with stat cards (Pending / Approved / Paid / Rejected / amounts).
- **Payments Log** (`payments`) — the property-level monthly rent payment ledger.

The property/rooms/bookings management blocks and their backend endpoints still exist (legacy), but the **properties** and **bookings** tab buttons are no longer rendered in the current UI — the "Raise Rent" request flow superseded them.

Raise Rent workflow (table `rent_requests`)

Status lifecycle: `pending` → `approved` → `paid` (or `rejected`); requests can also be flagged **inactive** (rental relationship ended / room vacated — no more rent expected, and "when we inactive not payment log").

Key fields: `request_no` (auto `RR-YYYY-####`), `site_id` / `site_name`, `arrange_for` (`SEPL` or `Contractor`), `contractor_name`, `employee_user_id` / `employee_name`, `owner_name` / `owner_phone` / `owner_aadhar_url`, `room_photo_url` + `photo_taken_at` + `photo_lat` / `photo_lng` (live proof), `payment_mode` (`Bank` / `UPI` / `Scanner` — only the relevant sub-fields `bank_account` + `ifsc_code` / `upi_id` / `scanner_url` are kept), `rent_month`, `rent_amount`, `pay_by_day` (default 10), `status`, `inactive`.

Steps:

1. **Raise** (`POST /rent-requests`) — requires `owner_name`, `rent_month`, `arrange_for`. Auto-numbers `RR-YYYY-####`, validates payment mode and clears non-relevant payment fields, then push-notifies approvers (admins + anyone with `rentals.approve`).
2. **Approve / Reject** (`POST /rent-requests/:id/approve` | `/reject`, `rentals.approve`) — reject requires a reason (min 5 chars). Both push-notify the creator.
3. **Mark paid** (`POST /rent-requests/:id/mark-paid`) — records `paid_via`, `transaction_ref`, `receipt_url`; sets `status='paid'`, stamps `paid_by` / `paid_at`; notifies the creator.
4. **Mark inactive / active** (`/mark-inactive` | `/mark-active`) — toggle the inactive flag.

`GET /rent-requests/stats` gives counts (pending/approved/paid/rejected/inactive) and amounts (total paid, pending amount).

Properties / rooms / bookings / payments (model)

- **Properties** (`rental_properties`) — name, address, city/state/pincode, landlord contact, `monthly_rent`, `deposit_paid`, agreement start/end + file, bedrooms, capacity, amenities, `status` (`active` ...), `site_id`. Dashboard stats: active properties, rooms (total/occupied/vacant), monthly burn, deposit locked, agreements expiring in 30 days, active bookings.
- **Rooms** (`rental_rooms`) — `room_name`, `capacity`, `status` (`available` / `occupied`), per property.
- **Bookings** (`rental_bookings`) — which `occupant_user_id` / `occupant_name` stays in which room, check-in/out dates, `rent_share`, `deposit_collected`, `status` (`active` / `completed`). Creating a booking marks the room **occupied** (transactional) and notifies the occupant; check-out marks it `completed` and frees the room if no other active bookings remain.
- **Payments** (`rental_payments`) — monthly rent to landlord, upserted on `property_id` + `period_month` (amount, paid date, mode, ref, receipt).

API endpoints — /api/rentals

Method	Path	Purpose	Permission
GET	/stats	Property/room/booking dashboard	view
GET	/properties /properties/:id	List / detail (rooms, bookings, payments)	view
POST/PUT/DELETE	/properties[:id]	Manage properties	create/edit/delete
POST	/properties/:property_id/rooms	Add room	create
PUT/DELETE	/rooms/:id	Manage room	edit/delete
GET	/bookings	List bookings	view
POST/PUT/DELETE	/bookings[:id]	Manage bookings	create/edit/delete
POST	/bookings/:id/check-out	Check out an occupant	edit
GET	/payments	Property monthly payment log	view
POST/DELETE	/payments[:id]	Record / delete payment	create/delete
GET	/rent-requests /rent-requests/stats	Rent-request list / stats	view
POST/PUT/DELETE	/rent-requests[:id]	Manage rent requests	create/edit/delete
POST	/rent-requests/:id/approve /reject	Approve / reject	approve
POST	/rent-requests/:id/mark-paid	Mark paid	edit
POST	/rent-requests/:id/mark-inactive /mark-active	Toggle inactive	edit

DB tables

- `rent_requests` — the "Raise Rent" workflow record (primary)
- `rental_properties`, `rental_rooms`, `rental_bookings`, `rental_payments` — the property model
- `sites`, `users` — site + occupant/employee references

8.6 Cross-module summary

Concern	Tool Rentals	Company Assets	Inventory	Tools	Room Rentals
Direction	Rent in from vendor	Own (IT/office)	Own (consumable)	Own (returnable)	Rent in (housing)
Assigned to	Site (via vendor)	Employee	Warehouse / Site	Site or Person	Employee / Contractor
Movement log	<code>rental_tool_history</code>	<code>company_asset_movements</code>	<code>stock_movements</code>	<code>tool_movements</code>	booking + payment rows
Auto-numbering	RT-YYYY-####	AST-YYYY-####	—	T-YYYY-####	RR-YYYY-####
Approval gate	Ajmer (Stage 2 & 4)	—	—	—	<code>rentals.approve</code> (rent requests)
Auto PO	Yes (Stage 2)	No	Feeds vendor PO via store allocation	No	No
Valuation	rate × days	purchase_price (no depreciation)	moving weighted avg	purchase_price	rent_amount / monthly_rent

Key integration: **Inventory office free-stock is consumed by the Procurement indent-approval flow** — the approver sees per-line office/site stock, can allocate "from store" (splitting the indent line into a store child + procure parent), and the system journals `ISSUE OUT` movements plus a printable `SI/YYYY/####` stock-issue note, reducing the quantity sent to the vendor PO.

9. Tasks & Service Desk

This section documents the five modules that make up the **Tasks & Service Desk** area of the SEPL ERP: **Delegations**, **PMS Tasks**, **Checklists**, **Complaints**, and **Help Tickets**. Together they cover internal task delegation, project-tied tasks, recurring routine checks, customer service complaints, and the in-app support desk that every staff member can reach.

All five share a common philosophy:

- A **task / item** is raised by one person, **assigned to** another, and moves through a **status pipeline**.
- Most are gated by the **role-permission matrix** (`role_permissions` joined via `user_roles`); `role = 'admin'` always bypasses.
- A **scope model** (`mine` / `given` / `all` , plus `followup` for PMS) controls which slice of rows a user sees.
- Push notifications, WhatsApp links, and email-rule events fire on key transitions.

Module	Page (client)	Route (server)	API base	Primary table
Delegations	<code>pages/Delegation.jsx</code>	<code>routes/delegations.js</code>	<code>/api/delegations</code>	<code>delegations</code>
PMS Tasks	<code>pages/PMSTasks.jsx</code>	<code>routes/pmstasks.js</code>	<code>/api/pms-tasks</code>	<code>pms_tasks</code>
Checklists	<code>pages/CheckLists.jsx</code>	<code>routes/hr.js</code>	<code>/api/hr/checklists</code>	<code>checklists</code> , <code>checklist_completions</code>
Complaints	<code>pages/Complaints.jsx</code>	<code>routes/complaints.js</code>	<code>/api/complaints</code>	<code>complaints</code>
Help Tickets	<code>pages/HelpTickets.jsx</code> + <code>components/HelpTicket.jsx</code>	<code>routes/support.js</code>	<code>/api/support</code>	<code>support_tickets</code>

9.1 Delegations

Business purpose

Delegations are free-form **internal tasks** that a senior person assigns to a team member, with a due date, an optional project tag, and a **proof-of-completion** requirement. The assignee uploads proof; an admin reviews and approves or rejects. It replaces the "I told them on WhatsApp and forgot" problem with an auditable task list and a per-person workload view.

Who uses it

- **Admin** and any user whose role matrix grants `delegations.create` or `delegations.can_approve` can **raise** tasks. (The MD, Ankur Kaplesh, is on a non-admin role but has full delegation rights via the matrix.)
- **Any user** can be an assignee and is the default recipient view.
- An **EA / supervisor** — defined as anyone with `can_approve` on the `delegations` module — gets the **"All tasks"** tab across every user and may upload proof on anyone's behalf.

Main screen, tabs & columns

The page opens on a **Workload Dashboard** (one row per person) followed by the task list.

Scope tabs (top of list):

- `Assigned to me (mine)` — default for everyone.
- `All tasks (all)` — shown only to admin / EA.

Filters: status dropdown, assignee name (admin/EA only), and a due-date range (`date_from` / `date_to`).

Task-list columns: S.No. · Task ID (`TSK-####`) · Description · Project · Assigned To · Due / Completed · Status · Upload Proof · Extension · Actions.

Workload Dashboard columns (`GET /dashboard`): Person · Total · Active · Completed · Delayed · Avg Delay (Days) · WIP Limit · Status.

- **WIP Limit** defaults to **5** per person.
- **Status** is derived:
 - `Overloaded` — active tasks > WIP limit.
 - `Constraint` — $\geq 25\%$ of tasks delayed **or** average delay > 5 days.
 - `OK` — neither.
- "Active" counts statuses `pending` , `submitted` , `rejected` ; "delayed" = `pending` / `submitted` with a past due date.

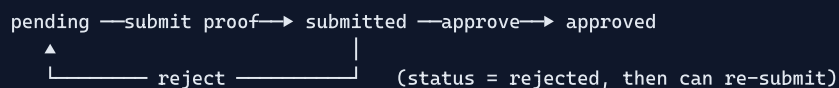
Key fields

Field	Notes
<code>title</code>	Auto-derived from first line of the description (first 80 chars); the UI no longer asks for it separately.
<code>description</code>	Required.
<code>assigned_by</code> / <code>assigned_to</code>	User IDs.
<code>due_date</code>	Optional.
<code>project_name</code>	Free text; not tied to any master list. Editable inline via <code>PATCH /:id/project</code> .
<code>attachment_url</code>	Optional brief/reference attached at creation.
<code>proof_url</code>	Uploaded on submission.
<code>status</code>	<code>pending</code> → <code>submitted</code> → <code>approved</code> / <code>rejected</code> .
Extension fields	<code>requested_due_date</code> , <code>extension_reason</code> , <code>extension_status</code> , <code>extension_reviewed_at/by</code> .

Step-by-step workflow

1. **Raise** — admin/EA fills description + assignee (+ optional due date, project, attachment). A duplicate guard blocks an identical *description* + *assignee* + *due-date* and points to the existing `TSK-` code. A push notification fires to the assignee.
2. **Work & submit proof** — the assignee uploads a proof file (`POST /:id/submit`). Status becomes `submitted` . Admin or EA can submit on the assignee's behalf (mam's EA uploads photos/PDFs received over WhatsApp).
3. **Review** — **admin only** approves (`approved`) or rejects with a mandatory reason (`rejected`). Rejected tasks resurface at the top of the assignee's list.
4. **(Optional) Extension** — the assignee requests a new due date with a reason (`POST /:id/request-extension`); **admin** approves (updates `due_date`) or rejects.

Assignment & status pipeline



- **Who can submit proof:** assignee, admin, or EA.
- **Who can approve/reject:** admin only.
- **Who can edit/delete:** the original assigner or admin (any status).

Recurrence / scheduling

Delegations have **no recurrence** — each is a one-off task. (Recurring routine work is handled by Checklists, §9.3.)

Notifications

- On **create**: web-push to the assignee ("📄 New Delegation").
- No email events are wired for delegations (unlike complaints / tickets).

API endpoints

Method	Path	Purpose	Who
GET	/api/delegations	List; ?scope=mine given all, ?status, ?assignee_id, ?date_from/to	Auth (scope gated)
GET	/api/delegations/dashboard	Per-person workload	Auth
GET	/api/delegations/stats	Homepage widget counts for current user	Auth
POST	/api/delegations	Create task	admin or create / approve perm
PUT	/api/delegations/:id	Full edit	assigner / admin
PATCH	/api/delegations/:id/project	Inline edit project tag	assigner / admin
POST	/api/delegations/:id/submit	Upload proof	assignee / admin / EA
POST	/api/delegations/:id/approve	Approve	admin
POST	/api/delegations/:id/reject	Reject (reason required)	admin
POST	/api/delegations/:id/request-extension	Request new due date	assignee / admin
POST	/api/delegations/:id/approve-extension	Approve extension	admin
POST	/api/delegations/:id/reject-extension	Reject extension	admin
DELETE	/api/delegations/:id	Delete	assigner / admin

DB table — delegations

id, title, description, assigned_by, assigned_to, due_date, status (pending / submitted / approved / rejected), proof_url, submitted_at, reviewed_at, reviewer_id, reject_reason, created_at. Migration-added columns: project_name, attachment_url, requested_due_date, extension_reason, extension_status, extension_reviewed_at, extension_reviewed_by.

9.2 PMS Tasks

Business purpose

PMS (Project Management System) Tasks are **delegations tied to a specific Business Book project**. The creator (typically CRM) picks a project; the task automatically captures the project's name and its **CRM owner** (read from the latest Client PO upload). The lifecycle mirrors delegations, but approval rights extend to the project's CRM owner so they can sign off on work for "their" site.

Who uses it

- **CRM executives** and anyone with pms_tasks.create raise tasks; admin bypasses all gates.
- **Assignees** carry out the task and submit proof.
- **Approvers:** the original assigner, the **project's CRM owner** (matched by name), or admin.

Main screen, tabs & columns

Scope tabs:

- My Tasks (mine) — default.
- Given by me (given).
- Followup (all active) (followup) — everyone's non-approved tasks.
- All (admin) (all) — admin / pms_tasks.approve only.

The selected scope is **persisted to the URL** (?scope=...) so a page refresh keeps the follow-up view.

Filters: status, CRM (assigner), assignee, and a date range (on COALESCE(due_date, created_at)).

Columns: Task ID · Project · Created By · Description · Assigned To · Due · Status.

Key fields

Field	Notes
<code>project_id</code>	Required; references <code>business_book(id)</code> .
<code>project_name_snapshot</code>	Captured server-side at create (site name → project name fallback).
<code>crm_name</code>	Captured from the most recent Client PO's <code>crm_name</code> for that project.
<code>description</code> / <code>title</code>	Title derived from first line of description.
<code>assigned_by</code> / <code>assigned_to</code> , <code>due_date</code> , <code>attachment_url</code> , <code>proof_url</code>	As in delegations.
<code>status</code> , extension fields	Identical to delegations.

The create form's project picker is fed by `GET /projects` , which returns each Business Book row with its latest `crm_name` pre-joined and de-duplicated on project/company/client.

Step-by-step workflow

1. **Pick project** — the creator selects a project; the CRM name and project label auto-fill (and cannot be spoofed by the client because they are re-derived on the server at insert).
2. **Create** — description + assignee (+ optional due date / attachment). Push to assignee ("  PMS — ").
3. **Submit proof** — assignee (or a PMS executive with `approve` , or admin) uploads proof → `submitted` .
4. **Approve / reject** — assigner, **CRM owner**, or admin. CRM-owner matching is name-based: exact match or first-token match (e.g. task CRM "Sushila" matches user "Sushila Kumari").
5. **(Optional) Extension** — same request/approve cycle as delegations (extension approval is admin-only).

Assignment & status pipeline

Identical states to delegations (`pending` → `submitted` → `approved/rejected`). The distinction is **who may approve**: `canApprovePmsTask` = admin **or** assigner **or** CRM owner (`isCrmOwner` name match).

Recurrence / scheduling

None — one-off, like delegations.

Notifications

- Web-push to the assignee on create. No email events.

API endpoints

Method	Path	Purpose	Who
GET	/api/pms-tasks/projects	Project dropdown w/ CRM pre-join	Auth
GET	/api/pms-tasks	List; ?scope , ?status , ?crm_id , ?assignee_id , ?date_from/to	Auth (scope gated)
POST	/api/pms-tasks	Create (server captures project + CRM)	create perm
PUT	/api/pms-tasks/:id	Edit (may re-target project)	assigner / admin
POST	/api/pms-tasks/:id/submit	Submit proof	assignee / approve / admin
POST	/api/pms-tasks/:id/approve	Approve	assigner / CRM owner / admin
POST	/api/pms-tasks/:id/reject	Reject (reason required)	assigner / CRM owner / admin
POST	/api/pms-tasks/:id/request-extension	Request extension	assignee / approve / admin
POST	/api/pms-tasks/:id/approve-extension	Approve extension	admin
POST	/api/pms-tasks/:id/reject-extension	Reject extension	admin
DELETE	/api/pms-tasks/:id	Delete	assigner / admin

DB table — pms_tasks

id , title , description , project_id , project_name_snapshot , crm_name , assigned_by , assigned_to , due_date , status , proof_url , submitted_at , reviewed_at , reviewer_id , reject_reason , requested_due_date , extension_reason , extension_status , extension_reviewed_at , extension_reviewed_by , created_at . (attachment_url added by migration.)

9.3 Checklists

Business purpose

Checklists are **recurring routine tasks** (daily / weekly / fortnightly / monthly / quarterly / yearly / once) assigned to a person or department. Each occurrence requires a proof — a photo, PDF, file, a text note, or nothing — and every completion goes into an **admin approval queue**. This gives mam a day-by-day view of which routine work was done, by whom, with proof, and whether she has signed it off.

Note: Checklists live under the **HR** route file (routes/hr.js) and are served from /api/hr/checkLists , even though the UI is its own page.

Who uses it

- **Admin** creates, edits, deletes, bulk-uploads checklists, and approves/rejects completions.
- **Every user** sees the checklists assigned to them (or unassigned ones, which apply to everyone) and marks them done.

Main screen, tabs & columns

Three views (toggle buttons):

1. **By Date** (by-date , default) — pick a date; see every checklist active that day with its completion + approval status. Includes a per-person summary and a "Today" jump.
2. **Manage / Current** (current , admin) — the master list grouped by assignee, plus Add / Bulk-add / Edit / Delete.
3. **Followup** (followup) — a back/forward window (default ±7 days) showing missed / today / done-pending / done-rejected rows so nothing slips.

Per-row completion states surfaced in Followup: missed , today , done_rejected count as still-pending.

Key fields

Field	Notes
<code>title</code> / <code>description</code>	Title derived from description.
<code>frequency</code>	<code>daily</code> , <code>weekly</code> , <code>fortnightly</code> , <code>monthly</code> , <code>quarterly</code> , <code>yearly</code> , <code>once</code> .
<code>due_date</code>	Anchor date; recurrence is computed from it (see below).
<code>due_time</code>	Optional HH:MM slot.
<code>assigned_to</code>	Person; <code>NULL</code> = applies to everyone.
<code>department</code>	Auto-filled from the assignee's <code>users.department</code> if not supplied.
<code>recurrence_start_date</code> / <code>recurrence_end_date</code>	Window; non-admins cannot complete outside it.
<code>proof_type</code>	One of <code>photo</code> / <code>pdf</code> / <code>file</code> / <code>text</code> / <code>none</code> (default <code>photo</code>); enforced on completion.
<code>proof_label</code>	Free-text hint of <i>what</i> to attach.
<code>fortnight_days</code>	For <code>fortnightly</code> , the two days of month (default <code>1,15</code>).

Completion (`checklist_completions`): `checklist_id` , `user_id` , `completion_date` , `proof_url` , `notes` , `submitted_at` , plus approval columns `approval_status` (default `pending`), `approved_by` , `approved_at` , `approval_note` . Unique per (`checklist` , `user` , `date`) .

Step-by-step workflow

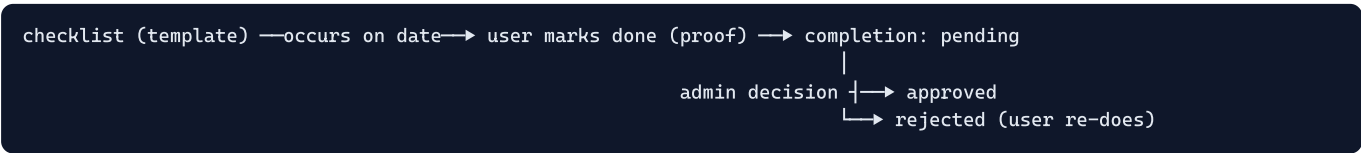
1. **Create** (admin) — single task, or **Bulk add**: paste many lines that share frequency / assignee(s) / dates / proof type. Bulk supports per-line overrides via `|` or tab — `description` | `proof_label` | `proof_type` | `time` — and a comma-separated **time** column expands into one row per time slot. Bulk can also assign to multiple users at once. A separate **Excel** path (`bulk-template.xlsx` → `parse-excel` → `bulk`) lets admin upload a spreadsheet.
2. **See today's tasks** — each user's dashboard widget / By-Date view lists checklists due that day, computed by frequency (below).
3. **Mark done** — the user attaches the required proof (photo/pdf/file) or note (text), optionally back-dating within the recurrence window. Re-submitting overwrites the proof (UPSERT) and resets approval to `pending` .
4. **Approve / reject** (admin) — each completion is decided via `.../completions/:id/decision` with `approved` or `rejected` + optional note.

"Due today" computation (`/my-today`)

Frequency	Due when
<code>daily</code>	every day
<code>weekly</code>	same weekday as <code>due_date</code>
<code>monthly</code>	same day-of-month as <code>due_date</code>
<code>quarterly</code>	every 3rd month on the <code>due_date</code> day
<code>yearly</code>	same month-and-day as <code>due_date</code>
<code>once</code>	exactly <code>due_date</code>

(`fortnightly` rows are created as fixed day-of-month occurrences via `fortnight_days` .)

Assignment & status pipeline



The checklist row itself carries a `status` (`pending` / `in_progress` / `completed` / `overdue`) used for the master list; the live per-day state comes from the completion's `approval_status` .

Notifications

No push/email events are wired on checklist actions; visibility is via the dashboard widget and the Followup view.

API endpoints (all under /api/hr)

Method	Path	Purpose	Who
GET	/checklists	List master checklists	Auth (own / all)
POST	/checklists	Create one	admin
POST	/checklists/bulk	Bulk create (multi-line, multi-user)	admin
GET	/checklists/bulk-template.xlsx	Download Excel template	Auth
POST	/checklists/parse-excel	Parse uploaded Excel (no writes)	Auth
PUT	/checklists/:id	Edit	admin
DELETE	/checklists/:id	Delete	admin
GET	/checklists/my-today	Today's due tasks for current user	Auth
POST	/checklists/:id/complete	Mark done (proof/notes, optional back-date)	Auth (assignee)
GET	/checklists/by-date?date=	All tasks + completion status on a date	Auth (own / all)
GET	/checklists/followup?back=&forward=	Missed/upcoming window	Auth
POST	/checklists/completions/:id/decision	Approve / reject a completion	admin

DB tables

- **checklists** — id, title, description, frequency, due_date, status, assigned_to, created_by, created_at; migration-added: due_time, department, recurrence_start_date, recurrence_end_date, proof_type, proof_label, fortnight_days.
- **checklist_completions** — id, checklist_id, user_id, completion_date, proof_url, notes, submitted_at, UNIQUE(checklist_id, user_id, completion_date); migration-added: approval_status, approved_by, approved_at, approval_note.

9.4 Complaints

Business purpose

The Complaints module is the **customer service desk**. Customers (or CRM on their behalf) register a complaint; CRM/admin assigns a **site engineer**; the client receives a **4-digit OTP** over WhatsApp; the engineer closes the complaint only after entering the OTP the client reads back — proving the customer actually confirmed the fix. The flow is structured as **Step 1 (Assign)** and **Step 2 (Resolve)**, with planned-vs-actual dates that compute a delay.

Who uses it

- **Customers** register via a **public, unauthenticated** form.
- **CRM / admin** (complaints.create / edit) register internally, assign engineers, and run the OTP/WhatsApp center.
- **Site engineers** (any user, picked from the engineer dropdown) carry out the fix and verify the OTP.

Main screen, tabs & columns

Tabs:

- All Complaints (all)
- + Register New (register)
- Step 1 – Assign (step1) — open complaints with no step1_assigned_to.
- Step 2 – Resolve (step2) — open/in-progress complaints that are assigned but have no service_report.
- Resolved (resolved) — status resolved or legacy closed.

The list supports search (name / complaint no / company / mobile) and category/status filters. Each row opens an **OTP / WhatsApp Center** modal driving the assign → OTP → verify flow.

Stat tiles (/stats): Total · Open · In Progress · Resolved (counts resolved + legacy closed) · by category.

Key fields

Field	Notes
<code>complaint_number</code>	Auto sequence <code>CMP-####</code> (starts at 1000).
<code>client_name</code> , <code>company_name</code> , <code>mobile_number</code>	Registration; name + mobile + problem are required.
<code>category</code> , <code>customer_type</code> , <code>complaint_type</code> , <code>state</code> , <code>emp_name</code>	Classification.
<code>problem_detail</code>	Required; mirrored into legacy <code>description</code> .
<code>step1_planned_date</code> / <code>_actual_date</code> / <code>_time_delay</code> / <code>_assigned_to</code>	Step 1 (assign). Delay = actual – planned in days.
<code>step2_*</code>	Step 2 (resolve) equivalents.
<code>service_report</code>	Engineer's closure note.
<code>assigned_engineer_id</code>	The locked engineer user (migration-added).
<code>resolution_otp</code> , <code>otp_generated_at</code> , <code>otp_verified_at</code> , <code>otp_attempts</code>	OTP state (migration-added).
<code>priority</code>	<code>low</code> / <code>medium</code> / <code>high</code> / <code>critical</code> .
<code>status</code>	<code>open</code> → <code>in_progress</code> → <code>resolved</code> (closed legacy).

Step-by-step workflow

- Register** — public (`POST /public`) or internal (`POST /`). A `CMP-` number is generated and a **Twilio WhatsApp + SMS confirmation** is auto-sent to the customer (fire-and-forget). Internal create also fires the `complaint.created` email rule and returns a click-to-send `wa.me` registration link as a fallback.
- Assign engineer (Step 1)** — `POST /:id/assign` with `engineer_user_id`. This locks the engineer, flips status `open` → `in_progress`, generates a fresh **4-digit OTP**, and returns **two WhatsApp links**: one to the engineer (job details, **no OTP**) and one to the client (carrying the OTP). Fires the `complaint.assigned` email rule.
- Dispatch WhatsApp** — when an admin clicks a link, `POST /:id/whatsapp/sent` timestamps which message (`register` / `engineer_assign` / `client_assign`) was sent — pure audit trail.
- Resolve (Step 2)** — the engineer enters the OTP the client reads off WhatsApp: `POST /:id/verify-otp`. On match → status `resolved`, `resolved_date` set, OTP wiped, `complaint.resolved` email fired. On mismatch → `otp_attempts++` with remaining-attempts feedback; locked out after 5 attempts.
- Resend OTP** — `POST /:id/resend-otp` issues a fresh OTP + new client WhatsApp link and resets the attempt counter.

`PUT /:id` is the general editor used to progress Step-1/Step-2 dates, assignment text, service report, status, and priority; it computes `step1_time_delay` / `step2_time_delay` automatically.

Assignment & status pipeline

```
open —assign engineer→ in_progress —verify OTP→ resolved
      ▲ (resend OTP)
```

- **Assign / edit / OTP actions:** `complaints.edit`.
- **Create:** `complaints.create`. **Delete:** `complaints.delete`. **View:** `complaints.view`.

Recurrence / scheduling

None — complaints are one-off, but they carry **planned vs actual dates** for both steps to measure SLA delay.

Notifications

- **WhatsApp / SMS:** auto Twilio confirmation on register; click-to-send `wa.me` links to engineer and client on assign (OTP only to client).
- **Email rules:** `complaint.created`, `complaint.assigned`, `complaint.resolved` (recipients resolved to creator, engineer, and director).

API endpoints

Method	Path	Purpose	Who
POST	/api/complaints/public	Public customer registration	none (public)
GET	/api/complaints	List (?status , ?category , ?search)	view
GET	/api/complaints/stats	Tiles	view
GET	/api/complaints/:id	One complaint	view
POST	/api/complaints	Internal create	create
PUT	/api/complaints/:id	Update (Step 1 / 2 progression)	edit
DELETE	/api/complaints/:id	Delete	delete
POST	/api/complaints/:id/assign	Assign engineer, mint OTP, return WA links	edit
POST	/api/complaints/:id/whatsapp/sent	Timestamp a sent message	edit
POST	/api/complaints/:id/verify-otp	Engineer verifies → resolve	edit
POST	/api/complaints/:id/resend-otp	New OTP + client link	edit

DB table — complaints

id, complaint_number, client_name, company_name, mobile_number, category, problem_detail, customer_type, complaint_type, emp_name, step1_planned_date/_actual_date/_time_delay/_assigned_to, step2_planned_date/_actual_date/_time_delay/_assigned_to, service_report, legacy_installation_id / po_id / description, priority, status (open / in_progress / resolved / closed), resolved_date, resolution_notes, created_by. Migration-added: state, remarks, updated_at, assigned_engineer_id, resolution_otp, otp_generated_at, otp_verified_at, otp_attempts, client_register_msg_sent_at, engineer_assign_msg_sent_at, client_assign_msg_sent_at.

9.5 Help Tickets

Business purpose

Help Tickets are the **in-app support desk** for staff. Any user can raise a ticket about a bug, a how-to question, an access issue, or a request, optionally directed at a specific colleague. There is a full **Help Tickets page** for triage and a **floating Help widget** (drag-to-move button, bottom-right) that is available on every screen.

Who uses it

- **Every staff member** can raise tickets (from the floating widget or the page).
- **Assignees** work the ticket and respond.
- **Raisers** decide when their own issue is closed.
- A **follow-up role** — anyone with `help_tickets.can_see_all` or `help_tickets.can_approve` — gets admin-like triage powers (see/close/reassign/respond on any ticket) without being a full admin. This is how mam gives one PC the job of chasing every ticket.

Main screen, tabs & columns

Help Tickets page tabs (with count badges):

- `Assigned to me ( mine )` — default.
- `Raised by me ( given )`.
- `All tickets ( all )` — labelled "(admin)" or "(follow-up)"; shown only to admin / follow-up role.

Filters: status dropdown + search (ticket no / subject / person).

Columns: Ticket (TK-####) · Subject · Raised By · Assigned To · Priority · Status · When · (actions).

Floating widget (`components/HelpTicket.jsx`) has two tabs: **Tickets** (list of your tickets + "Raise New Ticket") and **Learner** (built-in how-to guides for common tasks like adding a Business Book entry or requesting a payment).

Key fields

Field	Notes
ticket_no	Auto sequence TK-#### (starts at 1000).
subject , description	Both required.
category	bug (default), feature_request , how_to , access_issue , data_issue , manpower , material , payment , other .
priority	low / medium (default) / high / urgent .
module	Optional tag for which ERP area the ticket concerns.
attachment_link	Optional uploaded file.
assigned_to	Optional; when set, that user gets it on their dashboard.
admin_response	Reply from assignee / admin.
status	open → in_progress → resolved / closed .
resolved_by , resolved_at	Stamped on close.

Step-by-step workflow

1. **Raise** — from the widget or page, fill subject + description (+ category, priority, module, optional assignee and attachment). POST /support .
 - If **assigned**, push goes to that user; if **unassigned**, push goes to every active admin. The ticket.created email rule fires.
2. **Work it** — the **assignee** marks in_progress and adds an admin_response . (PUT /:id .)
3. **Close** — only the **raiser**, admin, or follow-up role may set resolved / closed ; the ticket.resolved email rule fires. Reassignment is admin / follow-up-role only.

Assignment & status pipeline

open —assignee works→ in_progress —raiser/admin closes→ resolved / closed

Permission rules enforced server-side in PUT /:id :

- **Admin / follow-up role:** anything (status, priority, response, reassign).
- **Raiser** (user_id): may close their own ticket; **cannot** reassign.
- **Assignee** (assigned_to): may set in_progress + respond; **cannot** close.

Recurrence / scheduling

None.

Notifications

- **Push:** to the assignee on create, or to all admins if unassigned (" SOS ...").
- **Email rules:** ticket.created and ticket.resolved (recipients: creator, assignee, director).

API endpoints

Method	Path	Purpose	Who
GET	/api/support	List; ?scope=mine given all , ?status	Auth (scope/see-all gated)
GET	/api/support/mine	"Assigned to me" widget (active count + recent 5)	Auth
GET	/api/support/stats	Admin dashboard counts	Auth
POST	/api/support	Raise ticket	Auth
PUT	/api/support/:id	Update status / response / priority / assignee	raiser / assignee / admin / follow-up (rules above)
DELETE	/api/support/:id	Delete	admin

DB table — support_tickets

id , ticket_no , user_id (raiser), subject , description , category , priority , status
(open / in_progress / resolved / closed), attachment_link , module , admin_response , resolved_by , resolved_at , created_at ,
updated_at . (assigned_to added by migration.)

9.6 Cross-module summary

Concern	Delegations	PMS Tasks	Checklists	Complaints	Help Tickets
Code prefix	TSK-	(numeric id)	(numeric id)	CMP-	TK-
Scopes	mine / all	mine / given / followup / all	by-date / current / followup	tab-by-stage	mine / given / all
Recurrence	no	no	yes (7 frequencies)	no	no
Proof required	yes (file)	yes (file)	yes (photo/pdf/file/text/none)	OTP confirmation	optional attachment
Approver	admin	assigner / CRM owner / admin	admin (per completion)	engineer via OTP	raiser / admin / follow- up
Push	yes	yes	no	—	yes
WhatsApp / SMS	no	no	no	yes (Twilio + links)	no
Email rules	no	no	no	created/assigned/resolved	created/resolved

10. Dashboards & Executive Views

This section documents the five dashboard surfaces in the SEPL ERP and the machinery that feeds them. There are two tiers:

- **Home Dashboard** — the landing page every authenticated user sees after login. Personal, operational, low-altitude.
- **Executive dashboards** — the **War Room**, the **Operating Console (CMD, Stage 1)**, the **TOC View (CMD, Stage 2)**, and the underlying **CMD Audit feed**. These are admin-only and exist for the MD/CMD/COO to read the whole business in 30 seconds.

The three executive React pages (`/dashboard/war-room` , `/dashboard/cmd` , `/dashboard/cmd-toc`) and their two data endpoints (`/api/dashboards/kpi` , `/api/dashboards/cmd-detail`) are all **admin-gated**. Two scheduled jobs — a **07:30 audit snapshot** and a **09:00 CMD email** — turn the same data into a daily push to the director.

10.1 Overview — the dashboard landscape

Dashboard	Route	Who sees it	Data source	Purpose
Home Dashboard	<code>/</code> (index)	Every logged-in user	<code>GET /api/dashboard</code> + personal endpoints	Personal tasks, checklists, attendance, recent rows
War Room	<code>/dashboard/war-room</code>	Admin only (<code>AdminRoute</code>)	<code>GET /api/dashboards/cmd-detail</code>	Director's CMD/COO traffic-light board + "do-not-show" RBAC plan
Operating Console (CMD Stage 1)	<code>/dashboard/cmd</code>	Admin only	<code>GET /api/dashboards/cmd-detail</code>	Section-by-section operating screen across all functions
TOC View (CMD Stage 2)	<code>/dashboard/cmd-toc</code>	Admin only	<code>GET /api/dashboards/cmd-detail</code>	Same data re-framed as a Theory-of-Constraints decision board
CMD Audit feed	<code>GET /audit</code> (+ <code>/data-quality</code> , <code>/analytics</code> , <code>/kpi</code>)	External / server-to-server (bearer token)	Direct SQLite	12 KPI tiles + exception lists; feeds the daily CMD email

Front-end access control

The three executive pages are wrapped in an `AdminRoute` guard in `client/src/App.jsx` :

```
function AdminRoute({ children }) {
  const { isAdmin, loading } = useAuth();
  ...
  return isAdmin() ? children : <Navigate to="/" />;
}

<Route path="dashboard/cmd"      element={<AdminRoute><DashboardCMD /></AdminRoute>} />
<Route path="dashboard/cmd-toc"  element={<AdminRoute><DashboardCMDToc /></AdminRoute>} />
<Route path="dashboard/war-room" element={<AdminRoute><DashboardWarRoom /></AdminRoute>} />
```

A non-admin who navigates to any of these URLs is bounced back to the Home Dashboard. The Home Dashboard itself is **not** admin-gated — it adapts its content based on `isAdmin()` .

Back-end access control

Endpoint	File	Auth
<code>GET /api/dashboard</code>	<code>server/routes/dashboard.js</code>	<code>authMiddleware</code> (any logged-in user)
<code>GET /api/dashboards/kpi</code>	<code>server/routes/dashboards.js</code>	<code>authMiddleware</code> + <code>adminOnly</code>
<code>GET /api/dashboards/cmd-detail</code>	<code>server/routes/dashboards.js</code>	<code>authMiddleware</code> + <code>adminOnly</code>
<code>GET /audit</code> , <code>/audit/*</code>	<code>server/routes/auditReport.js</code>	Bearer token (<code>AUDIT_API_TOKEN</code>), no session

10.2 Home Dashboard

File: `client/src/pages/Dashboard.jsx` (~337 lines) **Endpoint:** `GET /api/dashboard` — `server/routes/dashboard.js`

Business purpose

The Home Dashboard is the post-login landing page. It is deliberately **personal and operational** rather than executive: it tells the logged-in user what *they* must do today (tasks, checklists, support tickets, their attendance), and shows a small "recent activity" strip. The old top-of-page KPI tile row was deliberately removed.

KPI tiles were removed

The component still fetches `GET /api/dashboard`, but a documented decision (mam, 2026-05-22) **removed the eight colour-coded KPI tiles** that used to sit at the top — Total Leads / Won Deals / Active Orders / Installations / Open Complaints / Employees / Pending Expenses / Candidates. Each module already has its own page with richer filters, so the tiles were duplicating numbers without adding value. The data fetcher is intentionally left intact (no schema change) in case the tiles are reintroduced behind an admin toggle later.

What the page renders today

Widget	Visible to	Source	Notes
ERP-culture mantra banner	Everyone	<code>ErpMantraBanner</code> component	Rotates by day-of-year so the whole team sees the same quote at standup
This Month's Attendance (mini calendar)	Non-admins only	<code>GET /api/attendance/my-month</code>	Admin is skipped on purpose — they monitor everyone, so the "absent" figure would be noise
Support Tickets Assigned to You	Anyone with active tickets	<code>GET /api/support/mine</code>	Only shown when <code>active > 0</code> ; lists ticket no, priority, module, raiser, status
My Tasks (delegations)	Everyone	<code>GET /api/delegations?scope=mine</code>	Pending/rejected delegations with inline proof-upload "Submit" button
Today's Checklists	Everyone	<code>GET /api/hr/checklists/my-today</code>	Pending vs done; inline "Upload Proof" to complete
Recent Leads	Everyone	<code>stats.recentLeads</code> (5 rows)	Company / status / date
Recent Orders	Everyone	<code>stats.recentOrders</code> (5 rows)	PO number / amount / status
Recent Complaints	Everyone	<code>stats.recentComplaints</code> (5 rows)	Number / description / priority / status

The `/api/dashboard` payload

`server/routes/dashboard.js` is a single `GET /` handler (behind `authMiddleware`) that returns aggregate counts plus three recent-row lists:

Group	Fields
<code>leads</code>	<code>total</code> , <code>new</code> , <code>qualified</code> , <code>won</code> (from the legacy <code>leads</code> table)
<code>orders</code>	<code>total</code> , <code>totalValue</code> , <code>inProgress</code> (from <code>purchase_orders</code>)
<code>installations</code>	<code>total</code> , <code>pending</code> , <code>inProgress</code> , <code>completed</code>
<code>complaints</code>	<code>open</code> , <code>inProgress</code>
<code>hr</code>	<code>employees</code> (active), <code>candidates</code> , <code>subContractors</code> (active)
<code>expenses</code>	<code>pending</code> sum, <code>approved</code> sum
<code>recentLeads</code>	last 5 leads
<code>recentOrders</code>	last 5 POs
<code>recentComplaints</code>	last 5 complaints

The aggregate count groups (`leads`, `orders`, `installations`, etc.) are the data that backed the removed tiles — kept live but no longer rendered. The React page consumes only `recentLeads`, `recentOrders`, and `recentComplaints`.

Inline proof upload

Both "My Tasks" and "Today's Checklists" support uploading a proof file inline. The shared helper POSTs the file to `/api/upload`, then:

- a checklist is completed via `POST /api/hr/checklists/:id/complete` with the returned `proof_url`;
- a delegation submits via `POST /api/delegations/:id/submit` with the proof URL, after which it awaits admin approval.

10.3 The executive data layer

All three executive React pages read from **one consolidated endpoint** so the War Room, the Operating Console, and the TOC View can never disagree about the numbers. There are two compute functions and two routes.

Routes — `server/routes/dashboards.js`

```
router.use(authMiddleware);

// Same payload as /audit/kpi, but session-authed + admin-gated.
router.get('/kpi', adminOnly, (req, res) =>
  res.json(computeKpiPayload(getDb(), req.query.days)));

// Extended payload for BOTH CMD pages (Stage 1 + Stage 2) in one round-trip.
router.get('/cmd-detail', adminOnly, (req, res) =>
  res.json(computeCmdDetail(getDb(), req.query.days)));
```

- `/api/dashboards/kpi` re-uses `computeKpiPayload` from `server/routes/auditReport.js` — identical JSON to the external `/audit/kpi`, but authenticated by the user's normal JWT session instead of a bearer token, and locked to admins.
- `/api/dashboards/cmd-detail` calls `computeCmdDetail` from `server/utils/cmdDashboard.js` — a richer payload designed so the **same response feeds both** the Operating Console (Stage 1) and the TOC View (Stage 2). One fetch loads every section.

Both routes accept an optional `?days=N` window (clamped 7–365; defaults 90 for `cmd-detail`, 30 for `kpi`).

`computeCmdDetail(db, days)` — `server/utils/cmdDashboard.js`

A single function returns every list, breakdown, and chart series the two CMD pages need. Money figures are raw rupees. When the ERP does not yet capture the source data for a metric (e.g. EBITDA %, solar kW live, LTI safety days, quote-loss reasons), the field returns `null`; the front-end renders an em-dash with a "needs capture" tooltip rather than fabricating a number.

The response top-level keys:

Key	Contents
<code>pulse</code>	The 8-number headline: bank balance, free cash, runway days, order book (+ count), revenue MTD, CCC (with DSO/DIO/DPO), DPR adherence %, open snags, free inventory, WIP locked/unbilled, revenue per FTE, quote lead-time, lead→PO %
<code>cash</code>	Bank row, AR outstanding + 4 aging buckets, AP outstanding, top-5 debtors (with a derived "action today"), statutory dues calendar, 30-day cash forecast series, cost-of-inaction/day
<code>sales</code>	Funnel (leads→qualified→quoted→POs→in-execution→billed→collected), vertical mix, lead-source mix, 12-week booking trend (MEPF vs Solar), top customers, pipeline by stage, quote lead-time distribution, loss reasons, pending quotes, conversion by source
<code>operations</code>	Active sites (deduped), snags by priority, snag aging buckets, DPR adherence, on-time milestone %, sites past target date, materials-in-transit, tools-out
<code>it</code>	<code>sentry_active</code> boolean (green if Sentry DSN configured)
<code>inventory</code>	Total / free-to-use / reserved / slow-moving / dead-stock values
<code>procurement</code>	Top vendors by spend (with paid %)
<code>people</code>	Active FTE, headcount by dept, revenue per FTE (+ by dept), attendance today (present/late/leave/absent), KPI top/bottom 3
<code>customer</code>	Complaints by priority, predictive flags (slip-risk / churn-risk / cash-gap)
<code>data_quality</code>	Junk-PO list + count + total ₹ affected

NOTABLE DERIVED METRICS

- **Active sites** are deduped by linked `business_book` project (legacy PO re-uploads created duplicate site rows; mam, 2026-05-30). Sites with no BB link fall back to a normalized name key.

- **Cash conversion cycle (CCC)** = **DSO + DIO – DPO**, each computed over the selected window. **Free cash** = **bank – dues falling in the next 30 days**. **Runway** = **bank ÷ average daily outflow** over the last 30 days.
- **Top-5 debtors** each get an `action_today` string derived live from days-overdue: 90+ "Legal notice today", 60–89 "CEO call today", 30–59 "Site visit today", else "Follow-up email today".
- **Statutory dues** are pulled live from `statutory_dues_calendar` (label + `due_day`), resolved to "GST due 20-Jun" for the current/next month, with a red/amber/green status by days-out — replacing four old hardcoded `null` rows.
- **Funnel counts** come from the live `sales_funnel` (**11-stage**), not the near-empty legacy `Leads` table (which had made the War Room show "1 lead").
- **Junk POs**: a `business_book` PO number is "junk" if blank, a known dummy (`5252525` , `141414` , ...) or shorter than 10 characters. The count and ₹ total scan all junk rows so the Stage-1 escalation banner shows the true figure.
- **Materials-in-transit** (`indents` in `po_sent` / `dispatched`) and **tools-out** (`tools` in `in_use`) were previously literal em-dashes in the War Room; both are now live (mam, 2026-05-30 audit).
- **IT systems light** is a live boolean: green when a Sentry DSN exists in `app_settings` , amber otherwise — replacing a hardcoded amber light.

Shared UI kit — `client/src/components/cmdDashboardUi.jsx`

Both CMD pages import a common component/style library so they share one look:

Export	Role
<code>C</code> , <code>fmtINR</code> , <code>fmtNum</code> , <code>fmtPct</code>	Palette + formatters (₹ rendered as cr/L/K)
<code>PageHeader</code>	Top banner with title, tag, subtitle, right-rail meta
<code>SectionHead</code>	Section divider headings
<code>KpiTile</code>	Single coloured-stripe KPI tile (<code>accent</code> red/amber/green/blue/...)
<code>Card</code> , <code>MiniStat</code> , <code>Pill</code> , <code>Row</code>	Layout primitives
<code>FunnelBar</code> , <code>HBar</code> , <code>HeatCell</code> , <code>TicksList</code>	Chart primitives
<code>ConstraintBanner</code> , <code>TocStep</code>	TOC-specific blocks (binding constraint, Exploit/Subordinate/Elevate moves)
<code>StageTabs</code>	The two-tab switcher between <code>op</code> (Operating Console) and <code>toc</code> (TOC View)
<code>DataGap</code>	"needs capture" placeholder when a metric is <code>null</code>

10.4 War Room (Director's War Room)

File: `client/src/pages/DashboardWarRoom.jsx` (≈837 lines) **Route:** `/dashboard/war-room` (admin only) **Data:** `GET /api/dashboards/cmd-detail?days=90`

Business purpose

The War Room mirrors mam's HTML spec `SEPL_CMD_COO_Dashboard_v1.html` . It is the director's "single screen of glass" combining a CMD (director) board, a COO (operations) board, and an explicit **RBAC / do-not-show plan** for the future five-role rollout. All traffic lights and numbers are computed live from `/api/dashboards/cmd-detail` ; the default window is 90 days and is adjustable.

Three tabs

Tab	Audience	Contents
CMD VIEW (Director)	MD/CMD	Seven sections (below)
COO VIEW (Operations)	COO	Execution-only operating screen
DO-NOT-SHOW LIST	IT/Admin	What to hide from which role + the RBAC build sheet

CMD VIEW — seven sections

1. **Traffic Light (30-second read)** — six `TrafficCard` s: Cash, Sales, Delivery, People, Systems, Data Quality. Each light (red/amber/green) is auto-computed from the `cmd-detail` payload. A banner fires when ≥2 of Cash/Sales/Delivery/Data-Quality are red.
2. **Top 3 Bottlenecks (₹/day cost)** — the most expensive constraints, costed from the cost-of-inaction estimate.
3. **Today's 3 Decisions** — the three calls the director must make today, each with an owner and a deadline.

- 4. **Cash · Sales · Delivery** — hard numbers only: cash position, live sales funnel, delivery health.
- 5. **Pareto · Predictive · Exceptions** — "20% of customers = 80% revenue", predictive flags (next 14 days), and anomalies (>2σ today).
- 6. **Accountability** — top/bottom performers this week (Friday view), SLA breaches and culture flags.
- 7. **IT Head Watchlist** — the active IT build queue (e.g. RBAC for 5 roles).

COO VIEW

Execution-focused: today's site map (DPR + snag + risk), people (attendance/behaviour/performance with top/bottom KPI), procure-to-pay & inventory health (top vendors, inventory exceptions, 30-day cash-gap watch), and customer voice (complaints & tickets). Closes with the **COO routine**: 09:00 read the screen → 09:15 call the top-2 RED projects.

DO-NOT-SHOW LIST

A governance tab, not a data tab. It documents:

- **Hide from Sales / Junior Ops** — cash runway, bank balance, statutory dues (CMD + CFO only); customer churn-risk scores (CMD + COO + Sales Head only).
- **Hide from CMD daily** — daily attendance roll (COO & HR; CMD sees only anomalies); per-engineer DPR text (COO + line manager only).
- An **RBAC build sheet** matrix (Module × CMD/COO/CFO/HR/Sales/Site-Eng) to hand to the IT head.

10.5 Operating Console — CMD Stage 1

File: client/src/pages/DashboardCMD.jsx (≈540 lines) **Route:** /dashboard/cmd (admin only) **Data:** GET /api/dashboards/cmd-detail?days=90

Business purpose

The Operating Console mirrors mam's SEPL_CMD_TOC_Dashboard Stage-1 spec. It is a section-by-section **operating screen** that walks the whole business cycle top to bottom. It uses the shared UI kit and the StageTabs switcher to flip to the TOC View. Header tag: CMD VIEW , title "SEPL Operating Console".

Sections (in order)

#	Section	Key widgets
1	Pulse · 8 numbers that decide today	KPI tiles: Bank balance (+ runway), Order book (+ PO count / active sites), Revenue MTD, CCC days (DSO + DIO – DPO), DPR adherence, Open snags (+ oldest), Free inventory, Lead → PO %
2	Business cycle	Lead → Quote → PO → Site → Bill → Cash funnel bars + escalation banner
3	Vertical mix	Order-book split by business_book.category
4	Sales · Order Book · Customers	Top customers, pipeline by stage, conversion by source
5	Site execution · MEPF discipline split	DPR adherence, snag aging, sites past target
6	Procurement · Vendors · Inventory	Top vendors by spend, inventory split, cash position
7	Cash · AR aging · Collection	AR buckets, top debtors, 30-day cash forecast
8	People · Attendance · KPI	Attendance today, headcount, KPI top/bottom
9	Data quality · Junk POs in book	Junk-PO list + escalation count

The footer prints the refresh source (/api/dashboards/cmd-detail), spec_version , and generated_at so the viewer can see how fresh the data is. Metrics with no source data render through DataGap as an em-dash.

10.6 TOC View — CMD Stage 2

File: client/src/pages/DashboardCMDToc.jsx (≈546 lines) **Route:** /dashboard/cmd-toc (admin only) **Data:** GET /api/dashboards/cmd-detail?days=90 (same endpoint as Stage 1)

Business purpose

The TOC View mirrors mam's `SEPL_CMD_TOC_Dashboard_v3.html` . It renders the **same cmd-detail data** as the Operating Console but re-framed through the **Theory of Constraints** (Identify → Exploit → Subordinate → Elevate → Repeat). Instead of "here are all the numbers", it asks "what is *the one* binding constraint today, and what three moves attack it?" Header tag: `CMD · TOC v3` .

The binding-constraint engine

`bindingConstraint(data)` inspects the pulse metrics and picks the single binding constraint. For example, if Lead → PO conversion is below the 40% TOC threshold, the pipeline leak is named the binding constraint until conversion improves (focus the quote-in-4-days rule and subcontractor close-out discipline). If every pulse metric is inside its threshold, it advises running a weekly TOC review so the system does not drift.

Sections (in order)

#	Section	Focus
1	Pulse · 8 numbers (TOC-aligned)	Cash on hand, free cash, CCC, etc.
2	Identify · Today's binding constraint	ConstraintBanner — the single chosen constraint + why
3	Problem #1 · Cash flow	CCC waterfall, AR aging, 30-day forecast
4	Problem #2 + #3 · Quote lead time · Lead → PO conversion	Quote-LT distribution, conversion by source
5	Problem #4 · Operations / project management	DPR, snags, sites past target
6	Inventory · TOC view	Every ₹ in inventory = ₹ not in cash (free/reserved/slow/dead)
7	Problem #5 · People — who to hire, whom to fire	Headcount, revenue per FTE, KPI top/bottom
8	Today's 3 moves · Exploit · Subordinate · Elevate	TocStep cards — e.g. hire a Collections Officer + deploy invoice-on-milestone automation

It closes with a TOC-discipline reminder: don't fix everything — identify the binding constraint, exploit it, then repeat.

Stage 1 ↔ Stage 2 navigation

Both pages render `<StageTabs active=... onChange=...>` . Selecting the other tab calls `nav('/dashboard/cmd')` or `nav('/dashboard/cmd-toc')` . Because both pages hit the same `cmd-detail` endpoint, switching tabs never re-derives numbers differently.

10.7 CMD Audit feed (/audit)

File: `server/routes/auditReport.js` (≈1014 lines) **Mount:** `app.use('/audit', require('./routes/auditReport'))` — **outside /api/*** on purpose, so an external scheduler can hit `securederp.in/audit` directly.

Business purpose

The Audit feed is a **read-only, token-authenticated JSON API** built to the Master Prompt v3 spec. Mam's MD requested it so an automated caller (the daily 9 AM email job, or any external scheduler) can pull a full data-integrity audit **without being given a user login**. It is the canonical source for the executive KPIs and for the exception lists that surface data-quality problems.

Token authentication

The router's own middleware (kept separate from the cookie/session `middleware/auth.js` , because this traffic is server-to-server) checks a shared secret:

- Set `AUDIT_API_TOKEN` (≥8 chars) in the server environment (`pm2 set ERP:AUDIT_API_TOKEN <token>` or `.env`).
- Callers pass it as `Authorization: Bearer <token>` **or** `?token=<token>` .
- If the token is unconfigured → `503 audit_token_unconfigured` .
- If the provided token mismatches → `401 unauthorized` .

Endpoints

Method	Path	Returns
GET	/audit	12 KPI tiles + 8 exception lists + DB metadata + summary
GET	/audit/data-quality	Per-table row counts, null-rate scorecard, quality score
GET	/audit/analytics?days=N	30-day rolling activity totals, by-day series, top vendors/clients
GET	/audit/kpi?days=N	Operating-cycle KPI payload (DSO/DIO/DPO/CCC, AR aging, WIP, funnel, rev/FTE, margin variance)

Note: `computeKpiPayload` is exported from this file and re-used by the session-authed, admin-gated `/api/dashboards/kpi` route, so the in-app dashboards and the external feed share one implementation.

GET /audit — the 12 KPI tiles

Each tile is `{ id, label, value, unit }`:

id	Label	Unit
active_sites	Active Sites	count
open_pos	Open Purchase Orders	count
mtd_sale_value	MTD Sale Value (ex-GST)	inr
mtd_received	MTD Cash Received	inr
outstanding_receivables	Total Outstanding Receivables	inr
overdue_receivables	Overdue (>60d) Receivables	count
dpr_submitted_today	DPRs Submitted Today	count
dpr_missing_today	Sites Missing DPR Today	count
pending_payment_requests	Pending Payment Requests	count
pending_indents	Pending Indents	count
active_employees	Active Employees	count
cheques_open	Cheques Awaiting Action	count

(The list yields 12 tiles; the spec is referred to as the "12 KPI tiles".)

GET /audit — the exception lists

The response groups exceptions under named keys; each carries a `description`, an `items` array, and a computed `count`. Items can be flagged `severity: 'critical'`.

Key	What it flags
<code>duplicates</code>	Records sharing key identifiers — BB client+project pairs, duplicate vendors / customers / item-master rows, and (critical) duplicate <code>purchase_orders.po_number</code>
<code>arithmetic_errors</code>	Computed total ≠ recorded total: sales/purchase bills where <code>amount + gst ≠ total</code> (>₹1), POs where <code>total ≠ Σ po_items.amount</code> (>₹10), DPR <code>grand_total_a - grand_total_b ≠ profit_loss</code>
<code>missing_required</code>	Blank critical fields: BB (<code>lead_no</code> / <code>client_name</code> / <code>po_amount</code>), POs, vendors (no phone/email, or missing GST/address/contact-person), customers, sites (address), active employees (phone)
<code>stale_records</code>	Open work-items past SLA: POs <code>received</code> >30d, receivables 90+ not escalated, complaints open >14d, snags open >30d, indents pending >14d
<code>schema_drift</code>	Columns the code depends on (<code>EXPECTED_COLUMNS</code>) that are missing/renamed — surfaced as <code>critical</code> so a bad deploy is caught before downstream queries break
<code>cashflow_recon</code>	One <code>company_name</code> rolling up >1 distinct <code>client_name</code> in the Cash Flow tracker (surprising rollups; mam, 2026-05-16)
<code>cashflow_sale_drift</code>	Invariant check: Cash-Flow project Sale total must equal the raw <code>business_book.SUM(sale_amount_without_gst)</code> ; any drift means a regression (e.g. a fan-out JOIN) — guards the bug fixed in commit 7d87429
<code>geofence_violations</code>	Attendance punches recorded outside the configured site geofence (haversine to nearest active geofence, beyond radius + 500 m buffer; >3 km flagged critical)

The top-level `summary` returns `kpi_count` , `total_exceptions` , and `critical_exceptions` , alongside `database` metadata (path, size, last-modified).

GET /audit/data-quality

Walks the same `EXPECTED_COLUMNS` set and, per table, reports row count, the last rowid, per-column `null_count` / `null_pct` , and a `quality_score` (100 minus the worst null-rate on a required column). A row-count-weighted `overall_quality_score` is returned at the top.

GET /audit/analytics

A 30-day (configurable via `?days=N`) rolling activity view:

- `totals` — new rows created in the window per entity (business book, POs, sales/purchase bills, DPR, indents, payment requests, cheques, complaints, snags).
- `by_day` — day-by-day series for DPR, sales bills, and cash inflows.
- `top_vendors` (by spend) and `top_clients` (by sale value).

GET /audit/kpi

The operating-cycle payload (also served at `/api/dashboards/kpi`): cash-conversion-cycle (`dso` , `dio` , `dpo` , `ccc`), AR outstanding + aging, AP outstanding, inventory total / free-to-use, sales in window, bank position, WIP (book/billed/unbilled), funnel (leads, won, lead→PO %, quote lead-time), revenue per FTE (overall + by department), on-time milestone %, and project-margin variance (avg + worst-5 POs). All money in raw rupees.

10.8 Daily scheduled jobs

Two crons (registered in `server/index.js`) turn the audit data into a daily push to the director. They mirror the snapshot/email cadence and both skip Sunday.

07:30 — daily audit snapshot

File: `server/scripts/dailyAuditSnapshot.js` **Registration:** `scheduleDailyAuditSnapshot()` (skip via `ERP_DISABLE_AUDIT_SNAPSHOT=1`)

Every morning at **07:30 local time** it pulls the same JSON the `/audit` , `/audit/kpi` , `/audit/data-quality` , and `/audit/analytics` endpoints return and writes it to `data/audit-snapshots/<YYYY-MM-DD>/<endpoint>.json` . Purpose:

1. the 09:00 email can read that snapshot instead of rerunning all queries;
2. the CMD/COO/Sales/Finance dashboards can show "as of this morning" values without hitting the DB on every load;
3. a permanent point-in-time history (bank / AR aging / WIP / CCC) for trend analysis.

Retention is **90 days**; older snapshot folders are pruned on each run. The job requires `AUDIT_API_TOKEN` to be configured and self-schedules to the next 07:30 with a drift-corrected `setTimeout` then a 24-hour interval (the same pattern as the nightly DB backup).

09:00 — CMD audit email

File: `server/scripts/dailyCmdEmail.js` **Registration:** `scheduleDailyCmdEmail()` (skip via `ERP_DISABLE_CMD_EMAIL=1`, Sunday off)

At **09:00 local time** it reads the 07:30 snapshot (`data/audit-snapshots/<today>/kpi.json` + `audit.json`) and emails a formatted HTML summary to the director address from Admin → Email Settings (`app_settings.email_director_to`). It degrades gracefully:

- SMTP not configured in `app_settings` → logs a skip, no error.
- Today's snapshot folder missing → computes the KPI payload **live** via `computeKpiPayload` so the email still goes out (marked "live fallback — 7:30 snapshot missing").

The email body mirrors the War Room CMD aesthetic (inlined CSS so every mail client renders it) and surfaces the pulse numbers, cash/CCC/AR, funnel, WIP, and exception counts.

Manual trigger

`server/index.js` also exposes an admin-only manual fire: `POST /api/admin/cmd-email/send-now` runs the same `runOnce()` immediately (without waiting for the 09:00 tick) for testing or an on-demand send.

10.9 Summary

- **Home Dashboard** (/) is the only non-admin dashboard — personal tasks, checklists, support tickets, attendance, and recent-row strips. Its old KPI tiles were intentionally removed; the data fetcher is kept live.
- **War Room, Operating Console (CMD Stage 1)**, and **TOC View (CMD Stage 2)** are admin-only React pages, all guarded by `AdminRoute` and all fed by the single admin-gated `GET /api/dashboards/cmd-detail` endpoint (`computeCmdDetail`) — guaranteeing the three executive views never disagree.
- The **CMD Audit feed** (/audit , plus /data-quality , /analytics , /kpi) is a token-authenticated, session-less JSON API: 12 KPI tiles, 8 exception lists, a data-quality scorecard, and rolling analytics. Its `computeKpiPayload` is re-used by the in-app `/api/dashboards/kpi` route.
- Two crons consolidate it into a daily director push: a **07:30 snapshot** to disk (90-day retention) and a **09:00 CMD email** (snapshot-backed, live fallback, Sunday off), with an admin manual-trigger endpoint.

11. Administration & Settings

This section documents the administrative and configuration surface of the SEPL ERP: the **Activity Log** (per-user keystroke tracking), **Location Tracking** (GPS geofence sites), **Database Backups**, the **AI Agent** (Claude-powered "Ask ERP" assistant + settings), **Email (SMTP)** configuration, **Email Triggers** (rule-based automated mails), and the **Audit Log** (every mutating request).

Every screen described here is **admin-only**. The relevant API routers each call `authMiddleware` followed by an `adminOnly` guard (or an equivalent `req.user.role !== 'admin'` check), so non-admin users cannot reach them even by hitting the API directly. The one partial exception is the AI Agent, whose read-only `/ask` and `/status` endpoints are gated by a granular `ai_agent.view` permission so non-admins can use the chatbot if granted — but the API-key settings page itself stays admin-only.

Users, Roles & Permissions are covered in detail in [Section 1](#). This section only cross-references them where another feature depends on them (for example, Email Triggers resolving recipients "by role", or the `track_location` opt-out flag set from User Management). See Section 1 for the full account-management, role, and permission model.

11.1 Activity Log (Daily Activity / Word Count)

Business purpose

Lets management measure how much real data-entry work each employee did on a given day. The metric is **total characters typed** into the ERP (the spec mam gave: typing `monika` counts as 6 characters). It answers questions like "who actually entered data today, and in which modules" and doubles as a fraud/attribution investigation tool.

Who uses it

Admin only (and, in practice, the MD reviewing daily output). Frontend: `client/src/pages/admin/WordCount.jsx`. API: `server/routes/wordcount.js` (mounted at `/api/admin/word-count`, `adminOnly`).

How it works (what is automated)

There is **no separate tracking table** — the Activity Log is derived entirely from the existing `audit_log` rows. For the chosen date (or date range) the server:

1. Selects every `audit_log` row in the window with `action IN ('CREATE', 'UPDATE', 'DELETE')`.
2. Parses each row's `body_summary` (a compact JSON copy of the request body) and walks it recursively.
3. Counts the **characters** in every string value, after filtering out non-content noise: pure numbers, URLs, file paths, ISO dates, hex/UUID-ish strings, `[REDACTED]` secrets, and identifier/metadata keys (`id`, `user_id`, `created_at`, `rate`, `amount`, `latitude`, etc. — see `SKIP_KEYS`).
4. Aggregates the totals by user, by module (`entity_type`), and by action.

Because `body_summary` is capped at 2000 chars by the audit middleware, very large submissions undercount; such rows are flagged `truncated` and the UI shows an amber "lower bound" caveat banner.

Main screen / fields

- **Hero card** — total characters entered for the selected date, plus entries, active users, and modules-used counters.
- **Date / range picker** — single day or `date_from` → `date_to`.
- **By User** table — characters + entries per user; **click a row** to open the drill-down.
- **By Module** table — characters + entries per ERP module (friendly labels from `MODULE_LABELS`, e.g. `quotations` → "BOQ & Quotations").
- **By Action** pill row — CREATE / UPDATE / DELETE breakdown.
- **"What's New in ERP" changelog** — auto-pulled from the deploy git log for the same date range (via `/api/admin/changelog`), so the MD can see which new features/fixes shipped each day. This is a sibling admin endpoint, not part of the word-count router.

Drill-down & verification tools

Clicking a user opens a modal listing every entry they made that day with: time (forced to **Asia/Kolkata**), **Δ gap** from the previous action (red `<60s`, amber `<5m`), action, module, entry label, **IP**, and **device** (shortened user-agent). A **burst detector** flags rapid-fire runs (`<60s` gaps) to surface batch-click / automation patterns.

A **Verify user** button (`/user-check/:user_id`) opens an account-forensics panel: the `users` row (including `created_at`), audit footprint (total actions, first/last action, distinct IPs, active days), the IPs that used the account, and the last 30 login events. This was built

to investigate "this user entered rows before their account existed" complaints (shared-session / reused-JWT scenarios).

API endpoints

Method & path	Purpose
GET /api/admin/word-count	Aggregates for a date/range (?date= or ?date_from=&date_to= , optional ?user_id=) — totals + <code>by_user</code> / <code>by_module</code> / <code>by_action</code> .
GET /api/admin/word-count/detail	Per-record breakdown for the drill-down (adds <code>ip</code> , <code>user_agent</code> , <code>body_preview</code>).
GET /api/admin/word-count/user-check/:user_id	Account-vs-audit forensic check: user row, audit summary, IPs, recent logins.
GET /api/admin/changeLog	(Sibling endpoint) git-log changelog for the same date range.

DB tables

- `audit_log` — the sole data source (read-only here).
- `users` — joined by the Verify-user check.

11.2 Location Tracking

Business purpose

Shows where each employee is, using the GPS pings already collected by the Attendance page. Two questions it answers: "where is everyone **right now**" (Live) and "where did one person go **between punch-in and punch-out**" (Timeline).

Who uses it

Admin only. Frontend: `client/src/pages/admin/Locations.jsx` . API: `server/routes/locations.js` (mounted at `/api/admin/locations` , `adminOnly`).

Data source (what is automated)

No new write path. The Attendance page pings `/attendance/track-location` every ~30 seconds while open, inserting into the `location_tracking` table. These admin views are **read-only** over that data. Users can be excluded via `users.track_location = 0` (set from User Management; admins and named excludes opt out). `COALESCE(track_location, 1)` means legacy users default to tracked.

Geofence sites

Registered sites (office / project geofences) live in `geofence_settings` (`site_name` , `latitude` , `longitude` , `radius_meters` default 200 , `active`). A ping's `site_name` is resolved against these circles by the attendance write path; the Timeline view overlays active geofences as faint blue circles on the route map.

Live tab

One card per user showing their **most recent** ping inside a staleness window (5 min – 12 hours; default 30, auto-refresh every 60 s). Each card classifies the user:

- **Site name (green)** — pinging now, inside a registered site.
- **Outside any site (amber)** — pinging now, not in any geofence.
- **OFFLINE — last at X (grey)** — no ping for 15+ min (`FRESH_MAX_MIN`); the cached site label is shown as stale so a green pill doesn't "lie" that they're still there.
- **GPS OFF (red)** — network reached the server but no GPS fix (permission denied / timed out); stored as `site_name = 'GPS_OFF'` .

Status chips count and filter each bucket. Each card links to **Google Maps** and to that user's **Today's Timeline**.

Timeline tab

Pick an employee + date. Shows punch-in/out times & addresses, hours worked, total distance moved, and ping count, plus an embedded **route map** (red polyline, green start / red last-seen markers, blue geofence circles) and a movement table. Each ping is tagged by **phase** — `before` in / `during` work / `after` out — based on the attendance punch times, and carries the straight-line distance from the previous ping (Haversine).

A **teleport detector** flags any ping requiring sustained travel above **120 km/h** (`SUSPICIOUS_KMH`) as `suspicious` (likely fake-GPS app or cell-tower glitch). Suspicious pings are tagged **FAKE**, **excluded from total distance**, and dropped from the drawn route. A "Draw Route

on Google Maps" button down-samples to ≤ 11 points (Google free-tier waypoint limit).

API endpoints

Method & path	Purpose
GET /api/admin/locations/live	Latest ping per user within <code>?stale_minutes=</code> (1–720, default 30); skips opted-out users; sorts GPS-off first, then in-site, then most-recent.
GET /api/admin/locations/timeline	<code>?user_id=&date=YYYY-MM-DD</code> — all pings for one user/day with distance-from-previous, phase tags, suspicious flags, attendance punches, and active geofences.
GET /api/admin/locations/users	Picker list — distinct users who have any ping and haven't opted out.

DB tables

- `location_tracking` — GPS pings (read here, written by Attendance).
- `geofence_settings` — registered site circles.
- `attendance` — punch in/out times & addresses for phase tagging.
- `users` — names, departments, `track_location` opt-out.

11.3 Database Backups

Business purpose

Protects the single-file SQLite database against corruption / accidental wipes. A consistent snapshot is taken automatically every night, the last 30 are kept on disk, and admin can download any of them (or trigger one on demand) before a risky operation.

Who uses it

Admin only. Frontend: `client/src/pages/admin/DatabaseBackups.jsx`. API: `server/routes/backups.js` (mounted at `/api/admin/backups`, `adminOnly`). Backup engine: `server/scripts/backup-db.js`.

How it works (what is automated)

- **Safe snapshots.** Uses `better-sqlite3`'s native `.backup()` API rather than a file copy. SQLite runs in WAL mode, so a plain `cp` could grab the DB mid-write and corrupt it; the backup API produces a guaranteed-consistent copy even while the server is writing.
- **Nightly schedule.** `scheduleNightly()` (started from `server/index.js` on boot) runs a backup at **02:00 local time** using a self-rescheduling one-shot `setTimeout` (so DST/timezone shifts are picked up each day). Can be disabled with `ERP_DISABLE_BACKUP_SCHEDULER=1`.
- **Retention.** Keeps the most recent **30** backups (`KEEP_COUNT`); older files are deleted on each run.
- **Storage location.** `ERP_BACKUP_DIR` env override, else `/root/erp-backups` on the VPS or `../..backups` on Windows — deliberately **outside** the `data/` folder so a data wipe doesn't also nuke the backup history.
- **Filenames.** `erp-YYYY-MM-DD-HH-mm-ss.db`.

Main screen / fields

- Summary cards: total backups, latest-backup age + timestamp, storage path.
- A blue recommendation banner nudging mam to **Download** the latest backup weekly to a cloud-synced laptop folder (VPS backups don't survive losing the VPS itself).
- Backup table: filename (latest tagged), created time, size, and a per-row **Download** button.
- **Backup Now** button for ad-hoc snapshots.

API endpoints

Method & path	Purpose
GET /api/admin/backups	List backups (filename, size, created_at) + the backup directory.
POST /api/admin/backups/run	Trigger a backup immediately; returns the new filename + size.
GET /api/admin/backups/:file/download	Stream a backup file for download. Filename is validated against the strict <code>erp-...db</code> pattern so the route can't be abused to read arbitrary VPS files.

DB tables

None — backups operate on the database **file** (`data/erp.db`), not on a table. The list is built from the filesystem.

11.4 AI Agent ("Ask ERP" assistant)

Business purpose

A floating "Ask ERP" chat bubble available on every page that answers natural- language questions about SEPL's own data, searches the live web for market prices, and trains staff through built-in module guides — in English or Hindi. mam's framing: *"real ai agent which scan from all over not only from my ERP."*

Who uses it

- The **chat bubble** is shown to any user with the `ai_agent.view` permission (component `client/src/components/AIAgentChat.jsx`); it checks `/api/ai-agent/status` to render only when configured.
- The **AI Settings** page is **admin-only**: `client/src/pages/AISettings.jsx`, backed by `server/routes/aiAgent.js`.

What the assistant can do

The `/ask` endpoint gives Claude **three tools**:

1. **query_database** — runs a single **read-only** SQL query (`SELECT / WITH` only) against an allow-listed set of business tables (`READABLE_TABLES` — leads, customers, item_master, quotations, POs, payments, DPR, attendance, employees, etc.; sensitive auth tables are excluded). A `validateSelect` filter rejects multiple statements and any mutating keyword (`INSERT/UPDATE/DELETE/DROP/ALTER/PRAGMA/...`), and a `capSql` guard auto-adds `LIMIT 501` when a query has none (max 500 rows returned). The schema digest is injected into the system prompt.
2. **web_search** — Anthropic server-side web search, used proactively for material/commodity prices, GST lookups, vendor details, etc. The assistant presents the ERP's stored rate **and** today's market rate side by side.
3. **get_module_guide** — returns built-in step-by-step guides for major workflows (DPR, Indent, Price Required, Vendor PO, Purchase Bill, Sales Bill, Delivery Challan, Order Planning, Business Book, Quotation, Cash Flow, Expense, Attendance, DPR loss alert) so the bot can train staff when asked "how to..." / "kaise karte hai...".

The assistant **can only READ** — it cannot edit, delete, or send anything. The system prompt also injects the **current user's identity** (name, role, designation) so "who am I / show my attendance" works without re-asking, and instructs Hindi/Roman-Hindi replies for training questions while keeping module and button names in English.

The agent loops up to `MAX_TOOL_ITER` (5) tool rounds. Responses are streamed with a 12 s whitespace **heartbeat** (and `X-Accel-Buffering: no`) so nginx's 60 s `proxy_read_timeout` doesn't kill long Opus + web-search calls; the overall Anthropic timeout is 150 s. Errors ride back in the JSON body (status is locked to 200 once streaming starts) and the frontend checks `data.error` before `data.answer`.

Where the API key is set, and the model

- The **Anthropic API key** is pasted in **Admin** → **AI Settings** and stored server-side in the `app_settings` key-value table under `ai_api_key` (no `.env` edit, no SSH). `GET /settings` only returns a **masked** key (`sk-ant-...abcd`) and an `api_key_set` flag — the secret is never sent back to a browser.
- **Provider** (`ai_provider`) is `anthropic` (the only option).
- **Model** (`ai_model`) is selectable in the UI. Adaptive thinking + `effort: high` + web search are enabled for Opus / Sonnet-4.6-class models (`supportsAdaptive` regex); the default stored model id is `claude-opus-4-7`. (Note: the in-product UI model picker labels may lag the deployed default — the authoritative default is whatever `app_settings.ai_model` holds, falling back to `claude-opus-4-7`.)

There is also a **rate-intelligence** feature on this router (used inside the BOQ form, gated by `quotations.view`): `/rate-suggestion` and `/item-history` surface the last rate quoted to a client + 6-month stats from `item_price_history`.

API endpoints

Method & path	Auth	Purpose
GET /api/ai-agent/settings	admin	Provider, model, <code>api_key_set</code> , masked key.
PUT /api/ai-agent/settings	admin	Save provider / model / API key.
GET /api/ai-agent/status	<code>ai_agent.view</code>	Whether the chatbot is configured (so the bubble renders).
POST /api/ai-agent/ask	<code>ai_agent.view</code>	The chatbot — <code>{ question, history? }</code> → <code>{ answer, sql_runs, ... }</code> .
GET /api/ai-agent/rate-suggestion	<code>quotations.view</code>	Last-quoted + 6-month rate stats for a BOQ item.
GET /api/ai-agent/item-history	<code>quotations.view</code>	Full price history for an item.

(The same router also hosts the **Email Settings** endpoints — see §11.5.)

DB tables

- `app_settings` — stores `ai_provider`, `ai_api_key`, `ai_model`.
- `item_price_history`, `item_master`, `leads` — rate-intelligence reads.
- All `READABLE_TABLES` — read at query time by the chatbot's SQL tool.

11.5 Email (SMTP) Settings

Business purpose

Holds the outbound SMTP credentials the ERP uses to send automated mail — the DPR loss-streak alert to the director and, more broadly, every Email Trigger (§11.6). Like the AI key, it's configured inside the ERP so mam can paste Gmail / SendGrid / Mailgun credentials without touching `.env`.

Who uses it

Admin only. Frontend: `client/src/pages/EmailSettings.jsx`. API: the `server/routes/aiAgent.js` router (`/api/ai-agent/email-settings`, `adminOnly`). Sending helper: `server/lib/email.js`.

Main screen / fields

Field	Setting key	Notes
SMTP Host	<code>email_smtp_host</code>	e.g. <code>smtp.gmail.com</code>
Port	<code>email_smtp_port</code>	default <code>587</code>
Use TLS/SSL	<code>email_smtp_secure</code>	<code>'1'</code> for 465/TLS, blank for STARTTLS
Username (full email)	<code>email_smtp_user</code>	
Password / App Password	<code>email_smtp_pass</code>	never echoed back; only a masked tail is returned. Gmail needs a 16-char App Password — a normal password will not work
From address	<code>email_from</code>	optional display From; falls back to the SMTP user
Director email	<code>email_director_to</code>	default alert recipient; falls back to <code>director@securedengineers.com</code>

A status banner shows configured / not-configured, and a **Send test** button mails a confirmation to a chosen address.

How it works (what is automated)

`lib/email.js` builds a `nodemailer` transport from the `app_settings` keys. `sendEmail()` is **graceful**: if host/user/pass aren't all set it returns `{ skipped: true }` instead of throwing, so callers (loss-streak alert, email triggers) never break their own request path when SMTP is unconfigured. `nodemailer` is lazy-required so the server still boots without it in dev. A per-call `from` override is supported (used by per-rule dynamic senders), though Gmail/most providers only honour a From that matches the authenticated account or a verified alias.

API endpoints

Method & path	Purpose
GET /api/ai-agent/email-settings	Return SMTP config (password masked, <code>pass_set</code> flag).
PUT /api/ai-agent/email-settings	Save SMTP config.
POST /api/ai-agent/email-test	Send a test email (defaults to the director address).

DB tables

- `app_settings` — all `email_*` keys.

11.6 Email Triggers (rule-based automated emails)

Business purpose

Lets admin build **dynamic email rules**: when `<event>` fires AND `<conditions>` match, email `<recipients>` using a `{{variable}}` template. This generalises one-off alerts into a configurable engine (mam 2026-06-03: "lots of email with trigger and pattern, dynamic").

Who uses it

Admin only (rule CRUD). Frontend: `client/src/pages/EmailTriggers.jsx` . API: `server/routes/emailRules.js` (`/api/email-rules`). Engine: `server/lib/emailRules.js` . Event catalog: `server/lib/emailEvents.js` . It sends through the same SMTP transport as §11.5.

Main screen / fields

The page lists existing rules (name, event, recipients summary, ON/OFF toggle, last-fired + fire count, and test/edit/delete actions). The rule editor modal captures:

- **Event** — chosen from a grouped catalog (Indent, DPR & Site, Payments & Bills, Complaints & Support, HR).
- **Recipients** — any combination of: **from the record** (dynamic people such as the indent raiser, project CRM owner, director); **by role** (every active user holding the role, resolved live from `users` / `user_roles` / `roles`); and a **fixed** comma/newline list.
- **Conditions** (optional, AND-combined) — `field op value` rows with operators `equals` / `not equals` / `contains` / `>` / `<` . Empty = always send.
- **Template** — optional per-rule **From**, a **Subject** template, and a **Body** template, all with click-to-insert `{{variable}}` tokens drawn from the selected event's `vars` .
- **Enabled** checkbox (pause without deleting).

A **Send test** action renders the rule with realistic `SAMPLE_CONTEXT` data and can force a recipient so the admin previews it on themselves.

How it works (what is automated)

Server code calls `fireEmailEvent(eventKey, context)` at the relevant moment (e.g. `indent.approved` , `dpr.loss_streak` , `payment.requested` , `complaint.created` , `leave.requested`). This is **fire-and-forget** (`setImmediate` , never throws into the request path). The engine then, for each **enabled** rule on that event:

1. Evaluates conditions against the context (all must pass).
2. Resolves recipients (dynamic-from-record emails + fixed list + by-role users); skips if none resolve.
3. Renders the subject/body/From templates (`{{var}}` substitution, case- and space-tolerant; body escaped to minimal HTML preserving line breaks).
4. Sends via `lib/email.sendEmail` , then stamps `last_fired_at` and increments `fire_count` .

Events are **wired in across the codebase** — `fireEmailEvent` is called from `procurement.js` , `paymentrequired.js` , `dpr.js` , `attendance.js` , `support.js` , `complaints.js` , and `index.js` . Adding a new trigger source = (1) add an entry to `emailEvents.js` , (2) call `fireEmailEvent(key, context)` with the documented `vars` and `*_email` people; the UI and engine pick it up automatically.

Event catalog (live events)

Group	Event key	Fires on
Indent	<code>indent.raised</code> , <code>indent.crm_approved</code> , <code>indent.ll_approved</code> , <code>indent.approved</code> , <code>indent.rejected</code>	Indent lifecycle stages
DPR & Site	<code>dpr.loss_streak</code>	3+ consecutive site loss days
Payments & Bills	<code>payment.requested</code> , <code>payment.approved</code> , <code>payment.rejected</code> , <code>bill.uploaded</code>	Payment approval flow & purchase bills
Complaints & Support	<code>complaint.created</code> , <code>complaint.assigned</code> , <code>complaint.resolved</code> , <code>ticket.created</code> , <code>ticket.resolved</code>	Complaint / help-ticket lifecycle
HR	<code>leave.requested</code> , <code>leave.decided</code> , <code>task.assigned</code>	Leave & delegation events

Each event declares its `vars` (template variables), `people` (dynamic recipient emails resolvable from context), and condition-able `fields` .

API endpoints

Method & path	Auth	Purpose
GET <code>/api/email-rules/events</code>	auth	Event catalog + role list + sample context (drives the editor).
GET <code>/api/email-rules</code>	admin	List all rules.
POST <code>/api/email-rules</code>	admin	Create a rule.
PUT <code>/api/email-rules/:id</code>	admin	Update a rule.
PUT <code>/api/email-rules/:id/toggle</code>	admin	Enable/disable.
DELETE <code>/api/email-rules/:id</code>	admin	Delete.
POST <code>/api/email-rules/:id/test</code>	admin	Send a test render of one rule.

DB tables

- `email_rules` — one row per rule (`event_key` , `enabled` , `conditions` JSON, `recipients` JSON, `from_addr` , `subject_tpl` , `body_tpl` , `last_fired_at` , `fire_count`).
- `users` / `user_roles` / `roles` — recipient resolution "by role".
- `app_settings` — SMTP credentials used to actually send (via §11.5).

11.7 Audit Log

Business purpose

A tamper-evidence trail of **every mutating request** (create / update / delete, plus custom actions like login). It's both a compliance/forensics record and the raw data behind the Activity Log (§11.1).

Who uses it

Admin only. Frontend: `client/src/pages/admin/AuditLog.jsx` . Query API: `server/routes/audit.js` (`/api/admin/audit` , `adminOnly`). Capture: `server/middleware/audit.js` .

How it works (what is automated)

`auditMiddleware` runs **after** `authMiddleware` (so `req.user` is set) and, on response `finish` , inserts a row into `audit_log` . Key behaviours:

- Non-blocking** — logging is fire-and-forget; any exception inside the middleware is swallowed so it can never crash or slow the real request.
- Mutating methods only** — POST/PUT/PATCH/DELETE; the action is auto-derived (`POST`→`CREATE` , `PUT/PATCH`→`UPDATE` , `DELETE`→`DELETE`).
- Secret stripping** — body keys `password` , `current_password` , `new_password` , `token` , `authorization` , `secret` are [REDACTED] before the body is summarised; the summary is capped at 2000 chars.
- Entity inference** — `entity_type` is the path segment after `/api/` ; `entity_id` is the last numeric path segment.

- **Captured context** — user id/name/role, method, path, query, body summary, status code, **IP** (`x-forwarded-for` first hop), and **user-agent**.
- **Noise filter** — `SKIP_PATH_PREFIXES` excludes high-frequency / read-y endpoints (location pings, `/api/dashboard` , `/api/audit` itself, uploads, `/api/auth/me` , `/api/auth/login`). Login is logged by the login route itself (with proper attribution and `LOGIN_FAIL` rows), avoiding `(unknown)` duplicates.
- **Opt-out / debug** — `ERP_DISABLE_AUDIT=1` disables capture; `ERP_AUDIT_DEBUG=1` logs each step.

A helper `logAuditEvent()` lets routes record richer entries manually — friendly `entity_label` , custom actions (`APPROVE` / `REJECT` / `LOGIN` / `LOGIN_FAIL`), and `before_json` / `after_json` snapshots.

Main screen / fields

- Summary strip: total entries, unique users, modules logged, action types.
- Filters: free-text search (path / body / label / user), user, module (`entity_type`), action, and date range; paginated (default 50, max 500/page).
- Action pills are colour-coded (CREATE green, UPDATE blue, DELETE red, LOGIN violet, LOGIN_FAIL red); status code coloured by HTTP class.
- Click a row to view its full body and optional before/after snapshot.

API endpoints

Method & path	Purpose
GET <code>/api/admin/audit</code>	Filtered, paginated list (<code>user_id</code> , <code>entity_type</code> , <code>action</code> , <code>date_from</code> , <code>date_to</code> , <code>q</code> , <code>page</code> , <code>limit</code>).
GET <code>/api/admin/audit/meta</code>	Distinct users / entity types / actions for the filter dropdowns.
GET <code>/api/admin/audit/:id</code>	One entry with full before/after JSON.

Note: there is a separate `/audit` (no `/api` prefix, `routes/auditReport.js`) that serves the external "CMD Audit" KPI/exception report for the scheduled daily-audit email — that is a reporting endpoint, distinct from this activity Audit Log.

DB tables

- `audit_log` — the single capture table (`at` , `user_id` , `user_name` , `user_role` , `action` , `entity_type` , `entity_id` , `entity_label` , `method` , `path` , `query` , `body_summary` , `status_code` , `ip` , `user_agent` , `before_json` , `after_json`). Indexed post-migration.

11.8 Users, Roles & Permissions (cross-reference)

User accounts, the role catalogue, and the granular module-level view/create/edit/approve permission grid are managed under **Administration** as well, but are documented in full in **Section 1**. They intersect with this section in three places:

- **Email Triggers** resolve "by role" recipients from `users` / `user_roles` / `roles` (§11.6).
- **Location Tracking** honours the per-user `users.track_location` opt-out flag set from User Management (§11.2).
- The **AI chatbot** is gated by the `ai_agent.view` permission, while the AI key settings page is admin-only (§11.4).

See **Section 1** for the complete user/role/permission model.

12. Automations, Scheduled Jobs & Integrations

This section documents everything the SEPL ERP does **on its own** — the cron-style schedulers wired up at server boot, the one-shot maintenance scripts in `server/scripts/`, the third-party integrations (email, web-push, Twilio, Anthropic Claude, Sentry, Excel/PDF parsing), and the unified notifications model (in-app bell + push + email + SMS/WhatsApp).

All facts below are drawn directly from `server/index.js`, the `server/scripts/` directory, and the integration libraries under `server/lib/`, `server/services/`, and `server/utils/`.

12.1 How scheduling works (no node-cron)

The ERP does **not** use `node-cron` or any external scheduler. Every job uses the same hand-rolled **drift-corrected** `setTimeout` → `setInterval` pattern:

1. On boot, compute the milliseconds until the next target local time (e.g. next 02:00).
2. `setTimeout` fires once at that time.
3. From inside that first run, a `setInterval(fn, 24h)` keeps it repeating daily (or hourly / every 30 min for the higher-frequency jobs).

The nightly backup goes one step further: it **re-chains a fresh** `setTimeout` after each run instead of using a fixed interval, so DST / timezone shifts are absorbed automatically each day.

Because all timers are based on **local server time** (the VPS clock), "02:00" means 02:00 in the server's timezone. There is no UTC normalisation.

Each scheduler is wrapped in a `try/catch` in `server/index.js`, so if any one scheduler fails to start (e.g. a missing package), the server still boots and logs a `[<name>] Scheduler not started: <reason>` warning rather than crashing.

Disable flags (ENV)

Every recurring job can be turned off with an `ERP_DISABLE_*` environment variable (set to `1`). These are read at boot from `.env` or the `pm2` process env. Set them locally to keep dev machines quiet, or to pause a job in production without a code change.

12.2 Master table — every scheduled job

Job	Module (<code>erp/erp.js</code>)	Schedule (local time)	What it does	Disable flag
Nightly DB backup	<code>backup-db.js</code> → <code>scheduleNightly()</code>	Daily 02:00 (self-rechaining)	Copies the SQLite DB to <code>~/erp-backups</code> (VPS) / <code>../backups</code> (Windows); keeps the last 30 backups. Also exposed via <code>/api/admin/backups/*</code> for manual list/download/trigger.	<code>ERP_DISABLE_BACKUP_SCHEDULER</code>
Daily audit snapshot	<code>dailyAuditSnapshot.js</code> → <code>scheduleDailyAuditSnapshot()</code>	Daily 07:30	Writes the <code>/audit</code> JSON into <code>data/audit-snapshots/<date>/</code> so the 09:00 CMD email and the role dashboards render from a fixed "as of this morning" file.	<code>ERP_DISABLE_AUDIT_SNAPSHOT</code>
DPR auto-prompt	<code>dprAutoPrompt.js</code> → <code>scheduleDprAutoPrompt()</code>	Daily 18:00 (Sunday off)	Notifies every site engineer who hasn't submitted today's DPR for their active site(s); sends admins a rollout when overall adherence is below 50%.	<code>ERP_DISABLE_DPR_PROMPT</code>
Cash rollover	<code>cashFidelityCron.js</code> → <code>scheduleCashFidelity()</code>	Daily 00:00	Rolls over <code>cash_flow_daily</code> (creates the new day's row) so runway numbers don't drift on no-collection days.	<code>ERP_DISABLE_CASH_CRON</code>
Receivables ageing refresh	<code>cashFidelityCron.js</code> → <code>scheduleCashFidelity()</code>	Daily 01:00	Recomputes receivables ageing across the board — the primary truth, replacing the manual "Refresh Ageing" button.	<code>ERP_DISABLE_CASH_CRON</code>
Fire NOC auto-pilot	<code>fireNocCron.js</code> → <code>scheduleFireNocCron()</code>	Hourly (first run +60s after boot)	Keeps Fire-NOC cycle stages + statuses in sync with elapsed days via <code>syncAllActiveCycles()</code> . Also runs a one-time idempotent boot backfill for legacy rows.	<code>ERP_DISABLE_FIRE_NOC_CRON</code>
HR automations	<code>hrAutomationsCron.js</code> → <code>schedule()</code>	Every 30 min (first run +10s after boot)	Scans for interview reminders (next 24h), stale offers (> 7 days, not accepted/declined), and pending hiring-request approvals (> 24h). Creates in-app notifications + emails HR users. Dedupe-keyed so it never spams.	<code>ERP_DISABLE_HR_CRON</code>
Procurement schedule reminder	<code>procurementReminderCron.js</code> → <code>scheduleProcurementReminderCron()</code>	Daily 09:00 (plus a +60s boot catch-up)	Scans the procurement schedule for indent rows whose <code>end_date</code> is tomorrow's business day; posts an announcement + push for each. A dedup table prevents double-posts.	<code>ERP_DISABLE_PROCSCH_REMINDER</code>
Daily CMD email	<code>dailyCmdEmail.js</code> → <code>scheduleDailyCmdEmail()</code>	Daily 09:00	Reads the 07:30 audit snapshot (falls back to live <code>/audit/kpi</code>) and emails the director address from Admin → Email Settings.	<code>ERP_DISABLE_CMD_EMAIL</code>
Fortnightly installation billing	<code>installationBillingCron.js</code> → <code>scheduleInstallationBillingCron()</code>	Daily 08:00 check; acts only on	Auto-generates Type-3 (Against-Installation) sales bills from approved DPRs (work value ×	<code>ERP_DISABLE_INSTALL_BILLING</code>

Job	Module (<code>server/scripts/</code>)	Schedule (local time)	What it does	Disable flag
		the 1st & 16th (plus a +90s boot catch-up)	Against-Installation %). Idempotent; bills are approved but a human still clicks "Sent to Client".	

One-shot, idempotent jobs (run once on boot, then never again)

These run at boot, guard themselves with an `app_settings` flag (or a similar marker) so they execute **exactly once** after deploy, and then become no-ops on every subsequent restart.

Job	Module	Purpose	Disable flag
Item Master cleanup	<code>itemMasterCleanup.js</code> → <code>runOnce()</code>	Corrects item-master units to market values, removes duplicates, fixes spelling. Guarded by <code>app_settings.item_master_cleanup_v1</code> .	<code>ERP_DISABLE_ITEM_CLEANUP</code>
Itemwise rate import v1	<code>itemwiseRateImport.js</code> → <code>runOnce()</code>	Reads <code>data/itemwise-rates-2026-06-01.json</code> (140 rows) → updates <code>item_master.current_price</code> .	<code>ERP_DISABLE_ITEM_RATE_IMPORT</code>
Itemwise rate import v2	<code>itemwiseRateImportV2.js</code> → <code>runOnce()</code>	Second-pass broader CSV export (2,496 rate rows). Separate <code>app_settings</code> flag.	<code>ERP_DISABLE_ITEM_RATE_IMPORT</code> (shared)
Itemwise rate import v3	<code>itemwiseRateImportV3.js</code> → <code>runOnce()</code>	Third-pass 101 daybook-matched rows from <code>final item.xlsx</code> ; adds a code-ci fallback.	<code>ERP_DISABLE_ITEM_RATE_IMPORT</code> (shared)
Payroll exempt backfill	<code>payrollExemptBackfill.js</code> → <code>runOnce()</code>	Sets <code>employees.salary_exempt=1</code> for the named "always full salary" staff (Parul Goyal, Rajat Sir, Nitin Jain, Ankur Kaplesh, Pooja Kaplesh, D.S Kaplesh, Soma Kaplesh).	<code>ERP_DISABLE_PAYROLL_EXEMPT_BACKFILL</code>
Fire NOC boot backfill	<code>fireNocCron.js</code> → <code>backfillOnceOnBoot()</code>	One-time sync of 100+ legacy Fire-NOC rows stuck at <code>stage=CYCLE_CLOSE</code> / <code>status=active</code> . Guarded by an <code>app_settings</code> flag.	<code>ERP_DISABLE_FIRE_NOC_CRON</code> (shared with the hourly cron)

Note on the shared `ERP_DISABLE_ITEM_RATE_IMPORT` flag: setting it disables **all three** rate-import passes (v1, v2, v3) at once.

Manual / admin-triggered runs

Two of the schedulers expose an admin-only "run now" endpoint so mam can verify delivery without waiting for the cron tick (both require `role==='admin'`):

Endpoint	Fires
POST <code>/api/admin/procsch-reminder/run-now</code>	The procurement 1-day-before reminder scan (<code>procurementReminderCron.runOnce()</code>).
POST <code>/api/admin/cmd-email/send-now</code>	The daily CMD summary email (<code>dailyCmdEmail.runOnce()</code>).

The backup scheduler likewise has manual list / download / trigger endpoints under `/api/admin/backups/*`.

12.3 The `server/scripts/` directory — full inventory

Beyond the schedulers above, `server/scripts/` holds CLI one-shots run manually with `node server/scripts/<file>.js`. They are **not** wired into boot.

Scheduler / boot modules (described above)

`backup-db.js`, `dailyAuditSnapshot.js`, `dprAutoPrompt.js`, `cashFidelityCron.js`, `fireNocCron.js`, `hrAutomationsCron.js`, `procurementReminderCron.js`, `dailyCmdEmail.js`, `installationBillingCron.js`, `itemMasterCleanup.js`, `itemwiseRateImport.js`, `itemwiseRateImportV2.js`, `itemwiseRateImportV3.js`, `payrollExemptBackfill.js`.

Manual data-import / seed scripts

Script	Purpose
<code>import-customers.js</code>	Imports customers from <code>customers-seed.csv</code> into SQLite. Run once after deploy.
<code>import-payment-targets.js</code>	One-shot import of 57 payment-collection target rows (mam's "Payment collection followup" sheet, 28-Apr-2026). Each becomes a receivable.
<code>import-stock-sheet.js</code>	One-shot importer for <code>db/stock_sheet_import.json</code> (588 opening-balance rows: location/item/qty/unit/rate) → seeds inventory.
<code>update-client-addresses.js</code>	One-shot: updates Business Book billing + shipping addresses from <code>ADDRESS.pdf</code> (2026-06-04).
<code>customers-seed.csv</code>	Seed data file consumed by <code>import-customers.js</code> .

Maintenance / recovery scripts

Script	Purpose
<code>reset-admin-password.js</code>	Resets the local admin password (default <code>admin123</code> , or an arg).
<code>regenerate-emergency-code.js</code>	Regenerates the owner emergency recovery code; rewrites <code>data/RECOVERY.txt</code> and the <code>app_settings</code> hash.
<code>restore-business-book.js</code>	Surgical restore of ONE deleted Business Book order and all cascade-deleted children (sites, DPRs + work-items/manpower/machinery, attendance, po_items, planning, finance, POs, receivables, geofence) from a backup DB without overwriting current data.
<code>backfill-audit.js</code>	One-shot: synthesises <code>audit_log CREATE</code> rows for entries created before audit logging worked (the <code>ERP_DISABLE_AUDIT=1</code> bug period).

Diagnostic (read-only) scripts

Script	Purpose
<code>which-order.js</code>	For a sales bill (delivery note), shows which Business Book order it resolves to + that order's Against-Delivery %, via both lookup paths. Changes nothing.
<code>scan-backups-for.js</code>	Scans every nightly backup for a Business Book order (by name keyword) and reports how much of its data survives in each — use when you don't know the deletion date.
<code>audit-po-2026-0222.js</code>	Diagnostic for a specific purchase order (PO 2026-0222).
<code>fix-po-2026-0222.js</code>	Targeted fix for the same purchase order.

Subfolder

`server/scripts/data/` holds supporting JSON / data fixtures used by the import scripts.

12.4 Integrations

Each integration follows the same **graceful-degradation** contract: if its credentials or package are missing, the helper logs a one-line warning and returns a `{ skipped: true }` / no-op result rather than throwing — so a missing integration never blocks the user's real work or crashes the server.

12.4.1 Email — SMTP / Nodemailer

- **File:** `server/lib/email.js` (package: `nodemailer`).
- **Powers:** the daily CMD audit email, HR automation emails (interview / offer / approval nudges), and any alert that calls `sendEmail()`.
- **Credentials:** stored in `app_settings` (Admin → **Email Settings**), **not** in `.env`, so mam can paste Gmail / SendGrid / Mailgun credentials inside the ERP. Keys:
 - `email_smtp_host`, `email_smtp_port` (default 587), `email_smtp_secure` (`1` = 465/TLS, blank = STARTTLS), `email_smtp_user`, `email_smtp_pass` (app password — never logged), `email_from` (display From), `email_director_to` (default recipient; falls back

to `director@securedengineers.com`).

- **Per-rule From override:** callers may pass a `from` , used by email rules (2026-06-03). Note many providers (Gmail) ignore a From that isn't the authenticated account / a verified alias.
- **Graceful degradation:** `sendEmail()` returns `{ skipped: true, reason }` when host/user/pass are unset, or when `nodemailer` can't be required. It never throws. `isConfigured()` lets callers check first.

12.4.2 Web Push — VAPID / web-push

- **Files:** `server/lib/push.js` (helper) + `server/routes/push.js` (routes); package: `web-push` .
- **Powers:** browser/PWA push notifications for delegations, tickets, payments, announcements, procurement reminders, etc.
- **Credentials (VAPID keys): auto-generated on first boot** by `ensureVapid()` and persisted in `app_settings` (`vapid_public_key` / `vapid_private_key`) so pm2 restarts don't invalidate every device's subscription. The VAPID subject defaults to `mailto:admin@securedengineers.com` (overridable via `VAPID_SUBJECT`). `server/index.js` calls `ensureVapid()` on boot.
- **Routes (/api/push):** GET `/vapid` (public key), POST `/subscribe` , POST `/unsubscribe` , GET `/devices` , POST `/test` , POST `/broadcast` (admin-only).
- **Subscription hygiene:** a send that returns 404/410 (expired) auto-sets that subscription's `active=0` .
- **Fan-out helpers:** `pushToUser` , `pushToUsers` , `pushToAll` , plus fire-and-forget wrappers `notify` / `notifyMany` / `notifyAll` (via `setImmediate` , so a push failure can never block or break a route).

12.4.3 SMS & WhatsApp — Twilio

There are **two distinct WhatsApp paths**, which is important to understand:

(a) **Live Twilio (SMS + WhatsApp)** — `server/services/notify.js` (package: `twilio`).

- **Powers:** the "complaint registered" confirmation to the customer (`sendComplaintRegistered`) and ad-hoc one-off notices (`sendText` , e.g. procurement receiving-mismatch alerts). Both fire **WhatsApp + SMS in parallel**.
- **Credentials (ENV / pm2 env):**
 - `TWILIO_ACCOUNT_SID` , `TWILIO_AUTH_TOKEN` (client),
 - `TWILIO_WHATSAPP_FROM` (WhatsApp sender),
 - `TWILIO_SMS_FROM` (SMS sender).
- **Graceful degradation:** the Twilio client is lazy-loaded. If credentials are missing or the `twilio` package isn't installed, it warns **once** and returns `{ skipped: true, reason: 'twilio_unavailable' }` . Per-channel sends that lack a `From` return `{ skipped: true }` . Indian numbers are normalised to E.164 (`toE164` , defaulting to `+91`); invalid numbers skip cleanly. The module **never throws** — callers don't need try/catch.

(b) **WhatsApp click-to-send deep links** — `server/utils/whatsapp.js` (no API, no credentials).

- **Powers:** the Complaints flow's three messages (registration ack to client, assignment notice to engineer, assignment + OTP to client) as `wa.me` click-to-send links that mam (or her EA) clicks to fire from her **own** WhatsApp.
- It also centralises the message templates and a guessing-resistant 4-digit resolution OTP generator (`generateOtp`).
- The file notes that once a Business WhatsApp API is provisioned, callers can swap `whatsappLink()` for an `await sendWhatsapp()` in one place.

12.4.4 AI — Anthropic Claude SDK

- **File:** `server/routes/aiAgent.js` (and OCR/invoice flows); package: `@anthropic-ai/sdk` .
- **Powers:** the in-ERP AI agent and document understanding (e.g. uploading a vendor invoice PDF/image for extraction — the agent supports vision/OCR inputs). Anthropic/Claude imports also appear across procurement, orders, HR, quotations, influencers, procurement-schedule, price-requests, fire-noc, customers and resume parsing flows.
- **Credentials:** stored in `app_settings` (Admin → AI Settings), **not** env:
 - `ai_api_key` (secret; server-side only, **masked** on GET),
 - `ai_provider` (default `anthropic`),
 - `ai_model` (default `claude-opus-4-7`).
- **Behaviour:** the client is built per-call as `new Anthropic.default({ apiKey, timeout })` with a 150s timeout (`ANTHROPIC_TIMEOUT_MS`). Adaptive/extended features are enabled only for models matching `claude-opus-4-[67]|sonnet-4-6` .
- **Graceful degradation:** if `ai_api_key` isn't set, AI endpoints return a "not configured" response; GET `config` reports `{ configured: false }` .

12.4.5 Error monitoring — Sentry

- **File:** `server/lib/sentry.js` (package: `@sentry/node v8`).
- **Powers:** backend error monitoring. It is required as the **very first line** of `server/index.js` so v8 auto-instrumentation can hook Express/route files. `process.on('uncaughtException')` and `unhandledRejection` handlers forward to `sentry.captureException`, and `setupExpressErrorHandler(app)` is wired at the end of the middleware chain.
- **Credentials (ENV / pm2 env):**
 - `SENTRY_DSN` (activates the integration; without it the module is a complete no-op),
 - `SENTRY_TRACES_SAMPLE_RATE` (default 0.1 = 10%),
 - `SENTRY_RELEASE` (optional, e.g. git SHA),
 - `NODE_ENV` (used as the environment tag).
- **Privacy:** a `beforeSend` hook strips `authorization` / `cookie` / `x-auth-token` headers, the request body, and cookies before any event leaves the server — Sentry never sees passwords or JWTs. The DSN secret is masked in the boot log.

12.4.6 File parsing & spreadsheets

Library	Used for
<code>xlsx</code>	Excel import/export — itemwise rate imports, stock-sheet seeding, and various route-level Excel parsing/export.
<code>pdf-parse</code>	Text extraction from PDFs — resume parsing (<code>utils/resumeParser.js</code>) and PDF-sourced imports.
<code>mammoth</code>	DOCX → text extraction for resume parsing.
<code>multer</code>	Multipart file uploads. Disk storage under <code>data/uploads/</code> , 20 MB per-file limit, sanitised filenames. Served read-only at <code>/uploads</code> ; upload via <code>POST /api/upload</code> .

- **Resume parser** (`utils/resumeParser.js`): best-effort extraction of name / email / phone / address / LinkedIn from a candidate's PDF (`pdf-parse`) / DOCX (`mammoth`) / TXT, returning per-field nulls when not confidently detected. Powers HR auto-fill on resume upload.

12.4.7 Other libraries (HTTP / perf)

- `compression` — gzip on all responses (lazy-required; warns if absent, server still boots).
- `cors`, `express`, `better-sqlite3`, `bcryptjs`, `jsonwebtoken`, `dotenv` — core stack (not "integrations" but listed for completeness of the dependency surface).

12.5 Notifications model

The ERP delivers a notification through up to **four** channels, layered so that the loss of any single channel still leaves the user informed.

Layer	Mechanism	Where it lives	Reaches
In-app bell / notifications	Notification rows inserted into the DB (with <code>dedupe_key</code>), shown in the app bell	HR cron, schedulers, route handlers	Logged-in users
Announcements	<code>announcements</code> table + <code>/api/announcements</code> ; pushed for broadcasts	Procurement reminder cron, admin posts	All / targeted users
Web push	<code>server/lib/push.js</code> (<code>notify</code> / <code>notifyMany</code> / <code>notifyAll</code>)	Any route or cron	Subscribed browsers/PWAs
Email	<code>server/lib/email.js</code> (<code>sendEmail</code>)	CMD email, HR cron	SMTP recipients
SMS + WhatsApp	<code>server/services/notify.js</code> (Twilio) and <code>server/utils/whatsapp.js</code> (wa.me links)	Complaints, ad-hoc alerts	Customers / engineers on mobile

Design principles

- **Dedupe everywhere.** The HR cron uses `dedupe_key s ( interview_reminder:<candidate>:<date> , offer_expiry:<candidate> , approval_pending:<request> )`; the procurement reminder uses a dedup table. This makes re-running a cron (e.g. after a boot catch-up) safe — no spam.
- **Fire-and-forget.** Push uses `setImmediate` wrappers that never throw, so a notification failure cannot break the user action that triggered it.

- **Layered fallback.** A single event commonly creates an in-app notification **and** sends an email/push — if SMTP or push isn't configured, the in-app row still records it.
- **Skip-not-fail.** Email, Twilio, push and AI all return a `{ skipped }`-style result when unconfigured, so an under-configured environment (local dev, fresh deploy) keeps functioning.

12.6 Where credentials live — quick reference

Integration	Credential location
SMTP (email)	<code>app_settings</code> via Admin → Email Settings
Web Push (VAPID)	<code>app_settings</code> (auto-generated on first boot); <code>VAPID_SUBJECT</code> optional in env
Twilio (SMS/WhatsApp)	ENV / pm2 env — <code>TWILIO_ACCOUNT_SID</code> , <code>TWILIO_AUTH_TOKEN</code> , <code>TWILIO_WHATSAPP_FROM</code> , <code>TWILIO_SMS_FROM</code>
Anthropic Claude (AI)	<code>app_settings</code> via Admin → AI Settings (<code>ai_api_key</code> , <code>ai_provider</code> , <code>ai_model</code>)
Sentry	ENV / pm2 env — <code>SENTRY_DSN</code> (+ optional <code>SENTRY_TRACES_SAMPLE_RATE</code> , <code>SENTRY_RELEASE</code>)
Audit API token	ENV / pm2 env — <code>AUDIT_API_TOKEN</code> (<code>pm2 set ERP:AUDIT_API_TOKEN <token></code>); <code>/audit</code> returns 503 without it
Cron disable flags	ENV / pm2 env — <code>ERP_DISABLE_*</code> (see §12.2)

Rule of thumb: mam-managed, rotatable credentials (SMTP, AI key) live in the ERP's own Admin settings (`app_settings`) so they can be changed without a redeploy. Infrastructure-level credentials (Twilio, Sentry, audit token) and all cron toggles live in the server environment / `pm2` config.

13. Technical Reference

This section is the engineering reference for the SEPL Business ERP. It documents the runtime architecture, technology stack, repository layout, the persisted data model, how to build and run the app locally, how it is deployed to the Hostinger VPS, and the security model. Every fact below was verified against the source (package manifests, `server/index.js`, `server/db/schema.js`, the deployment scripts, and the React entry points) — nothing here is aspirational.

13.1 High-Level Architecture

The ERP is a classic three-tier application packaged as a **single Node process that serves both the API and the compiled front-end**. There is no separate application server, no container orchestration, and no external database server — the entire datastore is a single SQLite file on disk.



Key architectural properties:

- **Client SPA $\rightleftharpoons$ Express API $\rightleftharpoons$ better-sqlite3 file DB.** The React single-page app talks only to `/api/*` (and the special `/audit` endpoint) over JSON. The Express layer is the only thing that touches the database.
- **One process, two responsibilities.** In production the same Express process both answers API calls and serves the Vite build from `client/dist`. If the build is missing it falls back to "API only mode".
- **Synchronous database access.** `better-sqlite3` is a synchronous, in-process driver. There is no connection pool and no network round-trip to the DB; a query is a direct function call against the embedded SQLite engine. This keeps the code simple (no async DB plumbing) at the cost of being single-machine.
- **In-process schedulers.** All background jobs (nightly backup, audit snapshot, DPR prompts, cash-fidelity rollovers, HR automations, procurement reminders, the CMD email, fortnightly installation billing, Fire-NOC autopilot) are cron-style timers started inside `server/index.js` on boot. There is no separate worker process — PM2 keeps the single process alive and the timers run within it.
- **PWA front-end.** The client ships a service worker and a web-app manifest so it installs to a phone home screen and can receive Web Push notifications even when the tab is closed.

Request lifecycle

1. Browser sends `GET/POST /api/ ...` with an `Authorization: Bearer <JWT>` header.
2. Express applies CORS, gzip compression, a 10 MB JSON body parser, and static cache headers.
3. The audit middleware registers a response-finish hook (fire-and-forget) so any mutating request is logged after it completes.

4. The matching router runs its own `authMiddleware`, which verifies the JWT and sets `req.user`. Sliding-session logic may attach a fresh token via the `X-Refresh-Token` response header.
5. The handler reads/writes `data/erp.db` synchronously through `better-sqlite3` and returns JSON.
6. A global Express error handler catches any thrown error (including synchronous SQLite throws) and returns a structured JSON error body instead of HTML.

13.2 Technology Stack

Layer	Technology	Version (declared)	Notes
Runtime	Node.js	$\geq 18.0.0$ (VPS installs Node 20)	Single process, <code>engines</code> enforced in root <code>package.json</code> .
Web framework	Express	<code>^4.19.2</code>	Route routers mounted under <code>/api/*</code> ; one special <code>/audit</code> mount.
Database	SQLite via better-sqlite3	<code>^11.0.0</code>	Synchronous, embedded; file at <code>data/erp.db</code> .
Auth	jsonwebtoken + bcryptjs	<code>^9.0.2</code> / <code>^2.4.3</code>	JWT bearer tokens (7-day sliding); bcrypt password hashing (cost 10).
Compression	compression (gzip)	<code>^1.7.4</code>	Enabled globally; 60–80% smaller JSON.
CORS	cors	<code>^2.8.5</code>	Open in dev (<code>*</code>), locked (<code>origin:false</code>) in production behind Nginx.
File uploads	multer	<code>^1.4.5-lts.1</code>	Disk storage to <code>data/uploads</code> , 20 MB limit.
Error monitoring	@sentry/node + @sentry/react	<code>^8.45.0</code>	Activated only when <code>SENTRY_DSN</code> is set; strips auth/cookies/body before send.
Web Push	web-push	<code>^3.6.7</code>	VAPID keys auto-generated and persisted in <code>app_settings</code> .
Email	nodemailer	<code>^8.0.7</code>	SMTP notifications (HR automations, CMD daily email).
WhatsApp / SMS	twilio	<code>^6.0.2</code>	Optional complaint-registration notifications (disabled if creds blank).
Document parsing	mammoth, pdf-parse, xlsx	—	Resume/BOQ/Excel ingestion and CSV/XLSX import-export.
AI	@anthropic-ai/sdk	<code>^0.40.1</code>	Backs the <code>/api/ai-agent</code> route.
Env config	dotenv	<code>^16.4.5</code>	Loads <code>.env</code> at boot.
Dev orchestration	concurrently	<code>^8.2.2</code>	Runs server + client together via <code>npm run dev</code> .

Front-end stack

Layer	Technology	Version (declared)	Notes
UI library	React + React DOM	<code>^19.2.4</code>	Function components + hooks.
Build tool	Vite	<code>^5.4.21</code>	Dev server on port 3000, proxies <code>/api</code> → <code>localhost:5000</code> .
Router	react-router-dom	<code>^7.14.0</code>	<code>ProtectedRoute</code> / <code>AdminRoute</code> / <code>ModuleRoute</code> guards.
Styling	Tailwind CSS + <code>@tailwindcss/forms</code>	<code>^3.4.19</code>	PostCSS + autoprefixer pipeline.
HTTP client	axios	<code>^1.15.0</code>	Single wrapper in <code>client/src/api.js</code> with auth + refresh interceptors.
Charts	recharts	<code>^3.8.1</code>	Dashboard analytics.
Maps	leaflet	<code>^1.9.4</code>	Geofence / attendance location views.
QR scanning	html5-qrcode	<code>^2.3.8</code>	Asset / tool scanning.
Toasts	react-hot-toast	<code>^2.6.0</code>	In-app notifications.
Icons	react-icons	<code>^5.6.0</code>	—
Error monitoring	@sentry/react	<code>^8.45.0</code>	Front-end crash reporting.
Linting	eslint (+ react-hooks / react-refresh plugins)	<code>^9.39.4</code>	<code>npm run lint</code> in <code>client/</code> .

A `__BUILD_STAMP__` constant is injected at build time by `vite.config.js` (`MM-DD HH:MM`) so a header badge reveals exactly which bundle a given phone is running — this exists to defeat stale-PWA-cache confusion.

13.3 Repository Layout

```
business-erp/
├─ package.json           # Root: backend deps + npm scripts (dev/server/build)
├─ package-lock.json
├─ render.yaml            # Render.com blueprint (alternative host)
├─ deploy-vps.sh          # One-shot Hostinger VPS provisioning script
├─ health-check.sh        # Post-deploy endpoint smoke test
├─ .env.example           # Sample environment file
├─ .gitignore             # Ignores node_modules, /data, .env, client/dist
├─ ERP-AUTOMATION-AUDIT.md # Project docs / audit notes (Markdown)
├─ INDENT-TO-DISPATCH*.md # Procurement process docs
├─ SALES-BILLING-MODULE.md # Sales billing design doc
├─ data/                  # (gitignored) runtime: erp.db, uploads, snapshots
├─ backups/               # (gitignored) DB backups on Windows dev box
├─ docs/sections/         # This documentation set
├─
├─ server/                # — Express backend —————
│  ├─ index.js            # App entry: middleware, route mounts, cron, static
│  ├─ erp.db              # (stray dev copy; canonical DB lives in data/)
│  └─ db/
│     ├─ schema.js        # ~5,300-line schema: all CREATE TABLE + migrations
│     ├─ fireNocSchema.js  # Fire-NOC tables (split out)
│     ├─ rentalToolsSchema.js # Rental-tools tables (split out)
│     ├─ nextSequence.js   # Document-number sequence generator
│     ├─ seedScoring.js    # Seeds 20 MIS scorecard templates
│     ├─ import-bb.js      # Business Book CSV importer
│     ├─ import-vendors.js # Vendor importer
│     ├─ items_seed.json   # Item-master seed data
│     ├─ vendors_import.json # Vendor seed data
│     ├─ bb_import.json    # Business Book seed data
│     ├─ labourRatesSeed.json # Labour-rate seed data
│     ├─ poFocSeed.json    # PO free-of-cost seed data
│     └─ stock_sheet_import.json # Opening-stock seed data
└─ routes/                # ~56 Express routers (one per domain area)
```

```

├── middleware/
│   ├── auth.js           # JWT verify, adminOnly, requirePermission factory
│   └── audit.js          # Mutating-request audit logger
├── lib/                  # Cross-cutting helpers
│   ├── sentry.js         # Backend Sentry init (no-op without DSN)
│   ├── push.js           # Web Push / VAPID management
│   ├── email.js          # Nodemailer transport
│   ├── emailRules.js     # Rule-based email routing
│   ├── emailEvents.js
│   ├── businessHours.js  # Working-day / cutoff helpers
│   ├── cashSync.js       # Cash-flow recompute helpers
│   └── fireNocSync.js
├── services/
│   └── notify.js         # Unified in-app/push/WhatsApp notification fan-out
├── utils/
│   ├── cmdDashboard.js   # CMD audit dashboard aggregation
│   ├── duplicateGuard.js # Idempotency / dedup helpers
│   ├── resumeParser.js   # CV parsing (mammoth/pdf-parse)
│   ├── validate.js       # Shared input validation
│   └── whatsapp.js       # Twilio WhatsApp wrapper
├── scripts/              # ~28 cron jobs + one-shot migrations/backfills
├── client/               # — React front-end —
│   ├── package.json      # Front-end deps + scripts (dev/build/lint/preview)
│   ├── vite.config.js    # Vite config + build stamp + /api dev proxy
│   ├── tailwind.config.js
│   ├── postcss.config.js
│   ├── eslint.config.js
│   ├── index.html        # SPA shell
│   ├── dist/             # (gitignored) production build output
│   ├── public/
│   │   ├── manifest.json # PWA manifest (standalone, theme color)
│   │   ├── sw.js         # Service worker (push receive + click routing)
│   │   ├── icon.svg / icons.svg / favicon.svg
│   │   └── sepl-logo.webp
│   └── src/
│       ├── main.jsx       # React root / providers mount
│       ├── App.jsx        # Router + route guards (Protected/Admin/Module)
│       ├── api.js         # axios instance + auth + token-refresh interceptors
│       ├── sentry.js      # Front-end Sentry init
│       ├── index.css      # Tailwind entry
│       ├── context/
│       │   └── AuthContext.jsx # Auth state, permissions, live-refresh on focus
│       ├── pages/         # ~72 page-level screens (one per module/view)
│       ├── components/    # ~25 shared UI components
│       ├── hooks/
│       │   ├── useDraggableFab.js
│       │   └── useUrlTab.js
│       ├── lib/
│       │   └── push.js     # Client push-subscription helper
│       ├── utils/
│       │   ├── compressImage.js
│       │   ├── dateIST.js  # IST date formatting
│       │   ├── exportCsv.js
│       │   └── numberToWords.js # Amount-in-words for printed documents
│       ├── assets/
│       └── data/

```

13.4 Data Model

All persistent state lives in a single SQLite database at `data/erp.db` (`DB_PATH = path.join(__dirname, '..', '..', 'data', 'erp.db')`). The schema is created and migrated by `server/db/schema.js` on every boot — `CREATE TABLE IF NOT EXISTS` statements are idempotent, and the file also contains numerous guarded `ALTER/rebuild` migrations (the `*_new` tables seen below are intermediate rebuild targets used by migrations, not steady-state tables). Two domains have their schema split into separate files: Fire-NOC (`fireNocSchema.js`) and rental-tools (`rentalToolsSchema.js`).

The tables below are grouped by business domain. Names are taken verbatim from the `CREATE TABLE` statements in `schema.js`.

Identity, Access & Audit

Table	Purpose
users	Application user accounts (bcrypt password hash, role, department).
roles	Named roles.
role_permissions	Per-module CRUD/approve/see-all flags for each role.
user_roles	Many-to-many user↔role assignment.
delegations	Temporary authority hand-offs between users.
audit_log	Append-only record of every mutating API request.
activity_log	General activity/event trail.
app_settings	Key/value app config (VAPID keys, migration flags, SMTP).
notifications	In-app notification feed.
push_subscriptions	Web Push endpoints per device.
announcements / announcement_reads	Broadcast messages + read receipts.

CRM — Leads, Funnel & Quotations

Table	Purpose
leads	Inbound enquiries / prospects.
lead_sources	Lookup of lead origin channels.
lead_followups	Scheduled follow-up actions on leads.
meetings	Logged client meetings.
sales_funnel	Opportunity pipeline rows with stage/value.
sales_funnel_audit	Stage-change history for the funnel.
sales_funnel_boqs	BOQs attached to funnel opportunities.
crm_funnel	CRM-side funnel view.
boq / boq_items	Bill-of-Quantities header and line items.
quotations	Customer quotations.
estimate_quotations	Internal cost estimates feeding quotations.
support_tickets / support_tickets_new	Support/enquiry tickets (rebuild target).

Orders & Business Book (Projects)

Table	Purpose
business_book	Master record of awarded projects/orders (client, PO, value).
order_planning	Planning data per order.
customers	Client master.
proj_projects	Project register for the Projects module.
proj_budgets	Per-project budget lines.
proj_work_orders	Work orders issued under a project.
proj_salary_entries	Project-charged salary entries.
proj_daily_wage_entries	Daily-wage labour entries.
proj_muster_roll	Attendance muster roll for project labour.
proj_mb_sheets / proj_mb_lines	Measurement-book sheets and lines.
proj_contractor_ra_bills / proj_contractor_ra_deductions	Contractor RA bills + deductions.
proj_client_ra_bills / proj_client_ra_deductions	Client RA bills + deductions.
project_finance	Project-level finance summary.

Procurement — Indents, Vendors, POs, GRN

Table	Purpose
indents / indents_new	Material indents (rebuild target).
indent_items	Indent line items.
indent_item_rates	Quoted rates per indent item.
indent_tracker	Indent lifecycle tracking.
vendors	Vendor/supplier master.
vendor_rates	Vendor price lists.
vendor_pos / vendor_po_items	Vendor purchase orders + lines.
purchase_orders / po_items	Purchase order header + lines.
po_foc_entries	Free-of-cost PO entries.
purchase_bills	Vendor purchase bills.
grn / grn_items	Goods-Receipt-Note header + lines.
procurement_schedule (built via schema/migrations)	Procurement timeline; drives 1-day-before reminders.
labour_payment_indents	Labour-payment indents raised against a site/sub-contractor.

Items, Inventory & Stock

Table	Purpose
item_master	Item/SKU catalogue with current price.
item_price_history	Historical price changes per item.
pipe_weights	Editable pipe weight master (MTR↔KG conversion).
price_requests	Requests for a price/rate to be filled.
warehouses	Stock locations.
stock_balance	Current on-hand quantity per item/warehouse.
stock_movements	Stock in/out ledger.
stock_issue_notes	Material issue notes.
tools / tool_movements	Tool register and check-in/out movements.
tools_list_submissions	Submitted site tool lists.
company_assets / company_asset_movements	Fixed-asset register + transfers.
labour_rates	Labour rate master (SITC-derived).

Sales Billing, Delivery & Receivables

Table	Purpose
sales_bills / sales_bill_items	Customer sales bills (4 sequential types) + lines.
sales_bill_status_log	Status transitions for sales bills.
debit_notes	Debit notes against bills.
delivery_notes	Goods delivery/dispatch notes.
installations	Installation records.
installation_bills	Installation-stage bills.
ra_bills / mb_bills	Running-account and measurement-book bills.
testing_commissioning	T&C records.
handover_certificates	Project handover certificates.
receivables	Outstanding receivable ledger (with ageing).
collections	Cash/cheque collection records.
collection_follow_ups	Follow-up actions on overdue receivables.
cheques / cheque_actions	Cheque register + lifecycle actions.
payments	Payment records.

Finance — Cash Flow, Payments & Statutory

Table	Purpose
cash_flow_daily	Daily cash position / runway rollover.
cash_flow_entries	Individual cash-flow line items.
payment_requests / payment_requests_new	Outbound payment requests (rebuild target).
payment_approvals	L1→L2→L3→Release approval trail for payment requests.
expenses	Expense claims/records.
statutory_dues_calendar	Statutory due-date calendar (GST/TDS etc.).

HR, Recruitment & Payroll

Table	Purpose
employees	Employee master (salary, exemption flags).
sub_contractors	Sub-contractor master.
candidates	Recruitment candidates.
candidate_docs / candidate_events	Candidate documents and event timeline.
hiring_requests	Headcount/hiring requisitions.
jd_templates / job_descriptions	JD templates and concrete JDs.
screening_questions / screening_answers	Screening Q&A.
interview_scorecards	Interview evaluation scorecards.
final_round_questions	Final-round interview questions.
induction_items	New-joiner induction checklist.
training_videos / training_assignments	Training content + assignments.
attendance / attendance_new	Daily attendance (rebuild target).
geofence_settings	Geofence radius/config for attendance.
location_tracking	Field-staff GPS pings.
leave_requests / leave_requests_new	Leave applications (rebuild target).
payroll_settings	Payroll rules/config.
payroll_runs	Monthly payroll runs.
payroll_advances	Salary advances.
manpower_required_overrides	Manual overrides to required-manpower slab.
manpower_project_settings	Per-project manpower assumptions.

Site Execution — DPR & Sites

Table	Purpose
sites	Active project sites.
dpr	Daily Progress Report header.
dpr_work_items	Work items completed per DPR.
dpr_manpower	Manpower deployed per DPR.
dpr_material	Material consumed per DPR.
dpr_machinery	Machinery used per DPR.
dpr_contractors	Sub-contractor lines per DPR.

Performance, Complaints, Rentals & Misc

Table	Purpose
<code>score_templates / score_kpis</code>	MIS scorecard templates and their KPIs.
<code>score_user_template / score_user_kpi_target</code>	Per-user scorecard assignment + targets.
<code>score_entries</code>	Recorded KPI scores.
<code>complaints</code>	Customer complaints (with WhatsApp/SMS on register).
<code>snags</code>	Snag-list defects.
<code>checklists / checklist_completions</code>	Checklist definitions + completions.
<code>pms_tasks</code>	Project-management tasks.
<code>rental_properties / rental_rooms / rental_bookings</code>	Rental accommodation register.
<code>rent_requests / rental_payments</code>	Rent requests and payments.
<code>email_rules</code>	Configurable email routing rules.

Migration scaffolding note: tables ending in `_new` (`indents_new` , `attendance_new` , `leave_requests_new` , `support_tickets_new` , `payment_requests_new`) are temporary rebuild targets created inside guarded migrations in `schema.js` — they are renamed into place, not kept as independent tables.

Seed & import data

On first boot the schema/seed code loads JSON fixtures from `server/db/` :

File	Loaded into
<code>items_seed.json</code>	<code>item_master</code> opening catalogue.
<code>vendors_import.json</code>	<code>vendors</code> master.
<code>bb_import.json</code>	<code>business_book</code> projects.
<code>labourRatesSeed.json</code>	<code>labour_rates</code> .
<code>poFocSeed.json</code>	<code>po_foc_entries</code> .
<code>stock_sheet_import.json</code>	opening stock balances.
<code>seedScoring.js</code>	20 MIS scorecard templates (<code>score_templates / score_kpis</code>).

All seeders are idempotent: re-running when rows already exist is a no-op.

13.5 Build & Run

All scripts are declared in the `root package.json` unless noted.

Command	What it does
<code>npm run dev</code>	Runs backend + front-end together via <code>concurrently</code> (server on :5000, Vite on :3000). The recommended local-dev command.
<code>npm run server</code>	Backend only — <code>node server/index.js</code> (also the <code>start</code> script).
<code>npm run client</code>	Front-end only — <code>cd client && npm run dev</code> (Vite dev server).
<code>npm run build</code>	Builds the front-end: <code>cd client && npm install && npm run build</code> → emits <code>client/dist</code> .
<code>npm run install-all</code>	Installs root + client dependencies.
<code>npm start</code>	Production start — <code>node server/index.js</code> .
<code>postinstall</code>	Automatically runs <code>npm run build</code> after <code>npm install</code> so a fresh deploy always has a current bundle.

Inside `client/` (`client/package.json`):

Command	What it does
<code>npm run dev</code>	Vite dev server (port 3000, proxies <code>/api</code> → <code>localhost:5000</code>).
<code>npm run build</code>	Production Vite build into <code>client/dist</code> .
<code>npm run preview</code>	Serves the built bundle locally.
<code>npm run lint</code>	ESLint over the client source.

Local development flow

1. `npm run install-all` (first time only).
2. Copy `.env.example` → `.env` and set at minimum `PORT`, `JWT_SECRET`, `NODE_ENV=development`.
3. `npm run dev`.
4. Open `http://localhost:3000` (Vite proxies API calls to the backend on `:5000`).

In **development** the backend serves API only and Vite serves the SPA with its proxy. In **production** the backend serves the SPA from `client/dist` itself and Vite is not involved. The DB file and `data/uploads` are created automatically on first boot if absent. Several dev-only `ERP_DISABLE_*` env flags turn off the in-process schedulers (see operations below).

13.6 Deployment & Operations

Hosting

Production runs on a **Hostinger VPS** behind **Nginx** at `securederp.in`, with the Node process supervised by **PM2** under the name `erp`. A `render.yaml` blueprint also exists as an alternative one-click Render.com deployment, but the VPS is the live environment.

Initial provisioning (`deploy-vps.sh`)

`deploy-vps.sh` is the one-shot bootstrap run as root on a fresh VPS. It:

1. Updates the OS and installs **Node.js 20**, build tools (`build-essential`, `python3`, `git`) and **Nginx**.
2. Installs **PM2** globally.
3. Clones the repo to `/root/erp`, runs `npm install`, then builds the front-end (`cd client && npm install && npm run build`).
4. Writes a production `.env` (`PORT=5000`, `JWT_SECRET`, `NODE_ENV=production`).
5. Starts the app with `pm2 start server/index.js --name erp`, then `pm2 save` and `pm2 startup` so it survives reboots.
6. Configures Nginx as a reverse proxy for `securederp.in` → `http://localhost:5000` with a 20 MB `client_max_body_size`.
7. Provisions free HTTPS via **Let's Encrypt / certbot** with auto-renewal.

Routine deploy flow

For day-to-day releases the flow is **push to main** → **SSH to the VPS** → **pull, rebuild, restart**:

```
# locally
git push origin main

# on the VPS
cd /root/erp
git pull
npm install          # postinstall rebuilds client/dist
pm2 restart erp
```

Because `client/dist` is gitignored, the front-end is rebuilt on the server (the `postinstall` hook handles this). Content-hashed asset filenames plus the no-cache headers on `index.html` (set in `server/index.js`) ensure browsers and the PWA pick up the new bundle on next load rather than serving a stale cache.

PM2

The single process is supervised by PM2 (`erp`). PM2 restarts the process on crash; `uncaughtException` / `unhandledRejection` handlers in `index.js` forward the stack trace to Sentry (and the console) before PM2 cycles the process so crashes are not lost silently. Logs are viewable with `pm2 logs erp`. Some per-environment secrets are set directly in PM2 (e.g. `pm2 set ERP:AUDIT_API_TOKEN <token>`).

Background jobs / schedulers

All of the following start inside `server/index.js` on boot and run in-process (each can be disabled with the noted env flag for dev):

Job	Schedule	Disable flag
Nightly DB backup (keeps last 30)	02:00 daily	<code>ERP_DISABLE_BACKUP_SCHEDULER</code>
Audit JSON snapshot	07:30 daily	<code>ERP_DISABLE_AUDIT_SNAPSHOT</code>
DPR auto-prompt to site engineers	18:00 daily (Sun off)	<code>ERP_DISABLE_DPR_PROMPT</code>
Cash-fidelity rollover + ageing recompute	00:00 / 01:00 daily	<code>ERP_DISABLE_CASH_CRON</code>
Fire-NOC autopilot	hourly (+ boot backfill)	<code>ERP_DISABLE_FIRE_NOC_CRON</code>
HR automations (reminders/offers/approvals)	every 30 min	<code>ERP_DISABLE_HR_CRON</code>
Procurement 1-day-before reminder	weekdays 09:00	<code>ERP_DISABLE_PROCSCH_REMINDER</code>
Daily CMD audit email	09:00 daily (Sun off)	<code>ERP_DISABLE_CMD_EMAIL</code>
Fortnightly installation billing (Type-3)	1st & 16th	<code>ERP_DISABLE_INSTALL_BILLING</code>

Backups are written to `~/erp-backups` on the VPS (or `../backups` on Windows) and are also listable/downloadable/triggerable by an admin via `/api/admin/backups/*`.

Health check & monitoring

- `health-check.sh` logs in with an admin account, obtains a JWT, then hits a fixed list of monitored endpoints, printing a pass/fail per route (treating `200` and `403` as healthy). Run it after a deploy or whenever someone reports an outage: `bash /root/erp/health-check.sh`.
- `GET /api/health` returns `{status:'ok', timestamp}` for uptime monitors.
- Sentry** captures backend and front-end exceptions when `SENTRY_DSN` is configured; it scrubs auth headers, cookies and request bodies before sending.
- The `/audit` endpoint (note: not under `/api`) exposes KPI tiles and exception lists for an external scheduler; it requires the `AUDIT_API_TOKEN` bearer token (or `?token=`) and returns `503` if the token is unset.

Environment variables

From `.env.example` plus variables consumed in code (`server/index.js`, `lib/sentry.js`, `lib/push.js`, `middleware/auth.js`, `routes/auditReport.js`):

Variable	Configures
<code>PORT</code>	HTTP listen port (default <code>5000</code>).
<code>NODE_ENV</code>	<code>production</code> locks CORS and enables static SPA serving.
<code>JWT_SECRET</code>	Signing secret for JWT auth tokens. Must be changed from the sample.
<code>AUDIT_API_TOKEN</code>	Bearer token guarding the <code>/audit</code> reporting endpoint.
<code>SENTRY_DSN</code>	Enables Sentry error monitoring (no-op if unset).
<code>SENTRY_TRACES_SAMPLE_RATE</code>	Sentry trace sampling (default <code>0.1</code>).
<code>SENTRY_RELEASE</code>	Optional release tag (e.g. git SHA).
<code>VAPID_SUBJECT</code>	Contact <code>mailto:</code> for Web Push (default <code>admin@securedengineers.com</code>). VAPID key pair itself is auto-generated and stored in <code>app_settings</code> .
<code>TWILIO_ACCOUNT_SID</code>	Twilio account SID for WhatsApp/SMS (blank = channel off).
<code>TWILIO_AUTH_TOKEN</code>	Twilio auth token.
<code>TWILIO_WHATSAPP_FROM</code>	WhatsApp sender (E.164, no <code>whatsapp:</code> prefix).
<code>TWILIO_SMS_FROM</code>	SMS sender number (E.164).
<code>ERP_DISABLE_*</code>	Per-scheduler kill switches (see job table above).

`.gitignore` keeps `node_modules/`, `/data/` (the live DB + uploads), `.env`, `client/dist/` and `.claude/worktrees/` out of version control.

13.7 Front-End Application Notes

- **api wrapper** (`client/src/api.js`): a single axios instance with `baseUrl: '/api'`. A request interceptor attaches the JWT from `localStorage`. A response interceptor (a) swaps in a fresh token whenever the server returns an `X-Refresh-Token` header (sliding session), and (b) on `401` clears the token and redirects to `/login`.
- **AuthContext** (`client/src/context/AuthContext.jsx`): holds `user`, `permissions`, `userRoles`, and `token`. On mount it calls `/auth/me` to hydrate state. It **live-refreshes permissions** by re-pulling `/auth/me` when the tab regains focus and every two minutes while active (debounced), so an admin's permission grant takes effect without a re-login. Background refresh failures never trigger an auto-logout.
- **Route guards** (`client/src/App.jsx`):
 - `ProtectedRoute` — requires a logged-in user, else redirects to `/login`.
 - `AdminRoute` — requires `isAdmin()`, else redirects to `/`.
 - `ModuleRoute` `module="..."` — requires `canView(module)`, else renders an in-place "Access Denied" panel.
- **PWA**: `public/manifest.json` declares a standalone installable app (SEPL Business ERP, theme `#1e40af`). `public/sw.js` is the service worker — it claims clients on activate, receives `push` events (showing a notification even when the tab is closed), and on notification click focuses an existing ERP tab or opens the deep link. `client/src/lib/push.js` manages the browser push subscription against the server's VAPID public key.

13.8 Security Notes

- **Authentication — JWT bearer tokens**. Login (`POST /api/auth/login`) returns a 7-day JWT signed with `JWT_SECRET` containing `{id, email, role, name}`. `authMiddleware` verifies the token on every protected route and sets `req.user`. A **sliding session** re-issues a fresh token (via the `X-Refresh-Token` response header) once the current token is more than a day old, so an active user is never logged out mid-session, while a fully idle session still expires at 7 days.
- **Password storage — bcrypt**. Passwords are hashed with `bcryptjs` at cost 10 (`bcrypt.hashSync(password, 10)`) and verified with `bcrypt.compareSync`. Plain passwords are never stored.
- **Authorization — role-based permission gating**. Access is enforced server-side via `requirePermission(module, action)` which joins `user_roles` → `role_permissions` and checks the relevant `can_view/can_create/can_edit/can_delete/can_approve` flag. The `admin` role bypasses all checks (`adminOnly` is also available for admin-only routes). The front-end mirrors this with `ModuleRoute`, but enforcement is authoritative on the backend — the client gate is UX only.
- **Audit logging**. `auditMiddleware` records every mutating request (`POST/PUT/PATCH/DELETE`) into `audit_log` after the response finishes, fire-and-forget so it never adds latency. It **redacts secret fields** (`password`, `current_password`, `new_password`, `token`, `authorization`, `secret`) from the logged body, auto-derives the action from the HTTP method, and skips high-frequency noise paths (location pings, dashboard polls).
- **CORS**. Open (`*`) in development for local tooling; in production `origin:false` because the SPA is same-origin (served by the same process behind Nginx), so cross-origin browser requests are rejected.
- **Transport security**. Nginx terminates TLS with a Let's Encrypt certificate and force-redirects HTTP→HTTPS; certbot handles auto-renewal.
- **Error hygiene**. The global Express error handler returns a JSON error body (never raw HTML/stack traces to the client). Sentry's `beforeSend` strips auth headers, cookies and request bodies so credentials never leave the server.
- **Upload limits**. Multer caps uploads at 20 MB and sanitises filenames (`[^a-zA-Z0-9--]` → `_`); the JSON body parser is capped at 10 MB.
- **Public surface**. Two endpoints are intentionally unauthenticated: `/api/public/*` (candidate offer accept/decline by token) and `/api/health`. Everything else under `/api` requires a valid JWT; `/audit` requires the separate `AUDIT_API_TOKEN`.

Hardening note for operators: the sample `JWT_SECRET` in `.env.example`, `render.yaml`, and `deploy-vps.sh` is a placeholder. Set a strong, unique `JWT_SECRET` (and a real `AUDIT_API_TOKEN`) per environment — the auth middleware falls back to a hard-coded default secret only if the env var is entirely absent, which must never happen in production.